Operational Proto-Introspection in Looped Language Models

Process-Quality Taps, Executable Branching, and the Readout–Control Boundary

Jan Kirin

github.com/VykosMolt | github.com/VykosMolt/Branching-Looped-Transformer | ORCID 0009-0005-2681-9027

The erratum to arXiv:2604.09870 (v2) remains the correction of record for the prior paper. This manuscript reports all current load-bearing values under the corrected evaluation protocol, states that protocol in full, and summarizes the two failure mechanisms that motivated it.

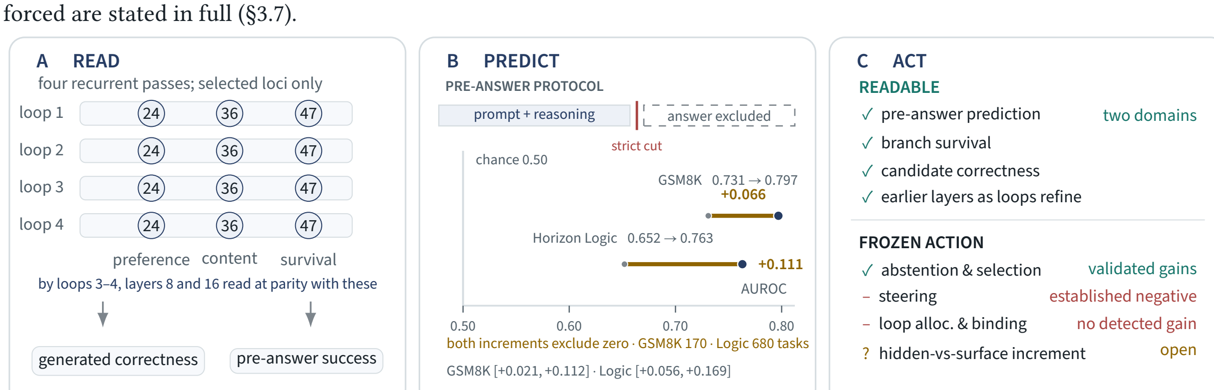
Acknowledgments. I sincerely thank Jonathan Williams for providing the Ouro-RLTT weights used as the primary experimental backbone in this work; without them, the RLTT results reported here would not exist. I owe Section 4.4 to Associate Professor Goran Đambić, PhD, of Algebra Bernays University, who asked whether the earlier layers might already be readable once the model had been round the loop a few times. I had not thought to check. They are.

Abstract

Can a language model read the quality of its own ongoing computation, and can an external intervention turn that readout into better outcomes? We test both questions in a frozen 2.6B looped transformer, Ouro-RLTT. The first answer is positive, on two domains. On GSM8K, a strict pre-answer probe — cut in code to exclude the answer region and the gold value — predicts eventual success at AUROC 0.797 with hidden plus length/log-probability shortcut features, versus 0.731 for shortcuts alone (+0.066; task-clustered 95% CI [+0.021, +0.112]; 170 tasks). On Horizon Logic, now powered by a prospective task-disjoint extension, the increment is +0.111 (CI [+0.056, +0.169]) — independently replicated on the new cohort (+0.095) and robust to an adversarial malformed-sibling shortcut. Low-capacity taps also read role-specialized properties from frozen trajectories — task-disjoint branch survival at 0.9697 oracle retention with the layer-47 channel causally load-bearing, content ranking at 0.6310 macro top-1, generated-branch correctness at AUROC 0.7755 — and recurrence moves *where* candidate quality can be read: physical layers 8 and 16 read only weakly on the first pass (0.410, 0.440) and are statistically tied with the established late-layer basis by loops 3–4 (0.633, 0.628); the trend replicates across the Ouro family and, qualitatively, in out-of-family Huginn (no comparable first-pass rotation detected). A non-looped control replicates a same-class readout; recurrence is not required for every readable signal.

The second answer divides at a line we locate. *Decision-level* uses of the readout convert into validated gains: hidden-state scores beat shortcut-only scores in all four sealed arms, and forced selection beats an exact matched-random null on 39 informative groups (34/39 vs 23.0/39) and on an all-well-formed pool (27/32 vs 64.8% expected; $p = 0.0086$) — content-sensitive selection established; the hidden-vs-surface increment stays open. *Generative control* does not: steering is an established negative, the four-task branch screen is bounded, and loop allocation (tap and native gate alike) and minimal LoRA direction-binding detect no gain. All of it runs through bit-exact branch/carry/prune machinery over Ouro's 192-slot recurrent cache; the negatives are not plumbing.

We call this decision-usable but not generatively controllable property **operational proto-introspection**. The model is not consulting our probes: we read its trajectories, and what we read governs whether and what to answer, not yet how it computes. All load-bearing values use source-item-disjoint splits and antisymmetrized pairwise evaluation; the protocol and every correction it forced are stated in full (§3.7).



Visual summary. Read, predict, act. Recurrent states carry readable process-quality signals — at earlier physical depth as loops refine, and strictly pre-answer on two domains; the readout converts to decision-level gains, not to generative control.

Results at a glance

A model that could read the quality of its own ongoing computation could, in principle, act on it — branching where it is uncertain, pruning what is failing, committing where it is confident. We test both halves in a frozen 2.6B looped transformer (Ouro-RLTT): clear evidence for the first; for the second, validated gains where the readout makes decisions, and none where it must steer the computation.

Readable. Before the model produces an answer, its intermediate states predict whether it will be correct. On GSM8K, a strict pre-answer probe — cut in code to exclude the answer region and the gold value — adds statistically significant information beyond length and log-probability shortcuts: AUROC 0.797 with hidden features versus 0.731 without (+0.066, 95% CI [+0.021, +0.112], paired task-clustered bootstrap over 170 tasks; no single task drives the effect). The same nested comparison on Horizon Logic is now powered: hidden features add **+0.111** AUROC (95% CI [+0.056, +0.169]; 84 held-out negatives), independently replicated on a prospective task-disjoint cohort (+0.095) and robust to an adversarial malformed-sibling shortcut. The signal decomposes into role-specialized readouts recovered by low-capacity taps: branch survivability at **0.9697** oracle retention task-disjoint (layer-47 channel causally load-bearing), content ranking at **0.6310**, and generated-branch correctness at AUROC **0.7755**.

Readable earlier, after refinement. Recurrence does not create the readability, but it moves it. A matched tap reads candidate quality at physical layers 8 and 16 only weakly on the first pass (macro top-1 **0.410** and **0.440**) and at **0.633** and **0.628** by loops 3–4 — statistically tied with the established late-layer basis (final-loop refits 0.618–0.647); the loop trend is significant at every tested layer. A tap trained at loop 2 or 3 and transplanted **frozen** to a later loop keeps essentially all locally refit performance (Spearman 0.90–0.99); one trained on the first pass loses 0.09–0.23. Coordinates rotate sharply at L1→L2 and are comparatively stable afterwards. The pattern replicates across the Ouro family and, qualitatively, in out-of-family Huginn — the depth trend; no comparable rotation detected (Figure 3). These are readout loci, not steering loci; no intervention was performed at them.

Actionable for decisions, not for generation. We build the machinery through which such a signal could be acted on: branch-specific KV-cache carry across Ouro's 192-slot recurrent cache, validated by a six-level correctness ladder, plus a **bit-exact suffix-recompute splice** cutting up to 88% of per-branch compute (§7.3). Five sealed conversions (§8.6) then locate where readouts become gains. Decision-level uses convert: hidden-state scores beat shortcut-only scores on the risk–coverage curve in all four sealed abstention arms, and forced selection beats an exact matched-random null on **39 informative groups** (**34/39** vs **23.0/39** — malformed-avoidance carries it; a shortcut-only selector reaches 35/39) and on a prospective **all-well-formed pool** (**27/32** vs 64.8% expected, $p = 0.0086$): content-sensitive selection established; the hidden-vs-surface increment (+0.09 [−0.06, +0.25]) stays open. Generative control does not convert: directional steering across seven methods yields unsigned effects only; per-task loop allocation fails a matched-histogram random control on two checkpoints (tap and native exit gate alike); a minimal sign-conditioned LoRA detects no direction–outcome binding. A rank-corrected two-null audit does not support the tidiest geometric explanation; the boundary — decision-level use versus generative control — remains empirical, its mechanism unresolved.

We call the readable, decision-usable property **operational proto-introspection**, defined narrowly against the self-report introspection literature and claiming nothing about consciousness, self-awareness, or autonomous control. The taps read hidden trajectories, not text; two are forward-looking (the pre-answer probe and the branch-survivability tap). The model is not consulting its own states: *we* read them, and what we read governs whether and what to answer, not how the model computes (whether it uses such information internally is untested). That gap makes training-time integration — a model never trained to align readable process-quality directions with writable control directions — the most direct next hypothesis; a minimal binding pilot (§8.6) bounds only its cheapest form.

Corrected numbers throughout. Every load-bearing current quantitative claim is reported under an audited protocol — source-item-disjoint splits with a zero-crossing integrity check, and antisymmetrized evaluation for pairwise scorers (§3.7); historical or diagnostic quantities with incomplete provenance are marked as such and are not load-bearing. The audit corrected five distorted figures (erratum to arXiv:2604.09870). Under the corrected protocol the relational preference signal is real but modest (0.565 versus 0.542 pointwise; paired Δ +0.023, 95% CI [+0.013, +0.033]), the claim that reasoning fine-tuning *installs* the readable signal is retracted, and a non-looped SFT transformer reads candidate quality at 0.568 task-disjoint, so this class of readout does not require recurrence.

Limitations are explicit: the pre-answer effect, though independently replicated on its second domain, is established on RLTT only — a powered Thinking replication is unresolved; the depth trend is replicated in one family plus one outside architecture, not universally; directional steering is an established negative; the bounded branch screen removes evidence for a gain without estimating a general deficit; content selection's hidden-vs-surface increment is unresolved at the current sample size; and no matched writable data exists at the early loci, so the geometry of the boundary stays unresolved.

How to read this paper

This is the complete account of the program, at three reading depths. *The one result* — the strict pre-answer finding, now on two domains — is Section 5 and Figure 5; a reader with ten minutes should read those and the evidence-status map (§10.2). *The main argument* — readable, readable progressively earlier, usable to decide but not to steer — is Sections 4.4, 5, 6, and 8, with the synthesis in Section 10. *The full record* — the taps, the executable substrate, the geometric explanations left unresolved, and the discarded mechanisms that turned into diagnostics — is the whole paper, written so that each negative result and retraction is auditable. The evaluation-integrity protocol (§3.7) governs every load-bearing current claim; cross-references point forward and back so any section can be entered directly.

1. Introduction

A standard transformer processes each token position through its layer stack once: intermediate activations exist, but the state at a given position is not revisited — there is no native trajectory of repeated refinement at a fixed position before the model moves on. Looped — or universal — transformers relax this by applying the same layers repeatedly, refining a hidden state across several iterations before decoding. This produces something a single-pass model does not have: an internal *trajectory*, a sequence of intermediate computational states that the model traverses on its way to an answer. Ouro-RLTT, the frozen 2.6B looped transformer we study, exposes such a trajectory at every generation step across four loop iterations and forty-eight layers.

The existence of this trajectory makes a question concrete that is harder to pose in a single-pass model: **do these intermediate states carry readable information about the quality of the model's own ongoing computation — and if they do, can that information be turned into better outcomes?** Concretely, before Ouro-RLTT emits an answer, do its loop states already encode whether the computation currently underway is likely to succeed, which of several candidate continuations is preferable, or whether a branch is worth pursuing? And if that information is present and externally readable, can an intervention built on it — steering, branching, selecting — actually improve what the frozen model produces? This paper answers the first question yes, on two domains, and adds a mechanistic refinement the loop makes possible: recurrent iterations move candidate-quality readability *earlier* in the reused physical stack. For the second question the answer splits along a line this paper locates precisely. At the *decision level* — whether to answer at all, and which already-generated candidate to commit to — the readout converts into validated gains: a calibrated-abstention improvement on both domains, and forced selection that beats an exact matched-random null even on a pool where every candidate is well-formed by construction. At the level of *generative control* — steering the computation, allocating its recurrent depth, injecting branches, or binding a writable direction to outcomes with light training — no tested intervention produces a validated gain. The line between those answers is this paper's subject; we use the shorthand "yes and no" in what follows for that fuller statement.

We are deliberate about the second question's phrasing, because the tempting version of it is wrong. We do not ask whether the *model* uses its own signal; the model is not consulting anything, and it does not know our probes exist. We ask whether *we* can use it — whether an external reader of the model's states can convert what it reads into capability through any frozen intervention. That is the question this paper answers, and §8 makes the distinction explicit.

One scope note belongs here rather than buried in a later section, because it constrains how the results should be read. We study a looped model, but we do not find that looping is what makes process-quality information readable. A conventional non-looped SFT transformer supports the same class of readout (§4.7), and the preference signal we probe is already present in Ouro's untrained base model (§3.5). What the loop supplies is the *trajectory* — an internal sequence of intermediate states, available at every position without spending output tokens — and the iterative depth that lets injected branches genuinely diverge across recurrent steps (§7.6). The readouts are a property of trained transformers; the branching substrate is where recurrence does distinctive work.

1.1 Relation to work on introspection in language models

There is a fast-growing body of work on whether language models can introspect, and it is important to state at the outset how our question differs from the one that literature asks, because we borrow its vocabulary while making a deliberately weaker claim.

The dominant paradigm operationalizes introspection through **self-report**. Binder et al. (2024) define introspection as acquiring knowledge that originates from a model's internal states rather than from its training data, and test it by finetuning a model to predict its own behavior, arguing that success implies privileged access to internal representations. Lindsey (2026) introduces the concept-injection setup — steering vectors for known concepts are injected into the residual stream, and the model is asked whether it notices an injected "thought" and what it is about — and reports that the most capable models detect such injections at modest rates with near-zero false positives, establishing accuracy, grounding, internality, and metacognitive representation as criteria for the reported signal. Comşa and Shanahan (2025) sharpen the conceptual bar, arguing that genuine introspection requires a *causal* connection between the internal state and the model's report of it, so that verbal mimicry of introspective language is insufficient. Subsequent work extends this paradigm to open models and probes its mechanisms: Pearson-Vogel et al. (2026) show that a model's residual stream reveals detection of a prior concept injection even when its sampled text denies it, and that detection requires reading information cached from earlier tokens.

Every method in this cluster shares a common shape: a representation is *manipulated* (by injection or finetuning), the model is *asked to report*, and the report is *validated* against criteria of causal grounding. This is introspection as self-knowledge that the model can articulate.

Our question is different along both axes. We do not manipulate the model's representations with foreign concepts, and we do not elicit or evaluate any self-report. Instead, we read *naturally-arising* intermediate states — the states the model produces in the ordinary course of solving a task — with small external probes, and ask whether those states contain information about the quality of the model's own ongoing computation. We never ask the model what it is thinking; we measure what its process states reveal to an outside reader, and then we ask whether an external intervention built on that reading can improve what the frozen model produces. This is a weaker property than self-report introspection: it makes no claim that the model has access to, represents, or can articulate its own states. To mark both the kinship and the distance, we call it **operational proto-introspection** — a readout-side precursor to, and not an instance of, the self-report introspection the above work investigates. Positively and compactly: *a hidden state is operationally proto-introspective if an external reader can recover, from that state alone, information about the quality or likely outcome of the model's own ongoing computation before that computation resolves.* We introduce the term only in this narrow operational sense here, and defer its full definition and defense until after the evidence is on the table (Section 9), because the word carries strong connotations that the evidence does not underwrite and we would rather earn it than assume it.

Two results from the self-report literature are worth flagging as directly relevant rather than merely adjacent. The finding that a model's residual stream carries injection-detection information its own output denies (Pearson-Vogel et al., 2026) is an independent demonstration that hidden states can contain more about a model's situation than its outputs report — the same premise our readouts rest on, generalized here from injected-concept detection to naturally-arising process quality. And that this hidden signal is recoverable from *cached* representations connects to the executable substrate we develop, in which readouts are computed over the model's own key/value cache during generation.

1.2 Reading is not controlling

A paper that only established readable process-quality signals would be a probing paper, and it would be one easy question away from its central weakness: *so what — can anything be done with them?* The contribution here is that we can answer that question, and the answer is a boundary. We build not only the readouts but an executable internal branching substrate — autoregressive, branch-specific key/value-cache carry with a bit-exact suffix-recompute splice — so that the readouts can be installed *inside* the generation loop and used to select among branches at run time, rather than analyzed offline. With that machinery in hand, we test whether the readable signals confer control. The answer divides cleanly. Used at the *decision level* — calibrated abstention, and forced selection among completed candidates, including on a pool where every candidate is well-formed by construction — the readout confers validated gains. Aimed at *generative control*, it does not: directional steering across seven methods is an established negative, branch injection deconfounded against matched sampling produces no gain in a bounded screen, per-task allocation of recurrent depth is not signal-driven for tap or native gate, and a minimal attempt to train one writable direction against outcomes detects no binding. The direction that *predicts* success is not a direction that, written back into the model, *produces* success. We call this the **readout–control boundary** — a boundary between decision-level use of readable states and generative control over the underlying computation — and it is the paper's load-bearing result. Notably, it is an *empirical* boundary: we test the tidiest available explanation — that the writable branch directions and the outcome-relevant directions occupy different subspaces — and, under a rank-corrected two-null audit, cannot support it, which sharpens rather than resolves the question of why the tested frozen conversions do not deliver a gain.

A negative result is only worth reporting if the experiment that produced it could have come out otherwise, and three conditions here make that the case. First, **the signal is genuinely there**: the taps identify oracle-containing branches at 0.9697 retention (task-disjoint), detect generated-branch correctness at AUROC 0.7755, and predict the model's own success before it answers. Failure to steer is not failure to read. Second, **the machinery genuinely works**: the branch substrate is validated by bit-exact identity checks — a zero-perturbation fork reproduces the reference exactly at prefill, the suffix-recompute splice is bit-exact across all 192 cache slots, and omitting the carry produces the expected large divergence. Failure to steer is not a plumbing bug. Third, **the obvious confounds are controlled**: the apparent gains from sampled branch injection dissolve against K-matched plain sampling, so what remains is not sampling luck. The negative survives the conditions under which a positive would have been believed. That is what makes it a boundary rather than an absence.

1.3 Contributions

1. **Pre-answer prediction of the model's own success, on two domains.** Hidden states predict whether the model's in-progress computation will succeed *before the answer exists*, adding significant information beyond length and log-probability shortcuts: on GSM8K, incremental AUROC +0.066, 95% CI [+0.021, +0.112] (paired task-clustered boot-

strap; leave-one-task-out stable), and on Horizon Logic — synthetic propositional entailment with proof depth as an explicit reasoning-horizon control and a deterministic truth-table verifier — incremental AUROC +0.111, 95% CI [+0.056, +0.169], pooled over the original cohort and a prospectively generated, task-disjoint extension. The strict cut excludes the answer region and the gold value in code on both domains. This is the paper's primary positive result and the empirical core of the proto-introspection framing. The extension cohort independently replicates the increment (+0.095, CI [+0.038, +0.157]) and the pooled result is robust to an adversarial malformed-sibling shortcut baseline (§5.3).

2. **Recurrent refinement moves candidate-quality readability earlier in the shared physical stack.** Across loops L1–L4 at physical layers 8, 16, and 24, a matched low-capacity antisymmetric tap reads candidate quality only weakly on the first pass at layers 8 and 16 (macro top-1 0.410 and 0.440; the tested layer-8 cell clears its own label-shuffle floor by just +0.060) and at 0.633 and 0.628 by loops 3–4 — statistically tied with the final-loop layer-24 refit (0.618) and with the established late-layer basis. Frozen cross-loop transplants separate two questions that a single accuracy number conflates: *is the information readable here?* and *is it carried by the same coordinates?* From loop 2 onward the answer to the second is yes (transplants retain the target-local performance, Spearman 0.90–0.99); from loop 1 it is no (0.09–0.23 lost, Spearman as low as 0.53). The largest representational rotation is concentrated at L1→L2. This is a readout-geometry result: no intervention was performed at these loci.
3. **The readout–control boundary, located precisely: decision-level use converts, generative control does not.** On the decision side, two validated conversions of readout into outcome gains. *Selective prediction*: hidden-state-based scores improve the risk–coverage trade-off over shortcut-only scores in all four sealed arms — hidden-plus-shortcut on Horizon, and hidden-alone on GSM8K, whose combined arm was not preserved (Horizon ΔAUARC +0.0124, CI [+0.0048, +0.0207]; at 70% coverage, selective accuracy 92.6% versus 88.4%). *Terminal selection*: on the expanded pool, forced choice beats an exact matched-random null (34/39 versus 23.0/39, $p = 2.44 \times 10^{-5}$) with the margin carried by malformed-avoidance; on a prospectively constructed pool where *every* candidate is well-formed, selection remains significant (27/32 versus 64.8% expected, exact $p = 0.0086$), establishing content-sensitive selection — while the hidden-state selector's advantage over a surface-only control there (+0.09, CI [−0.06, +0.25]) is not resolved, a distinction we keep explicit. On the generative side, uniformly no validated gain: directional intervention across seven steering methods is an established negative; the bounded four-task branch screen removes evidence for a frozen-fork gain without estimating a general deficit; per-task allocation of recurrent depth is not signal-driven — for the tap *or* the model's own exit gate — against a matched-histogram random control on two checkpoints; a 300-step LoRA probe changes diversity and parse behavior without improving reachability; and a minimal sign-conditioned LoRA detects no direction–outcome binding (+0.008, CI [−0.036, +0.050], indistinguishable from a shuffled control). A rank-corrected two-null subspace audit does not support the simplest span-misalignment explanation, leaving the boundary empirical and its mechanism unresolved, and motivating full training-time integration as the most direct next hypothesis.
4. **An executable internal branching substrate, validated by exact identity.** Autoregressive branch-specific KV-cache carry across Ouro's 192-slot recurrent cache (4 loops × 48 layers), established through a six-level correctness ladder — independent branch caches with no cross-contamination, batched equivalence, lineage-preserving prune/reorder, and a negative control confirming the carried cache is load-bearing (withholding it diverges to RMS ≈ 3.0). On top of it, a **bit-exact suffix-recompute splice**: because the KV cache stores keys and values but *not* the inter-layer residual stream, a perturbed branch normally forces a full re-prefill; capturing the residual at the perturbation boundary makes the perturbed state reconstructible with no forward pass, so only the affected suffix is recomputed — saving up to **88%** of per-branch layer passes while matching a full recompute bit-for-bit across all 192 slots. Branch/fork machinery over standard KV caches is well established (PagedAttention's copy-on-write, SGLang's RadixAttention, SpecInfer's tree attention), and depth-recurrent models already reuse KV across recurrent steps (Geiping et al., 2025; Zhu et al., 2025); the specific piece we have not found in prior work is the residual-capture suffix splice that makes a *mid-computation-perturbed* branch bit-exactly reconstructible over the loop×layer recurrent cache without a re-prefill. This is what makes the negative result credible — the machinery demonstrably works, and no frozen intervention through it produced a validated generative gain — and it is independently useful for search in looped architectures.
5. **Role-specialized readouts that work, and an architecture control that constrains their interpretation.** The readable signal decomposes into distinct taps — branch survivability (**0.9697** oracle retention under a task-disjoint split, with the layer-47 locus causally load-bearing: ablating the channel the tap reads collapses retention to **0.0417**), content ranking (**0.6310** task-disjoint across five domains), and generated-branch correctness (AUROC **0.7755**) — each recovered by a low-capacity probe on frozen states and consumed by a live branching scaffold. A task-disjoint domain-transfer study shows specialization pays where distinctions are hard (code-trained taps read coding at **0.953** against a general tap's 0.694) and is unnecessary where a general quality axis suffices (a balanced generalist matches

the reasoning specialist). And a **non-looped SFT transformer supports the same class of readout** (0.568 macro top-1, task-disjoint), so recurrence is *not* necessary for process-quality readability — a constraint on how these results should be read, and one we report against our own framing's interest.

1.4 Roadmap

Part I fixes notation and describes Ouro-RLTT as a hidden-state substrate (Section 2). Part II establishes the readout side, building to the paper's primary positive result: preference structure, reported under the corrected evaluation protocol that §3.7 states (Section 3), role-specialized readouts, the cross-loop localization study, and the non-looped architecture control (Section 4), the strict pre-answer success-prediction result on two domains (Section 5 — **the paper's headline; a reader with limited time should start there**), and generated-branch correctness, which pivots toward the control problem by showing that a decodable — and even retainable — signal does not become a reliable *substantive* commitment (Section 6). Part III presents the executable branch/carry/prune substrate (Section 7). Part IV presents the readout–control boundary, where the negative results land against the backdrop of both the readouts and the machinery, and closes with five sealed readout-to-gain conversions (§8.6) that locate the boundary between decision-level use and generative control (Section 8). Part V defines operational proto-introspection against the self-report literature, synthesizes the evidence, states limitations — foremost that strict pre-answer generality across checkpoints remains unresolved on Thinking — and lays out the training-time integration the frozen boundary motivates (Sections 9–13).

1.5 Relation to the prior project paper

This work builds directly on Kirin (2026a), which found that Ouro-2.6B's loop states encode human preference predominantly relationally, and introduced the separable, frozen-backbone evaluator paradigm (a lightweight ~5M-parameter head read off a frozen backbone) that we adopt throughout.

We correct that paper as well as extend it. Three of its reported figures — the 84.5% relational linear probe, the 21.75% pointwise linear probe, and the 95.2% nonlinear evaluator — do not survive audit: the first two were inflated by a leaked evaluation split, the third by a canonical-ordering prior (§3.3, §3.7). The prior paper's *direction* holds — preference is decoded more accurately relationally than pointwise — but its magnitudes do not, and its strongest claim, that preference is *unavailable* pointwise, is withdrawn: a clean pointwise probe reads 0.5418, significantly above chance. The corrected contrast is 0.5653 versus 0.5418 (paired Δ +0.0234), not 0.845 versus 0.2175. The correction of record is the erratum to Kirin (2026a, arXiv:2604.09870v2); §3 here reports the corrected science, and §3.7 states the protocol together with both failure mechanisms and all five corrected figures.

Beyond the correction, we extend the prior work in three directions it did not address: from a single preference signal to role-specialized process-quality readouts (Section 4), from candidate comparison to pre-answer prediction of the model's own success (Section 5), and from offline readout to the question of frozen control through an executable branching substrate (Sections 7–8). Where this paper reuses the prior setup — the HH-RLHF data, the hidden-state extraction, the evaluator head size, the epoch-2 checkpoint selection, and the antisymmetry-enforcement training protocol and its metric-deflation caveat — we note the reuse and cite Kirin (2026a) rather than re-deriving it.

The empirical bridge between the two papers was less tidy than the final section structure might make it look. Applied unchanged to a wrong domain — selecting among Ouro's candidate continuations on 100 Hendrycks MATH problems, a task whose labels are not preference labels — the HH-trained evaluator picked a correct-answer-containing continuation far more often than single-shot Ouro under the same harness. We do not treat that result as load-bearing (its exact figures, and the truncation confound that followed it, are detailed and caveated in §3.6); what mattered was the implication that the evaluator was reading something more general than an HH-specific preference artifact. That changed the question. The object of study became whether Ouro-RLTT's looped hidden states expose a broader family of process-quality signals whose geometry overlaps across preference, content, reasoning, correctness, and branch viability. Section 3 reconstructs the relational primitive; Section 4 shows why the mature answer is role-specialized taps rather than one universal evaluator.

1.6 Relation to latent reasoning and internal search

The branching substrate we build (Section 7) sits alongside two neighboring lines of work it should not be conflated with, and we state the distinctions here so later sections can reference them rather than repeat them. We emphasize at the outset that this is a positioning, not a performance comparison: the paper's substrate result is a *negative* one about frozen control, not a claim to outperform any search method.

The first neighbor is external, token-level search — tree-of-thought (Yao et al., 2023), beam search, and best-of-N — which wraps repeated model calls around sampled text and selects among decoded candidates. Our branching is not a text-level wrapper; it operates *inside* the model's own loop/layer key-value cache, forking and carrying per-branch state rather than re-invoking the model on strings. The performance relationship to best-of-N is, moreover, not left open: a K-matched plain-sampling comparison (which is best-of-N with oracle selection) is precisely the deconfound against which our frozen branching shows no gain (Section 8.2). We do not claim to beat best-of-N; matched best-of-N is the baseline our central negative result is measured against.

The second neighbor is latent reasoning in non-looped models, of which Coconut (Hao et al., 2024) is the clearest instance: it feeds a model's last hidden state back as the next input embedding rather than decoding it to a token, yielding a continuous "thought" that can hold several candidate next steps in superposition and explore them breadth-first before committing. Two differences matter. Coconut's branching is *implicit and superposed* within a single continuous trajectory and is *trained in* by a multi-stage curriculum; ours forks *explicit*, separately cached trajectories that are carried, scored, and pruned as distinct objects, on a *frozen* backbone. And the exploration lives in a different place: Coconut induces a latent trajectory along the *sequence* by spending token positions, whereas our branches diverge along the model's *depth*, across loop iterations — a distinction that turns out to be the substantive architectural reason the substrate suits a looped model specifically, developed in Section 7. The trained-vs-frozen contrast also bears directly on our boundary result and its proposed resolution (Sections 8, 12).

2. Ouro-RLTT as a Looped Hidden-State Substrate

2.1 The model

Ouro-RLTT is a 2.6B-parameter looped (universal) transformer: a fixed stack of forty-eight transformer layers applied over four *loop iterations*, so that a single token position is processed by the same weights four times, with the hidden state carried forward between iterations. Reasoning-oriented training (the RLTT variant) encourages the model to use these iterations to refine intermediate computation before committing to output. We use the model entirely **frozen**: no backbone weight is updated anywhere in this paper. Two explicitly labelled bounded-LoRA probes (§8.2 and §8.6, C5) add adapters rather than altering the backbone, and both exist to test a boundary, not to improve the model. Every readout in this work is an external probe trained on top of frozen activations.

The relevant consequence of the looped design is that each generation step yields not a single hidden vector per layer but a small **trajectory**: the same position revisited across four loop iterations produces four successive hidden states per layer, and it is this trajectory — the model's own iterative refinement of its computation — that we read.

2.2 Notation

We fix notation used throughout the paper.

- $h \in \mathbb{R}^D$ denotes a hidden-state *feature vector*. Unless otherwise stated, a feature is formed by pooling and concatenating loop states across a fixed set of tapped layers and loop iterations; in the primary configuration we tap three layers across four loop iterations at hidden width 2048, giving a feature dimension $D = 3{\times}4{\times}2048 = 24{,}576$. The three tapped layers are **24, 36, and 47**, over loop indices L1–L4, with pooling variants (mean, final-loop L4, or loop-concatenation) selected per role. One family of experiments uses a different basis: the powered HH preference probes of §3 read post-final-norm states at the four *loop boundaries* only ($4 \times 2048 = 8{,}192$), because the question there is about the loop trajectory rather than about layer-localized roles. Full extraction detail is in Appendix A.
- h_A, h_B denote feature vectors for two candidate continuations or branches A and B.
- $\Delta h = h_A - h_B$ denotes the pairwise hidden-state difference. Preference-relevant information is decoded *more accurately* from Δh than from h_A or h_B individually (0.565 vs 0.542; paired $\Delta = +0.023$, §3.2), which is why comparison taps in this work operate on differences. The advantage is real and modest; an earlier version of this work reported it as far larger, and that figure is retracted (§3.7).
- U_{inj} denotes the subspace spanned by the S1/S3 frozen injection/carry deltas — the directions the frozen branch mechanism can actually *write* into the residual stream — with $P_{U_{\text{inj}}}$ the orthogonal projector onto that span.
- d_{out} denotes the verifier-success outcome direction: the hidden-state direction separating verifier-correct from verifier-incorrect continuations.
- The **projection fraction** of the outcome direction into the injection span is

$$\pi_{\text{inj}}(d_{\text{out}}) \;=\; \frac{\|P_{U_{\text{inj}}}\, d_{\text{out}}\|^2}{\|d_{\text{out}}\|^2} \;\in\; [0,1],$$

  read against the random-direction baseline k/D where $k = \dim U_{\text{inj}}$ and D is the ambient feature dimension (Section 8).

We also distinguish a **candidate group** (a set of alternative continuations to be compared or selected among) from an **oracle-present group** (a set for which verifier/gold correctness labels are available for evaluation), and use "verifier-correct" for continuations a task-specific checker accepts and "gold" for reference answers.

2.3 The cache substrate

Ouro-RLTT's key/value cache is organized by both loop iteration and layer. We index cache state by a flattened slot $\text{slot} = u \cdot 48 + \ell$ for loop iteration $u \in \{0, 1, 2, 3\}$ and layer $\ell \in \{0, \dots, 47\}$, giving $4 \times 48 = 192$ cache slots per position. This organization is what makes the executable branch/carry substrate of Section 7 possible: a branch is a distinct trajectory through these 192 slots, and forks, carries, prunes, and reordering all operate over this indexed cache. We defer implementation detail to Section 7 and Appendix F, and note here only that the cache structure — loop × layer × position — is the object the readouts are computed over and the object the branch machinery manipulates. The two halves of the paper, readout and control, read and write the same substrate.

2.4 What we do and do not assume

We assume only that (i) the model's loop trajectory is a meaningful object to read, which the readout results justify post hoc, and (ii) the frozen backbone's behavior is stable and reproducible, which we verify by bit-exact identity checks on the branch machinery (Section 7). We do **not** assume that readable information is causally used by the model, that the model can report on its states, or that any readout direction is a control direction — indeed, the central negative result of the paper is that the last of these fails.

2.5 Operational terms

Several terms carry specific, consistent meanings throughout the paper; we collect them here for readers outside this project's vocabulary.

- **Readout / tap:** an external probe trained on frozen hidden states to predict a property of the model's computation or of candidate continuations. Readouts never modify the model.
- **Process-quality signal:** readable information about the *quality* of computation — likely success, stability, preference, content quality, branch survivability, or generated-branch correctness — as opposed to the model's current answer.
- **Branch:** an alternative continuation, or hidden-state trajectory, derived from the same prompt and computation via a fork at a chosen boundary.
- **Carry:** preserving a branch's own key/value cache state across subsequent computation, rather than restarting the branch from text alone.
- **Survivability:** whether a branch should remain in the search pool because it may still lead to a correct (oracle) continuation.
- **Selection:** forced choice of a single final branch or answer under commitment.
- **Steering:** direct intervention on hidden states along a learned or readout-derived direction during generation.
- **Readout vs. control (actionability):** *readout* is the ability to read a signal externally; *control / actionability* is the ability to act on that signal for gain. The paper separates two kinds: **decision-level** use — whether to answer, and which finished candidate to commit to — and **generative control** — changing what the frozen model computes or writes next. The central finding is that readout converts into the first and, in the frozen model, not into the second.

PART II **Readout**

3. Relational Preference Structure

This section establishes the readout side's foundation: preference-relevant structure is linearly decodable from Ouro's loop states, and more accurately from comparisons than from individual representations. Every load-bearing current quantitative claim here is reported under the corrected evaluation protocol stated in §3.7; historical or diagnostic quantities are marked explicitly — several published and draft-stage figures on this topic were distorted by evaluation artifacts, and the corrected effects are real and modest. Readers interested in *how* the original numbers went wrong, and in the general lessons, should read §3.7 in full; readers interested primarily in this paper's main results may read §3.7's protocol statement and skip to Section 5.

3.1 Setup and the two probes

We train probes to predict human preference labels (from HH-RLHF preference pairs; Bai et al., 2022) from frozen hidden features. Two probe families are compared under an identical protocol:

- a **relational** probe on the pairwise difference $\Delta h = h_A - h_B$, scoring $s(A, B) = w^\top(h_A - h_B)$ with no bias term, so that $s(B, A) = -s(A, B)$ exactly; and
- a **pointwise** probe on a single pooled representation h_A, asked to classify one candidate as chosen or rejected without seeing its partner.

Both use bias-free L-BFGS logistic readouts over mean-pooled representations from four post-final-norm loop boundaries (feature width $4 \times 2048 = 8{,}192$), on 40,000 HH source pairs under a strict **pair-disjoint** split (32,000 train / 8,000 evaluation source pairs). The pair-disjointness is essential and is the subject of §3.7.

3.2 Preference is decodable, and relational decoding is more accurate

On Ouro-2.6B-Thinking, held out on 8,000 unseen source pairs:

Probe	Held-out accuracy	95% CI
Relational (pairwise difference)	**0.5653**	—
Pointwise, final boundary	0.5462	[0.5411, 0.5513]
Pointwise, all four boundaries	0.5418	[0.5366, 0.5471]

Because both probes were evaluated on the *identical* held-out pairs, the correct comparison is paired. The relational probe's advantage over the matched pointwise classifier is $\Delta = +0.0234$**, 95% CI [+0.0132, +0.0334]** — significant, and modest. The same conclusion holds under a stricter framing: the pointwise classifier's scores can themselves be used to *rank* the two candidates in a pair, which yields 0.5554 pairwise accuracy; the dedicated relational probe still beats that, by +0.0099 (95% CI [+0.0004, +0.0195]).

Two claims follow, and one prior claim does not.

Supported: preference-relevant structure is linearly decodable from Ouro's loop states, and it is decoded more accurately relationally than pointwise. This is why every comparison tap in this work operates on Δh.

Not supported: that preference is *unavailable* pointwise. Earlier versions of this work — and the prior project paper (Kirin, 2026a) — reported a pointwise linear accuracy of 21.75%, below chance, and concluded that the absolute channel is not usably present. That figure was produced under a leaked split (§3.7) and does not survive correction: the clean pointwise accuracy is 0.5418, significantly *above* chance. We withdraw the strong claim. Preference is accessible both pointwise and relationally; the relational route is simply better, by about two points.

We also note what these magnitudes are not. At 0.54–0.57, none of these probes is competitive with a trained reward model on this data (typically 0.72–0.75; Lambert et al., 2024). The claim here is about the *organization* of preference information in a frozen model's hidden states — that it is relationally structured — not about achieving competitive preference prediction.

3.3 A high-accuracy fixed-order evaluator, and the first audit

A higher-capacity nonlinear evaluator — attention pooling over the trajectory, per-loop differences, a trained difference-LayerNorm, a GRU across loops, and a nonlinear scorer — trained to compare two candidates in a **fixed** presentation order (chosen always first) reaches roughly **95% test accuracy** (95.2% = 8,141/8,552 in canonical order). Kirin (2026a) reported this evaluator and read the number positively: as evidence that a nonlinear model surpasses the linear probe. It does not. A fixed-order pairwise evaluator can score highly by learning a presentation-order prior rather than relational discrimination, and this one did.

On the **full 8,552-pair HH-RLHF test set**, the evaluator's fixed-order accuracy is **0.9479** (95% CI [0.9431, 0.9525]) — reproducing the historical 0.9519, which lies inside this interval — but its **strict antisymmetrized accuracy**, the fraction of pairs on which the antisymmetric relational component of its score has the correct sign, is **0.6392** (95% CI [0.6291, 0.6493]). Roughly a third of the apparent accuracy was the ordering prior: a large learned first-position offset (the symmetric score component is on average 1.50× the antisymmetric one; 75.3% of pairs are scored "prefer the first argument" in both orders), not a degenerate constant — content still matters (normal/flipped score correlation −0.925), but the offset overwhelms the sign for most pairs. The score decomposition that makes this measurable is stated in §3.7; the audit artifact and the pooling/normalization ablations are summarized in Appendix B.

Antisymmetrization is therefore a mandatory audit for any fixed-order pairwise evaluator: fixed-order accuracy is not, by itself, a measure of relational discrimination. We report the historical 95.2% only as fixed-order, discovery-stage accuracy.

3.4 What survives, and how the results now order

Under honest, held-out evaluation the three preference numbers order as follows:

Reader	Clean accuracy	Protocol
Nonlinear evaluator, strict antisymmetrized	**0.6392**	full 8,552-pair disjoint test set
Linear relational probe	0.5653	40k pairs, pair-disjoint
Linear pointwise probe	0.5418	40k pairs, pair-disjoint

The nonlinear evaluator's *relational* component (0.639) is the strongest clean preference readout in the project — the additional capacity does buy genuine relational discrimination, once the order artifact is removed. This reverses a claim made in an earlier version of this work, which asserted that the linear probe (then reported at 84.5%) *outperformed* the antisymmetrized nonlinear evaluator. That inversion was an artifact of the leaked linear number; on clean splits, the ordering runs the other way.

This also revises the prior paper. Kirin (2026a) is correct that preference is encoded relationally rather than absolutely — the paired relational-over-pointwise advantage survives on clean data — but the magnitudes it reports (84.5% relational, 21.75% pointwise, and a 95.2% nonlinear headline) are all artifacts of the split and ordering defects documented here — two inflated, one deflated below chance. The corrected contrast is 0.5653 versus 0.5418, not 0.845 versus 0.2175. The direction of the prior paper's central finding stands; its size does not.

3.5 Training increases the linear preference signal, modestly

An earlier version of this work claimed a training-stage *localization*: that a fixed evaluator read Ouro-2.6B-Thinking at 95.2% and Ouro-RLTT at 95.0% but collapsed to 24% on the base model, and that reasoning fine-tuning therefore *installs* the readable signal. **That claim is retracted.** The historical base=24% cell has no surviving artifact and does not reproduce: the same saved evaluator, applied to the pinned base checkpoint under controlled preprocessing, returns ~95% canonical accuracy on base as well (95.0 / 95.0 / 94.5 across base / Thinking / RLTT, with flat antisymmetrized accuracy 58.0 / 59.5 / 60.0). There is no base-specific collapse. The likely cause of the original figure — a mismatched checkpoint, extraction locus, orientation convention, or transcription error — cannot be established without the artifact, and we do not speculate further.

The question the retraction leaves open — *does training change the readable preference signal at all?* — is answerable, and we answer it properly. Applying the clean pair-disjoint linear protocol of §3.1 to all three backbones on the identical 8,000 held-out pairs:

Backbone	Held-out accuracy	95% CI
Base Ouro-2.6B	0.5553	[0.5444, 0.5663]
Ouro-2.6B-Thinking	0.5653	[0.5543, 0.5760]
Ouro-RLTT	**0.5698**	[0.5586, 0.5804]

The marginal intervals overlap, but that is the wrong test: all three probes were evaluated on the *same* held-out pairs by construction, so the comparison is paired and the correct interval is a bootstrap over per-pair differences, which removes the pair-difficulty variance shared across backbones.

Comparison	Δ	95% paired CI	p	Significant (Bonferroni $\alpha = 0.0167$)
RLTT − Base	**+0.0145**	[+0.0043, +0.0248]	0.0046	**yes**
Thinking − Base	+0.0100	[−0.0001, +0.0201]	0.0546	no
RLTT − Thinking	+0.0045	[−0.0015, +0.0106]	0.1604	no

What this supports: the full training pipeline increases the linearly-decodable preference signal. RLTT reads higher than base by 1.45 points, and that ordering survives correction for multiple comparisons.

What it does not support: any claim that reasoning SFT *specifically* installs the signal. The Thinking-vs-base comparison is borderline and fails at 95% ($p = 0.055$); the RLTT-vs-Thinking comparison is indistinguishable from noise ($p = 0.16$). We therefore attribute the effect to the training pipeline as a whole, not to a stage within it. And crucially, the signal is **present in the base model** (0.5553, well above chance): training enriches a preference direction that pretraining already provides. The earlier framing — that the loop provides the substrate and reasoning training installs the content — is not supported and has been removed throughout.

The effect is small. A 1.45-point gain on a 55-point base is a real ordering, not a large one, and we present it as such.

3.6 The math-transfer shock that started the broader program

The transition from the prior relational-preference paper to the present work began as an empirical surprise rather than a preplanned theory of domain transfer. After training the HH-RLHF evaluator, we applied it, unchanged, to select among Ouro's candidate continuations on 100 Hendrycks MATH problems (Hendrycks et al., 2021) under a fixed harness. This was, in effect, a "wrong-domain" application: the evaluator had not been trained on math, proof search, answer checking, or verifier correctness, and a narrow preference-reader interpretation would predict either noise or a weak style/preference bias.

Instead, the evaluator's selected continuation contained the correct final answer on **47 of 100** tasks. This figure is a floor. On most of the remaining 53 tasks the evaluator did not choose a wrong answer; Ouro simply never produced a parseable one — it continued generating without committing to an answer — so those tasks are censored rather than failed, and the true selection accuracy given a scorable candidate is unknown and higher. Under the *same* harness, single-shot Ouro reached a correct answer roughly **2.7**× less often. Because the evaluator-guided selection and the single-shot baseline run under the identical harness and share Ouro's tendency to over-generate, this censoring affects the compared quantities alike, and the 2.7× is a fair same-harness comparison of how often each recovers a correct answer. The exact artifact for this historical run is unarchived; we flag the precise denominator and baseline definition for live-repo pinning and do not treat the multiplier as load-bearing evidence.

The result does not isolate mathematical understanding: part of the effect is best-of-selection simply having more chances to reach a parseable answer, and part may reflect the evaluator preferring complete over unfinished candidates. And it is distinct from — and was followed by — a sharper confound. When we tried to *build* on the result with broader math pilots, generated attempts were often verbose and truncated by token limits, truncation correlated with correctness, and a trained selector could learn "not truncated" as a proxy for "correct" rather than mathematical correctness itself; those pilots were demoted in favor of clean GSM8K with exact numeric parsing (§5, Appendix I). The clean origin result and the later confound are two different events: the same-harness 2.7× ratio is robust to the truncation-leakage problem that killed the broad pilots, which is why it survived as the motivating observation when they did not.

The important point for this paper is not the precise historical multiplier but that a preference-trained hidden-state reader transferred at all to a domain whose labels were not preference labels — surprising enough, and confounded enough, to motivate the auditable domain-transfer, tap, and pre-answer studies that follow. That was the first reason to suspect the evaluator was not merely memorizing HH-specific preference artifacts, but reading a broader latent geometry of candidate quality.

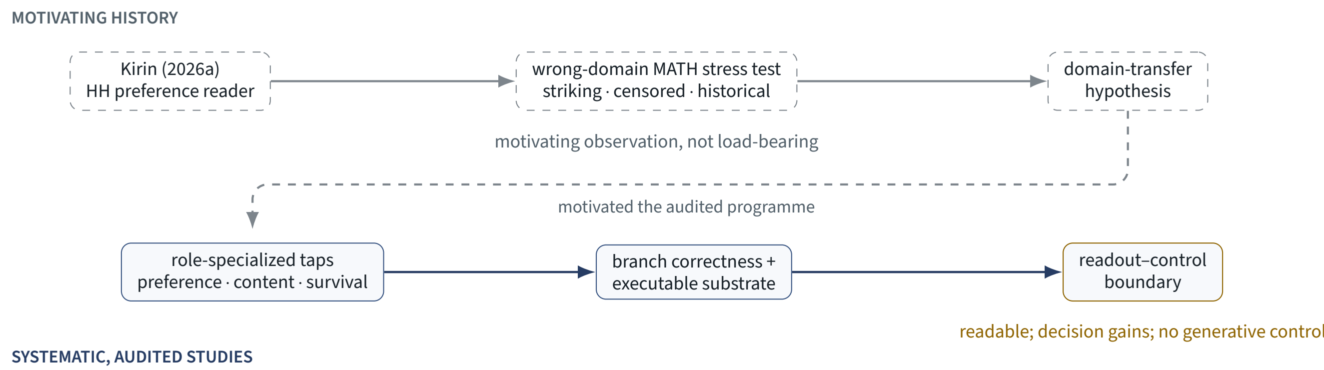
In hindsight, this surprise is the hinge between Kirin (2026a) and the present paper. It motivated the move from a single, high-capacity preference evaluator to a family of smaller, role-specialized taps. The right follow-up was not to declare the HH evaluator a universal reward model; the antisymmetry audit in this section is exactly why that would be too strong. The right follow-up was to ask which parts of the hidden trajectory support which kinds of quality judgments: preference, content relevance, survivability, generated-branch correctness, and pre-answer success. Section 4 is the systematic version of that question.

The path from the prior paper to the present one, and the structure it induced, is summarized in the lineage diagram.

3.7 Evaluation integrity: the corrected protocol

Every load-bearing current quantitative result in this paper is reported under a protocol adopted after a project-wide audit — historical or diagnostic quantities with incomplete provenance (e.g. the §3.6 math-transfer figures) are explicitly marked as such and are not load-bearing. The audit found five distorted figures — four inflated, one deflated below chance; three in the prior published paper (corrected in the erratum to arXiv:2604.09870), two in earlier drafts of this one — produced by two distinct and mutually invisible mechanisms. This subsection is self-contained: it states both mechanisms at the level of detail needed to apply or contest them, the protocol they force, and every correction that affects this paper's results.

Mechanism 1 — source-item leakage. When a dataset is built by constructing multiple rows from each source item (± difference orientations, chosen/rejected singletons, candidate families, branches per task), splitting those constructed rows independently leaks the source item across the train/test boundary. Held-out behavior becomes dependent on recognition of source items seen during training; depending on construction geometry, that dependence can inflate accuracy or systematically invert predictions below chance. The artifact need not look suspiciously good: the same defect produced both an inflated 0.845 and a below-chance 0.2175 that an earlier paper interpreted as a finding ("inverted polarity").



Project lineage. An unexpected out-of-domain transfer result (§3.6) reframed a single preference evaluator as a reader of a broader process-quality geometry, motivating role-specialized taps (§4), the branching substrate (§7), and the negative control results (§8).

How much leaks. The magnitude is not a matter of judgement. Let source items $x_1, \dots, x_N$ be the units whose independence the evaluation claims, and let each generate $m \geq 2$ constructed rows $r_{i1}, \dots, r_{im}$. If the split operates on rows with train fraction f, then the probability that a given evaluation row has **at least one sibling in training** is

$$1 - (1 - f)^{m-1}.$$

At the standard $f = 0.8$ with only $m = 2$ rows per item, that is already **80%** of evaluation rows — consistent with the 74% observed in this project's own pointwise probe (the second row of the correction table below, whose evaluation rows had their pair partner in training). The measured quantity silently changes: the held-out unit is no longer an *unseen* source item but a *recognized* one, and the evaluation reports how the probe responds to items it has already met rather than whether it generalizes. Nothing about the splitting code is wrong; the defect is in the unit the split is applied to, and that unit is invisible at the call site.

Why exact antisymmetry accelerates it. We had relied on exact antisymmetry as a structural safeguard — a bias-free scorer provably cannot prefer a candidate for the side it was presented on. Against orientation-row leakage that guarantee is worse than useless. For orientation rows $+\Delta_i$ and $-\Delta_i$ and a bias-free antisymmetric probe,

$$w^\top(-\Delta_i) \;=\; -w^\top\Delta_i,$$

so a probe that has fitted $w^\top\Delta_i > 0$ on the training row scores the held-out row $-\Delta_i$ correctly *because* it is antisymmetric. The memorized fact transfers with probability 1.

> **Proposition (exact leakage through equivariance).** Let rows $r_{ij} = g_j(x_i)$ be constructed from source items x_i by maps g_j forming a group action, with labels transforming under the same action. Suppose the *trained* scorer s satisfies the exact equivariance $s(g_j(x)) = \rho_j(s(x))$. Then, conditional on s having fitted one constructed row of a source item, its score on every sibling related by the action is determined exactly, and a row-level split converts source-item memorization into correct held-out predictions on precisely those siblings. Orientation rows under a bias-free antisymmetric scorer are the case where the premises hold rigorously: $g(x) = -x$, $\rho = -1$, and $s(-x) = -s(x)$ by construction.

The premises do real work, and we do not extend them beyond where they hold. Candidate families, mutants, and repeated branches per task are **not** exact symmetries of the scorer; there, transfer flows through shared features and is attenuated but rarely zero. We treat that extension as a heuristic rule of thumb — the more nearly the model class commutes with the construction, the more completely a row-level split leaks — and not as part of the proposition. The practical test it yields is concrete: for each construction map in the pipeline, ask whether the trained scorer can express the relation between a row and the other rows of the same source item.

Mechanism 2 — presentation-order exploitation. A pairwise evaluator trained and evaluated with its candidates in a fixed order can score highly by learning "prefer the first argument." Its data are properly split — no split check can see this; what crosses the boundary is a label, through the presentation (§3.3).

The decomposition that makes it visible. Any pairwise score splits into an antisymmetric relational component, which changes sign when the candidates are swapped, and a symmetric order-invariant component, which does not,

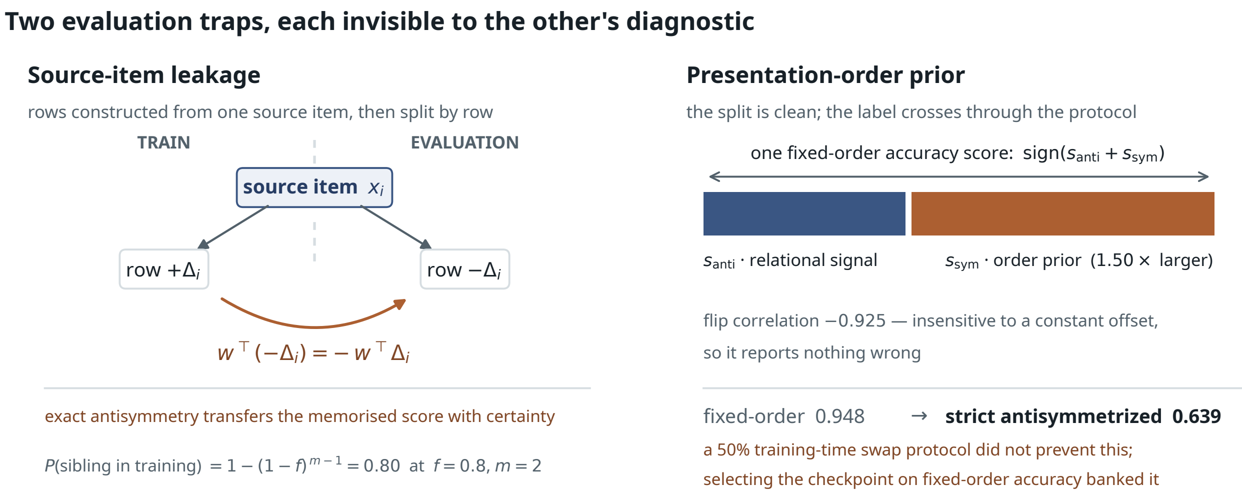


Figure 1. The two mechanisms behind the corrections of §3.7, and why each is invisible to the other's diagnostic. Left: constructed rows from one source item are split independently, so siblings cross the boundary and the held-out unit becomes a recognized source item; at f = 0.8 and m = 2 rows per item, 80% of evaluation rows already have a sibling in training, and exact antisymmetry transfers the memorized score to the negated sibling with certainty. Right: a fixed-order pairwise score is a sum of a relational antisymmetric component and a symmetric order prior; fixed-order accuracy scores their sum, so the prior is banked as accuracy, while flip correlation stays strongly negative and reports nothing wrong.

$$s_{\text{anti}}(A,B) = \tfrac{1}{2}\big[s(A,B) - s(B,A)\big], \qquad s_{\text{sym}}(A,B) = \tfrac{1}{2}\big[s(A,B) + s(B,A)\big].$$

Fixed-order accuracy scores $\text{sign}(s_{\text{anti}} + s_{\text{sym}})$; only s_{anti} carries relational information. **Strict antisymmetrized accuracy** — the fraction of pairs on which s_{anti} alone has the correct sign — is what removes the prior, and computing it requires scoring both presentation orders.

Four consequences follow, and each is visible in this project's own audited evaluator (Appendix B). First, **flip correlation cannot see the prior it coexists with**: the correlation between $s(A,B)$ and $s(B,A)$ measures whether content moves the score antisymmetrically and is blind to a constant or low-variance symmetric component — exactly where an order prior lives. Our evaluator held a normal/flipped correlation of −0.925 while its symmetric component averaged +1.265, 1.50× the antisymmetric one, and 75.25% of pairs were scored "prefer the first argument" in *both* orders. Second, the resulting gap is large: fixed-order 0.9479 against strict antisymmetrized **0.6392**. Third, **a training-time swap protocol does not discharge the audit.** This evaluator was trained with a full 50% swap, adopted specifically to prevent degenerate order solutions; a 50% swap makes a *converged* prior unlearnable in expectation, but the offset here was transient rather than converged. Fourth, and consequently, **checkpoint selection on fixed-order accuracy is itself an order-prior amplifier**: fixed-order accuracy is signal *plus* prior, so it is maximized in part by whatever maximizes the prior, and the selected epoch-2 checkpoint is where both peak. The audit must therefore be applied to the evaluated artifact, never inferred from the training recipe.

The two mechanisms are mutually blind, which is how both survived months of checking in a project that was auditing its work: a split audit cannot see an ordering prior, and antisymmetrization cannot see a leak.

The protocol, used for every load-bearing current quantitative claim in this paper:

1. **Split on source items, never constructed rows** (the HH pair, the task), and enforce an explicit integrity check that counts source items crossing the boundary and refuses to run at anything other than zero.
2. **Antisymmetrize every fixed-order pairwise evaluation**: score both presentation orders, decompose into symmetric and antisymmetric components, and report strict antisymmetrized accuracy together with the symmetric/antisymmetric magnitude ratio as the size of the prior. Flip-test correlation and sign-flip rate are order-sensitivity diagnostics, not accuracy corrections; select checkpoints on the antisymmetrized metric, and audit the trained artifact rather than the training recipe (§3.3).
3. **Power the clean evaluation adequately**: a source-disjoint split with $p \gg n$ returns chance whether or not signal exists (our first pair-disjoint audit, at 800 training pairs against 8,192 features, read 51% on every backbone and was

uninformative until rerun at 32,000 pairs). Decontamination and power are separate obligations: an underpowered clean rerun can falsely convict a real result.

4. **Audit call sites, not results**: a row-splitting helper applied to constructed data is a bug in a function, not in a result — enumerate every place it touches constructed rows and audit all of them at once, before deciding which numbers to trust. Intuitions about which numbers "look fine" are formed downstream of the leak: in this project the figure that survived scrutiny longest (0.9848) was contaminated, while the figure that looked broken (0.2175) was the leak announcing itself.
5. **Pin provenance**: model revisions and checkpoint hashes, dataset revisions, seeds, preserved raw per-example predictions, and code-auditable statistics. A third correction in this project was neither trap — a number with no surviving artifact, which could not be diagnosed, only retracted (§3.5). A number without an artifact is a claim without evidence, and this paper marks such quantities explicitly rather than treating them as load-bearing.

Corrections affecting this paper's results, each re-verified under a zero-crossing split:

Result	Mechanism	Reported	Corrected
Relational linear probe (§3.2)	orientation rows leaked	0.845	**0.5653**
Pointwise linear probe (§3.2)	pair partners leaked (74% of eval rows)	0.2175 (below chance)	**0.5418**
Fixed-order evaluator (§3.3)	canonical-ordering prior	0.952	**0.6392** (antisymmetrized)
CoreContent v2 (§4.6)	195 task IDs crossing	0.6691	**0.6310**
Branch survival (§6.2)	8 task IDs crossing	0.9848	**0.9697**

Where a genuine signal remained, it survived correction, although the direction and magnitude of the correction varied substantially; one separate training-stage claim did not reproduce and is retracted. Source-item leakage generalizes to evaluations constructed from multiple rows, variants, or candidates of a shared source. Presentation-order exploitation generalizes to pairwise scorers trained, evaluated, or selected under a canonical candidate order. The structural protection is not care but the integrity checks above.

4. Role-Specialized Hidden-State Readouts

Section 3 treated preference as a single relational signal. The second readout finding is that the readable content of the hidden trajectory is **not one scalar quality score** but a set of related yet distinguishable signals, each recoverable by its own low-capacity tap, and each localized to particular layers and loop iterations. We refer to these as *role-specialized readouts.*

4.1 Tiny taps on a frozen backbone

Each readout is produced by a small head — on the order of a few million parameters — trained on top of the frozen Ouro-RLTT trajectory; the backbone is never updated. That such small heads suffice is itself part of the claim: the information is present in the hidden geometry in a form a low-capacity reader can extract, rather than requiring a large external model to manufacture.

The move from the original evaluator to taps was therefore both scientific and methodological. The original HH evaluator was large enough to reveal that useful structure existed, and its GRU-over-loops design was useful for asking whether the refinement trajectory carried information. But the same capacity also made it able to exploit presentation-order regularities under fixed-order training. The tap program deliberately moved in the opposite direction: small heads, explicit pairwise differences, bias-free scoring, and swap-safe antisymmetry by construction. This made the readouts less expressive but easier to audit. A tap that succeeds under these constraints is stronger evidence that the information is present in the hidden-state geometry rather than manufactured by the external evaluator. Tap architectures and training details are given in Appendix C.

4.2 A decomposition into roles

Across training targets we find distinct, separately-decodable readouts spanning at least:

- **preference** — which of two candidates is preferred (Section 3);
- **content quality** — task-relevant quality of a single continuation's reasoning/content, as distinct from mere preference ordering;

- **branch survivability** — whether a branch is likely to persist rather than be pruned under the scaffold's own dynamics (§6.2);
- **generated-branch correctness** — whether a generated continuation is verifier-correct (Section 6).

These are related but not identical: a tap trained for one role does not transparently solve another, and the roles localize differently across the loop×layer grid. Two of these readouts are carried by tap families built for different pipeline stages by deliberately different methods — a **DualAnchor** family that prunes and retains branches *during* the looped branch/prune search, and a **CoreContent** family that ranks candidates *within* an already-handed-off survivor set. How each was constructed, and why the difference between them is a difference of stage rather than of "aspect," is the subject of §4.6; detailed metrics and the S3B2 relationship are in Appendix E.

4.3 Localization across loops and layers

The readouts are not uniformly distributed across the trajectory. Preference and comparison signals concentrate at particular tapped layers and loop iterations rather than appearing equally at every layer of the recurrent backbone. The canonical 24/36/47 basis was chosen from the earlier locus work, not from an architectural prior or from the final headline results. The project first ran pairwise locus and loop ablations on the HH evaluator (v2–v4), then normalization and bias-decomposition passes showed that the useful signal was relational and mid/late rather than tied to a single absolute state. Later all-layer and cached probes across coding, reasoning, and logic (v7), followed by evaluator-placement and multi-tap ensemble experiments (v8–v9), narrowed the useful region to mid-to-late decoder layers. The v10 Thinking-vs-RLTT loop-geometry pass then made layers **24, 36, and 47** the stable compact basis used by the later architecture-looped and DualAnchor baselines.

Operationally, the three layers serve different points along the same refinement trajectory. Layer 24 acts as a mid-depth representation before final consolidation, layer 36 as a late integration point, and layer 47 as the terminal/pre-output boundary where loop-to-loop spread and comparison signal were most consistently useful. We therefore use the 3 layers × 4 loops × 2048-dimensional feature basis as the default readout substrate, rather than storing all 48 layers at all loop steps. In the branch-survival line this basis was further turned into an explicit per-loop schedule, `L1_24 -> L1_36 -> L1_47 -> ... -> L4_24 -> L4_36 -> terminal L4_47`, which is why the same three loci recur in both the offline taps and the live branch/carry scaffold. Layer/loop localization per role is summarized in Appendix D and the extraction detail in Appendix A; role-specific variants may use mean pooling, final-loop L4 features, L1/L4 fusion, or full loop-concatenation.

4.4 Recurrent refinement moves candidate-quality readability earlier in the shared stack

The 24/36/47 basis was selected from *final-pass* geometry, and every tap in this project's history was fitted at those three loci. It therefore records where the signal is readable after the model has finished refining, and says nothing about *when* during the refinement it becomes readable, or whether the same coordinates carry it throughout. Because Ouro applies the *same* forty-eight layers on each of four passes, those are separable questions — physical layer 8 at loop 3 is the same weights as physical layer 8 at loop 1, applied to a state already refined twice — and they are only well-posed in a looped model, since in a feedforward stack "layer 8" happens once. We therefore extended the locus study downward, to physical layers **8** and **16** across all four loops, with layer 24 measured in the same run as an internal control and the historical L4_36 / L4_47 readouts as references.

Two evaluation modes answer two different questions, and conflating them is the main way this measurement can mislead.

- **Locus-local refit** asks *is the information readable here?* A fresh tap is trained and selected at each loop/layer cell independently, so each cell reports the best that a matched low-capacity reader can do at that locus.
- **Frozen cross-loop transplant** asks *is it carried by the same coordinates?* The tap selected at source loop u is applied **unchanged** — same weights, same feature convention, and by construction no bias, normalization, or threshold to re-fit — at a later loop $v > u$ of the same physical layer.

The protocol is the corrected one of §3.7 throughout. The target is the CoreContent v2 candidate-quality ranking over the same five domains, on the existing deterministic task/prompt-disjoint split (250 train / 60 validation / 120 held-out groups per domain; 8,592 candidates; **zero** task IDs crossing, asserted). The tap is the same bias-free `AntisymLinearNoNorm` head, trained by the locked pairwise procedure, with its 36-point hyperparameter grid selected on validation macro top-1 only. Intervals are task-clustered bootstraps (10,000 draws, clusters = task ID within domain), with identical draws applied to both members of every paired comparison; matched chance is the analytic tie-aware expectation, macro **0.2818**.

Readability rises across the loops, at every layer. Held-out macro top-1, with task-clustered 95% intervals; every cell excludes matched chance.

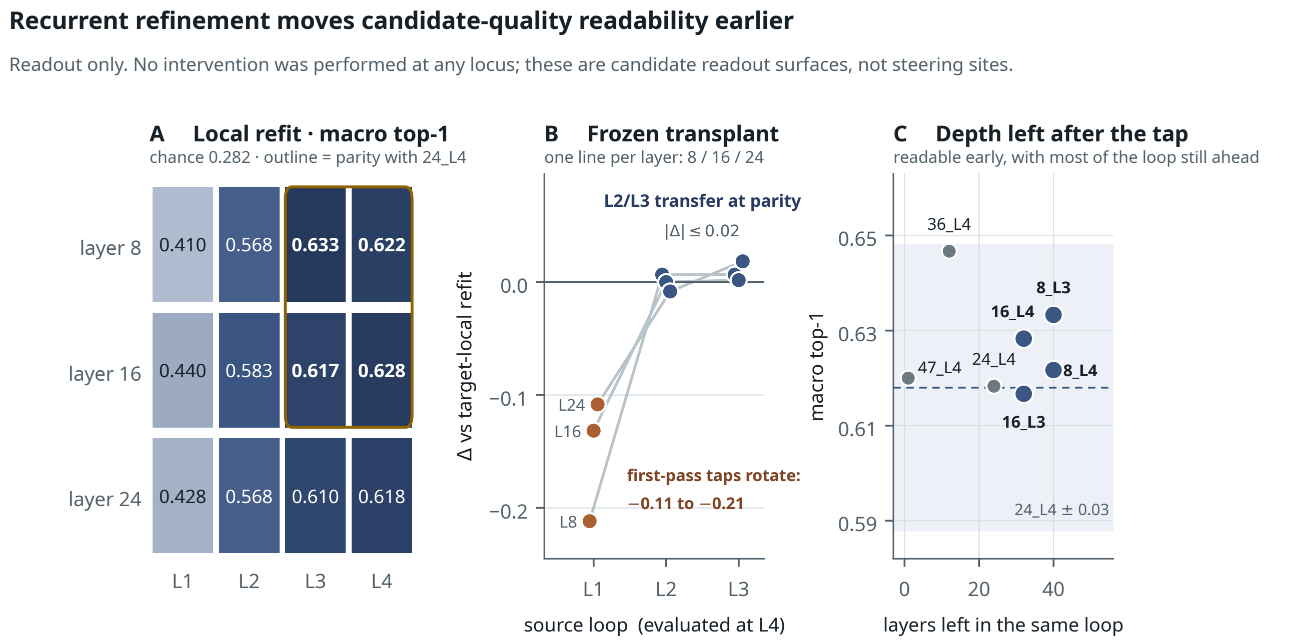


Figure 2. Recurrent refinement moves candidate-quality readability earlier in the reused physical stack. (A) Locus-local refit across loops and layers, with the historical L4 references; chance is 0.282. (B) Frozen cross-loop transplant, as the paired loss against the target-local refit: taps sourced at loop 1 lose 0.09–0.23, taps sourced at loops 2–3 lose nothing measurable. (C) The four early cells that reach parity still leave 32–40 layers of same-loop downstream depth after the tap point. All values are task-clustered; the L1→L2 rotation is the single largest coordinate change in the matrix.

Physical layer	Loop 1	Loop 2	Loop 3	Loop 4
8	0.4100 [.373, .447]	0.5683 [.532, .603]	**0.6333** [.597, .670]	**0.6217** [.585, .658]
16	0.4400 [.403, .477]	0.5833 [.547, .620]	**0.6167** [.580, .653]	**0.6283** [.592, .665]
24	0.4283 [.392, .463]	0.5683 [.532, .605]	0.6100 [.573, .647]	0.6183 [.580, .655]

References at the same protocol: L4_36 **0.6467** [.610, .683], L4_47 **0.6200** [.583, .657].

The loop trend is large and significant at every layer: L4 − L1 is **+0.212** [+0.163, +0.260] at layer 8, **+0.188** [+0.145, +0.235] at layer 16, and **+0.190** [+0.147, +0.235] at layer 24. By loop 3, the early layers have caught up with the established basis: the paired difference between 8_L3 and the final-loop layer-24 refit is +0.015 [−0.022, +0.050], and against the strongest historical reference L4_36 it is −0.013 [−0.047, +0.020] — statistical ties in both cases. Layers 8 and 16 at loops 3 and 4 are the four cells that clear the pre-declared margin (above chance *and* within 0.03 of the layer-24 final-loop refit). Loop 2 does not: 8_L2 and 16_L2 are far above chance but still 0.035–0.050 short, and we do not describe them as at parity.

The direction is coordinate-stable from loop 2 onward, and rotates at L1→L2. All nine transplants sourced at loop 2 or 3 retain the target-local performance (Δ within ±0.024; Spearman correlation with the target-local refit 0.90–0.99; pairwise sign agreement 0.92–0.97), and none of their intervals excludes zero. All nine transplants sourced at loop 1 lose materially (0.085–0.233; Spearman as low as 0.53 at layer 8), and every one of those deltas excludes zero. Each transplanted tap was required to reproduce its stored source-cell score exactly before target evaluation, and its weight hash was asserted unchanged across evaluation, so the loss is a property of the representation and not of the transfer machinery.

Physical layer	L1 → L4	L2 → L4	L3 → L4
8	0.4100 (−0.212), $\rho = 0.53$	0.6283 (+0.007), $\rho = 0.94$	0.6283 (+0.007), $\rho = 0.91$
16	0.4967 (−0.132), $\rho = 0.78$	0.6283 (0.000), $\rho = 0.95$	0.6300 (+0.002), $\rho = 0.99$
24	0.5100 (−0.108), $\rho = 0.77$	0.6100 (−0.008), $\rho = 0.90$	0.6367 (+0.018), $\rho = 0.95$

(Target macro top-1, with the paired difference against the target-local refit in parentheses and Spearman score correlation on identical held-out candidates. The full 18-transfer matrix is in Appendix D.)

Controls. A label-shuffle rerun of the full grid at four representative cells gives 0.350 (8_L1), 0.4267 (8_L4), 0.4033 (16_L2) and 0.3367 (24_L4). The shuffle floor sits well above naive chance — the alignment domain contributes a two-candidate 0.5 pairwise floor, and selecting over 36 grid points on validation adds optimism — so the informative comparison is each cell against *its own* shuffle counterpart. On that comparison the weak first-pass cell 8_L1 clears its shuffle by only +0.060, while 8_L4 clears by +0.195, 16_L2 by +0.180, and 24_L4 by +0.282. First-pass readability at layer 8 is therefore weak in absolute terms and barely distinguishable from its own shuffled control; the matched shuffle was run at four representative cells only, so we do not extend that second, stronger statement to 16_L1, which was not shuffle-tested. Ranking candidates by text length gives macro 0.323, so the taps are not length proxies. The new extraction path reproduces the production cache on 2,434 identical held-out candidates, and layers 8 and 16 are confirmed to be genuinely new loci rather than re-labelled cached ones. Twenty targeted tests pass. Full control detail is in Appendix D.

What this establishes, and what it does not. It establishes that process-quality information is linearly readable at physical layers 8 and 16 during loops 3 and 4 at parity with the historical mid/late basis; that readability increases sharply from the first loop to the later ones at every tested layer, with the L3 and L4 cells statistically indistinguishable from one another; and that the carrying direction is coordinate-stable across loops once at least one full pass has completed, with the largest rotation concentrated at the L1→L2 transition. The natural mechanistic reading is that the shared weights are semantically stationary for this readout only from the second pass onward: the first pass appears to use a distinct coordinate system for related information. It does **not** establish that a single universal quality scalar is being moved forward, that these loci are steerable, or that looping is necessary for candidate-quality readability — §4.7's non-looped control speaks directly against the last of these. No intervention was performed here. We label the four qualifying cells *candidate readout surfaces*: they are readable, shallow, and leave 40 and 32 layers of same-loop downstream depth respectively, which is exactly the profile an intervention study would want — and is not itself evidence that an intervention would work.

A cross-role boundary, reported because it constrains the reading. The same taps were applied, as an explicitly exploratory secondary readout, to the S3B2 generated-branch-correctness pool (16 tasks, 160 saved branches, leave-one-task-out; coding contributes zero positives, so this is under-powered by construction). Two things follow, and they point in different directions. Locally refit at each cell, the loop trend partly reproduces: at physical layer 16, pooled AUROC rises from 0.597 at loop 1 to 0.654 and 0.651 at loops 3 and 4. It does not reproduce at layer 24, where loop 1 (0.635) exceeds loop 2 (0.560), and layer 8 stays weak throughout (0.490–0.577), so we describe the corroboration as layer-specific rather than general. More informatively, the **frozen** CoreContent direction does not transfer to generated branches at all: transplanted AUROC lies between 0.354 and 0.475 at every one of the fourteen cells, at or below chance. This is consistent with the known characterization of the CoreContent tap as a corruption/quality detector rather than a plausible-wrong-branch ranker (Appendix E.2.2), and we read it as a role and distribution boundary — the direction that ranks constructed candidate families is not the direction that separates correct from incorrect *generated* branches — rather than as a contradiction of the primary result, which concerns a different target on a different pool.

Within-family replication and frozen transfer. Everything above concerns one checkpoint. To test whether the pattern is RLTT-specific, we re-ran the sealed protocol — the identical 2,150-group subset, splits, tap class, 36-point grid with validation-only selection, and bootstrap seed — on the other available Ouro checkpoints: the base 2.6B, the 2.6B-Thinking variant, and the 1.4B (24 layers; capture layers mapped proportionally to {4, 8, 12, 18, 23}). The signature replicates on all three (Figure 3A). Every early cell is weak on the first pass (macro top-1 0.408–0.473 across checkpoints) and reaches 0.60–0.65 by loops 3–4, with every loop contrast excluding zero; the L1→L2 rotation also replicates (L1-sourced taps lose 0.18–0.25 when frozen-transferred to L4 on every checkpoint, while L2/L3-sourced taps transfer at parity, Figure 3C). Beyond replication, the sealed RLTT taps themselves transfer **frozen across checkpoints**: scored directly on the base and Thinking feature spaces, they land within 0.028 macro top-1 of each sibling's own locally refit taps at every 2.6B-geometry cell, with score correlations of 0.97–0.99. The readable geometry is a property of the family, not of RLTT post-training. One scale difference is worth stating rather than averaging away: on the 1.4B, the early-loop cells (e.g. layer 12 at L3, 0.623) end *above* that model's own late references (18_L4 = 0.568, 23_L4 = 0.560) — the "late basis as the strongest readout" framing is a 2.6B fact, while the loop trend itself holds everywhere we looked.

Out-of-family replication on Huginn. Ouro is, to our knowledge, the only looped-LM family at this scale, so full cross-architecture replication is not available; what can be tested is the *class* claim — that readability of candidate quality increases with recurrent depth. We ran the same subset, splits, and sealed tap machinery on Huginn-0125 (Geiping et al., 2025), a 3.5B depth-recurrent transformer (2-layer prelude, 4-layer recurrent core, 2-layer coda, hidden 5280), reading the recurrent-core boundary at steps 1–8 of a single frozen forward per candidate. Readability rises with recurrence depth — macro top-1 0.332 at step 1 to 0.438 at step 8 (chance 0.282), step-8−step-1 contrast +0.107 [+0.058, +0.157] — but at a much lower absolute level than Ouro's, and **no comparable first-pass rotation is detected**: step-1-sourced taps transfer to steps 2–4 at parity and lose only −0.093 by step 8, where Ouro's L1-sourced taps lose 0.18–0.25 immediately (Figure 3C). The step-1 signal is weak enough that a small rotation would be hard to resolve, so we state the contrast conservatively:

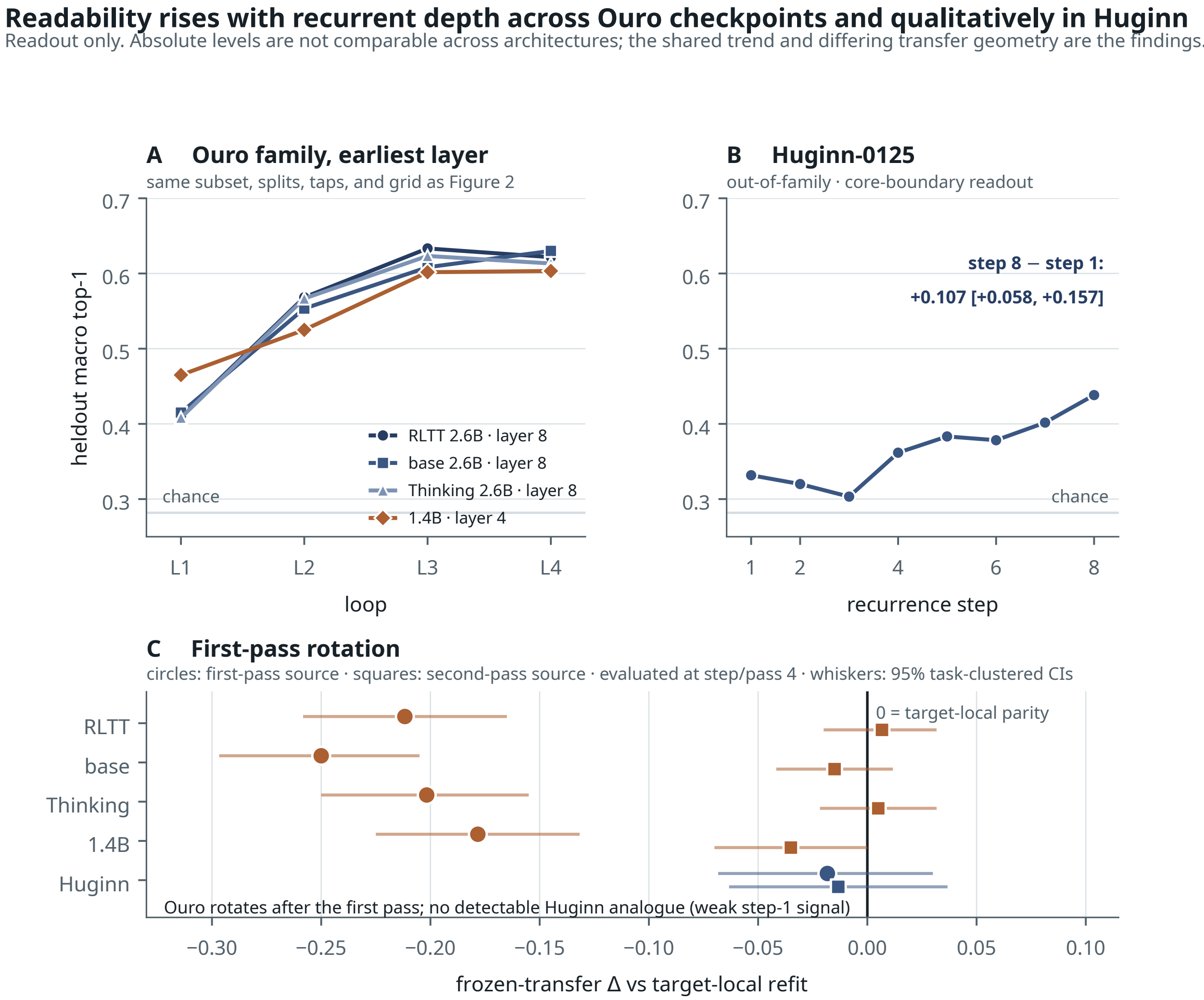


Figure 3. The recurrent-depth readability trend beyond RLTT. (A) Local-refit macro top-1 at the earliest captured layer across loops, for all four Ouro checkpoints (1.4B at its mapped layer 4); the identical sealed subset, splits, and training procedure as Figure 2. (B) Huginn-0125, an out-of-family depth-recurrent architecture, at its recurrent-core boundary across steps 1–8: the trend replicates qualitatively (step 8 − step 1 = +0.107 [+0.058, +0.157]) at a lower absolute level. (C) Frozen-transfer Δ against the target-local refit, first- and second-pass-sourced taps evaluated at pass/step 4; whiskers are 95% task-clustered bootstrap CIs. Every Ouro checkpoint shows a pronounced first-pass rotation (all four first-pass losses exclude zero); no comparable rotation is detected in Huginn, whose step-1 signal is weak and whose transfer losses are mostly unresolved rather than proven absent — across steps, only its step-5 and step-8 losses (−0.060, −0.093) exclude zero.

the depth trend replicates across architectures; the sharp first-pass coordinate rotation is, on present evidence, an Ouro property.

What generalizes and what does not. Three statements now have multi-checkpoint support: recurrent refinement progressively increases candidate-quality readability (all four Ouro checkpoints and, qualitatively, Huginn); the readable direction stabilizes from loop 2 onward in every Ouro checkpoint — in Huginn, step-1-sourced taps transfer at parity through step 4, which is *consistent with* early stabilization but does not establish it, because the weak step-1 signal limits what transfer parity can show; and the readable geometry is shared across the Ouro family (frozen cross-checkpoint transfer at parity). Two statements do not generalize and are scoped accordingly: the *earlier-in-physical-depth* formulation is specific to Ouro's reused-stack architecture (the Huginn probe reads one locus across steps and cannot address it), and the late-layer basis as the strongest readout surface is specific to the 2.6B geometry. Absolute readability levels are not comparable across architectures and we do not compare them.

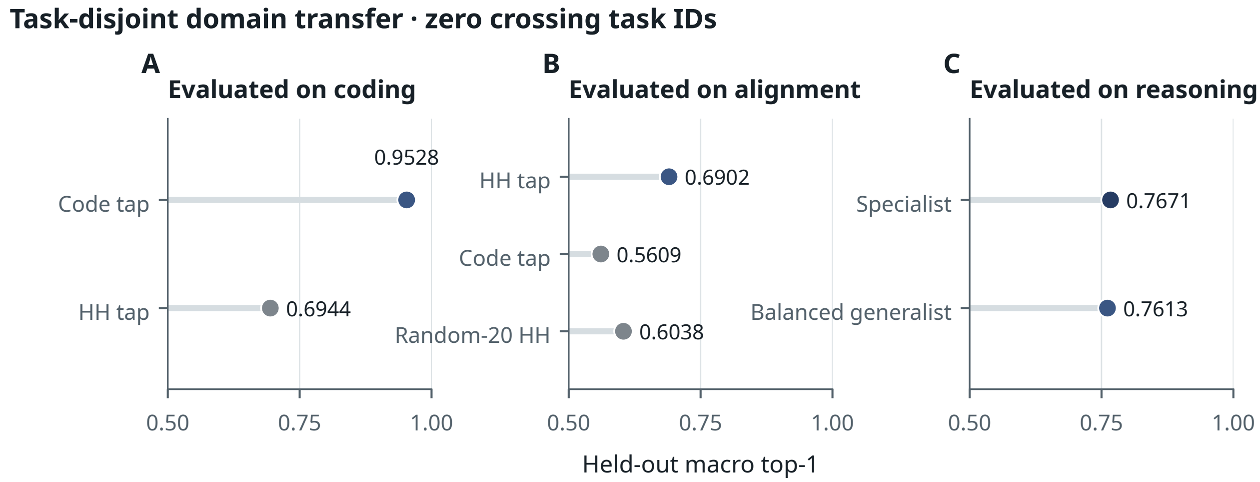


Figure 4. Clean task-disjoint domain transfer (zero task IDs crossing). Specialization pays where distinctions are hard (coding: 0.9528 vs 0.6944), is real but smaller for alignment, and is unnecessary for reasoning, where a balanced generalist matches the specialist.

4.5 Domain structure and the specialist finding

Do the readouts generalize across task domains, or does each domain need its own tap? We answer this on a deterministic **task-disjoint** split (zero task IDs crossing the boundary), with held-out sets ranging from 360 coding groups to 4,748 alignment groups and task-clustered bootstrap intervals.

Tap trained on	Evaluated on	Top-1	Pairwise
Code	coding	**0.9528**	**0.9650**
HH	coding	0.6944	0.8727
HH	alignment	**0.6902**	0.6831
Code	alignment	0.5609	0.5538
Reasoning	reasoning	0.7671	0.8870
Balanced (all-core)	reasoning	**0.7613**	0.8827
Random-20 HH subset	alignment	0.6038	0.5968

Four things follow. **Code specialization is strong**: a code-trained tap reads coding at 0.9528 against a general HH tap's 0.6944 — the largest specialization effect in the study. **Alignment specialization is real but smaller**: HH-trained 0.6902 versus code-trained 0.5609. **Reasoning does not need a specialist**: a balanced all-core generalist (0.7613 / 0.8827) essentially matches the reasoning specialist (0.7671 / 0.8870), so one general head suffices where the distinction is not adversarial. And **data scale matters independently of domain**: a tap trained on a random 20-pair HH subset reads alignment at 0.6038 against the full-HH tap's 0.6902, a gap attributable to training-set size rather than domain mismatch.

The pattern is therefore not "specialists always win." It is that specialization pays where the within-domain distinctions are hard (code, and to a lesser extent alignment) and is unnecessary where a general quality axis suffices (reasoning). The readouts are related but distinct, and no single universal quality head dominates every domain.

These numbers replace an earlier, contaminated domain-transfer study. The prior figures — including a reported 0.986 pairwise on reasoning — came from evaluations with tens of tournaments and from splits that predate the task-disjoint discipline of §3.7; at least one of those datasets had task IDs appearing on both sides of the split. We withdrew them and re-ran the study cleanly rather than report them with a caveat. The qualitative direction survived; the magnitudes did not, and the clean study is both more conservative and more informative than the one it replaces.

A domain that resisted, and what it revealed. Not every domain yielded. A repair pass on science did not bring its tap to parity, and the failure decomposed informatively. Source-specific repair partially cleared it: MMLU **anatomy** reached partial readiness (held-out positive-oracle 0.333, parse 1.0), while **chemistry, physics, and SciQ stayed excluded**, with parse rates collapsing to 0.0. The problem was not "science" as a domain but specific sources — and anatomy's gain remains fragile (3 held-out tasks).

The diagnostic that explains it is worth reporting on its own. A **convergence-hair** probe — originally built as a branch-*merging* mechanism, demoted to diagnostic when it could not clear its safety bar (§7.5) — tracks whether a task's branches are spreading apart or collapsing toward a common continuation. On chemistry and anatomy it fires: the branches converge, and they converge to a *no-good* branch (`CHEM_ANATOMY_NO_GOOD_CONFIRMED`). The failure is therefore not that the tap cannot read quality on these sources — it is that the model does not *generate* a correct branch for the tap to find. **No selector can pick an oracle that is not in the pool.** This is a distinct failure mode from the selection wall of §6 (where the correct branch is present and cannot be committed to), it locates a limit in branch *generation* rather than branch *evaluation*, and it is one more reason the forward-looking work in §12 is training-time rather than a cleverer frozen reader.

4.6 DualAnchor and CoreContent: two constructions, two stages

Two of the role-specialized readouts, DualAnchor and CoreContent, deserve fuller treatment, because they were built by opposite methods for different points in the branch pipeline, and the contrast is itself informative. They are not two attempts at the same tap; they occupy different pipeline stages, and each was constructed in the way its stage demanded.

DualAnchor — built by transplant, for branch survival. DualAnchor is the family that operates *during* the looped branch/prune search: at each loop/layer stage it scores the current candidates, prunes the weak ones, and passes survivors forward, with the goal of *retaining* branches that still contain a correct continuation. It was not trained from scratch. It reuses the two strongest existing content/action directions in the model — the `MIX_CODE_REASONING` and `MIX_OBJECTIVE_ALL` readouts — and grafts branch-validity signal onto them by weight-space transplant, so that two taps carry content *and* branch-viability information at once. This was a deliberate architectural choice: rather than maintaining separate content taps, separate branch taps, and separate bridge taps indefinitely, the program folded content and branch-validity into a single **dual-anchored** pair. The design reached its current form through a long incremental line (old-anchored transplant → two-tap selector → fresh-domain and HH-RLHF comparisons → layer-native re-hosting at 24/36/47 → branch-gap repair → an architecture-looped survival test across all four loops), and its defining property is asymmetric: survival is strong (stage oracle retention 0.9697 task-disjoint, terminal oracle retained 1.0000) while forced terminal commitment *on its own survivor pools* has never been quantified — those figures were withdrawn under audit and not replaced (§6.5). The content-sensitive selection established in §8.6 was measured on a separately constructed pool, not on DualAnchor survivors, so the survival-without-quantified-selection pattern of §6 stands for this family.

CoreContent — built by data, for terminal ranking. CoreContent occupies the *other* stage: it ranks candidates *within* the survivor set that DualAnchor hands off, choosing a final answer rather than managing the search. Mechanically it is the same digest-and-compare engine as the relational preference evaluator of Section 3 — a frozen forward pass, mean-pooled loop states, and an antisymmetric linear tap scoring layernorm($\text{state}_i - \text{state}_j$) $\cdot\, w$ — generalized from HH preference pairs to five content domains (alignment, reasoning, math, coding, logic). Its construction story is the inverse of DualAnchor's. The first version's hand-crafted content taps *lost* to a broad-objective baseline (`mixedhead_MIX_HH_OBJECTIVE`); the diagnosis was not that the tap architecture was wrong but that the per-domain training data was starved (coding had 30 groups, reasoning 5). The fix was data, not design: the v2 refit expanded the starved domains by factors of 27–520×, re-extracted frozen features, and refit the same small taps — at which point a crafted content tap beat the broad-objective baseline on held-out data. Under a **strictly task-disjoint** split (the corrected protocol of §3.7; zero task IDs crossing the boundary), the selected tap reaches held-out macro top-1 **0.6310**, against the broad-objective baseline's **0.5525**. This corrects a previously reported 0.6691, which was measured on a stored split in which 195 task IDs crossed the train/held-out boundary; removing that contamination costs 3.8 points and the readout survives it. One scope note is essential and easy to blur: this 0.6310 establishes task-disjoint candidate ranking on *CoreContent's own five-domain evaluation set* — it is CoreContent's designed *role* to rank within the survivor set DualAnchor hands off, but the clean number was **not** measured on actual DualAnchor survivor pools. Terminal ranking on real survivors is a separate, underpowered question whose earlier "same-survivors" comparison was withdrawn (§6.5) and remains unquantified; we report 0.6310 as candidate ranking, not as validated survivor-set selection. CoreContent is thus the Section 3 method applied to content quality, rescued by scale rather than by a new architecture — and, unlike the linear preference probe, it survives its own leakage audit substantially intact.

Why the difference matters. The two families are best understood not as reading "different aspects of a signal" but as solving the two halves of a search: *keep the right branches alive* (DualAnchor, survival) and *pick the right one at the end* (CoreContent, terminal ranking). That both were needed — and that neither subsumes the other — is part of the paper's larger finding that a readable signal decomposes by role and by pipeline position rather than collapsing into one universal quality score. The construction contrast (transplant onto existing directions vs. data-driven refit of fresh taps) also records a practical lesson: which method works depends on the stage, and a starved data regime can masquerade as an architectural failure. Detailed lineages, expansion figures, and the honest limitations of the CoreContent result — including a constructed-negative inflation and a genuine relevance ceiling — are in Appendix E.

4.7 Is the loop necessary? A non-looped architecture control

The readouts of this section are all measured on a looped model, which leaves open a question the paper must answer before attributing anything to recurrence: **is a looped architecture necessary for quality-relevant information to be linearly readable at all?** We test this directly by repeating the CoreContent-v2 protocol on a conventional, non-looped transformer.

The control backbone is MiniCPM-2B-sft-bf16: a standard decoder transformer with **40 physically distinct blocks** (no recurrent block reuse), hidden size 2,304, explicitly SFT-trained. We extract frozen features at two physical layers (24 and 36) with mask-valid mean pooling, run the identical CoreContent probe grid, and evaluate under a deterministic **task-disjoint** split (zero task IDs crossing the boundary).

Backbone	Held-out macro top-1	Held-out macro pairwise
Ouro-RLTT (looped), corrected task-disjoint	**0.6310**	—
MiniCPM-2B-sft (non-looped), task-disjoint	**0.5680**	0.7237

Per-domain, the non-looped control reads coding at 0.912, alignment 0.693, reasoning 0.520, math 0.389, and logic 0.326 — a domain profile broadly similar in shape to Ouro's, with coding unusually easy and logic hard for both.

What this establishes: a conventional non-looped SFT transformer contains linearly readable candidate-quality information that generalizes across strictly task-disjoint data. **A looped architecture is not necessary for this class of readout to exist.** This is a genuine constraint on the paper's framing, and it is worth stating in the strongest available terms rather than hedging: the readout side of this work is not, on current evidence, a property of recurrence.

What it does not establish. The 6.3-point gap between Ouro (0.631) and the control (0.568) is *not* an architecture comparison. The two models differ in backbone family, pretraining corpus, tuning objective, width, feature dimension, parameter budget, and tap geometry; any of these could account for the difference. We therefore do **not** attribute the gap to looping, and we do not claim the looped architecture is irrelevant either — only that its contribution is unmeasured. A causal architecture study would require matched looped and non-looped models trained on the same data, objective, initialization regime, and compute budget, which we have not run.

The consequence for the paper's argument is a narrowing, and it is worth being explicit about where the loop still does work. The *readouts* (this section, §3, §5) are not shown to require recurrence. The *branching substrate* (§7) is a different matter: the loop is what gives injected branches iterative depth to diverge across recurrent steps, which a single forward pass does not provide (§7.6). And the readout–control boundary (§8) is a claim about the frozen Ouro model specifically. What we can no longer say — and an earlier draft did say — is that looping is what makes process-quality information readable.

4.8 What this establishes: the taps work

The readouts of this section are not marginal effects. Collected in one place, on their own targets and under their own held-out protocols:

Readout	Target	Result
DualAnchor (survival)	keep oracle-containing branches alive through the loop	stage oracle retention **0.9697** (task-disjoint); terminal retention **1.0000**; causal: L47 ablation → 0.0417
CoreContent v2 (terminal ranking)	rank candidate groups across five domains; designed for terminal survivor-set ranking	macro top-1 **0.6310** task-disjoint (baseline 0.5525); coding **0.8956**
S3B2 (generated-branch correctness)	is this generated branch verifier-correct?	AUROC **0.7755**, pairwise **0.7338**
Pre-answer (§5)	will this in-progress computation succeed?	+0.066 AUROC over shortcuts, CI [+0.021, +0.112]

Three of these readouts have substantial effect sizes, and one is supported causally. DualAnchor's layer-47 channel is not merely correlated with branch survival: ablating it collapses oracle retention from 1.0000 to **0.0417** (§E.2.1). That is an intervention, not a probe. It establishes that the channel — the layer-47 locus the tap reads — is load-bearing for retention; it does not by itself prove that the tap's exact learned scalar is the causal variable, only that the locus it reads is one the branch dynamics depend on.

Audit status, stated precisely. The split-protocol failure of §3.7 was not confined to the preference probes. A systematic task-disjoint re-audit of this section's results found a fifth instance of the same error — the branch-survival evaluation had eight task IDs crossing the train/held-out boundary, and 26 of its 48 evaluation tasks were training-side. We re-ran everything that could be re-run. The current status:

- **Re-verified under a zero-crossing task-disjoint split.** CoreContent: 0.6691 → **0.6310** (−3.8 pts). Branch survival: 0.9848 → **0.9697** (−1.5 pts), terminal retention 1.0000 unchanged. Domain transfer: re-run from scratch (§4.5), with the contaminated figures withdrawn rather than caveated. In each case the effect survived decontamination with a modest loss — which is what a real signal does.
- **Established by intervention, not by any split.** Ablating the layer-47 channel collapses oracle retention from 1.0000 to **0.0417**. A leaked split cannot manufacture an ablation effect; this is the strongest single piece of evidence that the locus a tap reads is one the model's own dynamics depend on (that the layer-47 locus is load-bearing, not that the tap's exact scalar is the causal variable).
- **Withdrawn, not repaired.** The terminal-selection figures (§6.5) did not survive. The clean re-run leaves only two reward-diverse tasks — too few to establish a selection effect in either direction — and an "integrated" comparison that had been reported as pairwise accuracy on real survivor pools turned out to be macro top-1 on different candidate groups. Those numbers are retracted and no replacement is claimed.
- **Never exposed to the defect.** The S3B2 detection figures were produced under a leave-one-task-out split grouped by `task_id` with an explicit leakage check that passed (16 groups, 160 candidates; `s3b2_generated_branch_correctness_expanded_2026-06-17`), so they were never exposed to the row-level construction that caused the §3 leak. The N=8 selection slice remains underpowered (§6.4), but the detection protocol itself is source-disjoint by construction.

This matters for the argument that follows. The control results of Part IV are interesting precisely because the readouts are strong: a scaffold that could not tell good branches from bad would fail to steer for boring reasons. The taps can identify which branches contain a correct answer (0.9697 retention, task-disjoint), rank candidates by quality (0.6310), detect generated-branch correctness (AUROC 0.7755), and predict the model's own success before it answers (§5). What those signals buy — validated decision-level gains, and no validated generative control — is Part IV's subject.

5. Strict Pre-Answer Success Prediction

The readouts of Sections 3 and 4 read the model's hidden trajectory as it computes a candidate — which continuation is better, how good its content is, whether a branch is worth keeping. This section makes the sharpest available move: it asks whether the hidden state predicts the success of a computation that has **not yet produced an answer at all**. The pre-answer timing is what forecloses the obvious deflation — the probe cannot be reading a finished artifact, because no artifact exists yet — and that makes this the cleanest instance of the property the paper names, and the anchor of the proto-introspection framing. We report it on two domains: GSM8K arithmetic word problems (§5.1–5.2) and Horizon Logic, a depth-controlled propositional-entailment task with a deterministic verifier (§5.3). In both, the load-bearing quantity is what hidden features add to that domain's own best surface shortcuts.

5.1 Setup and the strict pre-answer cut

We evaluate on GSM8K grade-school math problems (Cobbe et al., 2021): **170 tasks**, **680 examples** total. For each, the model generates a solution trajectory, and a checker labels the final answer verifier-correct or not. The prediction task is: from hidden states extracted **before the answer is produced**, predict whether the eventual answer will be correct.

Everything turns on the **strict pre-answer cut**, so we state precisely what it excludes. A probe with access to the answer token or the numeric gold value would be reading the conclusion, not the process, and the result would be worthless — the finding would reduce to "a model that has written the right answer knows it has written the right answer."

The cut is defined and enforced in code, not applied as post-hoc filtering. Features are taken from the model's loop states over the *pre-answer* span of the trajectory only. Three things are excluded by construction: (i) the **answer region** — every token from the point at which the solution begins committing its final numeric answer onward; (ii) the **gold value** — the reference answer never enters the feature extraction path in any form, so the probe cannot be matching against it; and (iii) the **correctness label**, which is produced by an external checker *after* generation and is used only as a training target for the probe, never as an input. What remains is the trajectory of a computation that has not yet resolved.

This is what makes the result interpretable as *pre-answer*, and it is the assumption we most expect to be challenged. It has been re-verified in code as part of the audit programme described in §3.7, and the raw per-example features, predictions, and labels are preserved (`within_domain_recapture.pt`) so the cut can be inspected directly rather than taken on trust. Full extraction details in Appendix I.

5.2 Hidden states add information beyond shortcuts

A probe on pre-answer hidden features reaches **AUROC 0.745** (95% CI [0.707, 0.783]) for predicting eventual correctness. That number alone proves little: two trivial shortcuts carry some of the same information — solution **length** alone reaches AUROC 0.687, and token **log-probability** (confidence) alone reaches 0.569. The question that matters is whether the hidden state adds anything beyond "longer and more confident solutions succeed more often." It does.

Combining the two shortcuts gives a length+logprob composite at **AUROC 0.731**. The claim rests on what hidden features add *to that composite*: AUROC rises to **0.797**, an increment of **+0.066**. The hidden state contributes information the shortcuts do not contain.

Because the 680 examples are nested within 170 tasks and are therefore *not* independent, the interval must be estimated by resampling **tasks**, not examples; an i.i.d. bootstrap over examples would be anti-conservatively narrow. Under a paired **task-clustered** bootstrap (10,000 draws; each draw resamples 170 task IDs with replacement and retains all four candidates per sampled task), the 95% percentile interval on the increment is **[+0.021, +0.112]** (bootstrap mean +0.0658, SD 0.0235; zero one-class draws), which **excludes zero**. The result is not driven by any single task: leave-one-task-out re-estimation moves the increment only within [+0.056, +0.071], a maximum absolute change of 0.0098 from the full-data value. (A candidate-level bootstrap gives the narrower [+0.032, +0.100], as expected; we report it only as a diagnostic and do not use it for the claim.) Hidden states carry pre-answer information about eventual success that is **not** reducible to length or confidence.

> **Provenance.** The raw per-example predictions, features, and labels are preserved (`artifacts/reports/proto_introspection/within_domain_recapture.pt`: 170 tasks, 4 samples each, 680 examples, 407 positive / 273 negative), and the task-clustered interval above was recomputed directly from them (seed 20260710) rather than inherited. The *original* June interval ([+0.017, +0.114]) was described in its report as a task/group bootstrap, but the preserved analysis code does not contain the paired clustered-delta routine and the original draws were not saved, so its exact execution path is not code-auditable; the interval reported here supersedes it and independently verifies the significance claim.

5.3 A second domain: Horizon Logic

A single domain is a fragile base for the paper's headline claim, and it was the limitation an earlier version of this work named as its most significant. Two candidate second domains had already been tested and rejected at pre-flight for reasons about the datasets rather than the effect (§11). The domain that works is one where the reasoning horizon is a controlled variable rather than an accident of the corpus.

Setup. *Horizon Logic* is a set of synthetic propositional-entailment tasks with **proof depth 2–4** as an explicit reasoning-horizon knob and a deterministic truth-table verifier — a different verifier family and a different reasoning structure from GSM8K's arithmetic word problems. We generate 170 tasks × 4 candidates = **680 candidates** under a single frozen configuration (`max_new_tokens = 448`, temperature 0.7, top-p 0.95, seed 20260724), with a multiple-choice prompt that requires an explicit `FINAL ANSWER:` commitment line. The strict pre-answer cut is the text strictly before the first match of the answer marker, computed at the text level and re-tokenized through the same canonical extractor used for GSM8K, so the answer region and the gold value are excluded by construction exactly as in §5.1 (zero answer-region exclusion violations; mean pre-answer length 149 tokens). Splits are task-level and deterministic (80 train / 31 validation / 59 held-out; zero crossings), with hyperparameters chosen by task-grouped cross-validation on train and validation only and the held-out split opened once. Of the 59 held-out tasks, three have no scorable candidate at all (every candidate malformed), so the analysis basis is 56 tasks. The shortcut controls are the domain's four best surface predictors: pre-cut token count, mean and minimum pre-cut token log-probability, and a hit-max-tokens indicator.

A methodological choice that must be stated before the numbers. 25.0% of candidates are *malformed* — they never reach a parseable commitment, usually because they exhaust the token budget at higher proof depth. Malformed candidates have no valid verifier label, and we **exclude them from the AUROC label set** rather than folding them into "incorrect." Folding them in would let the classifier learn "did this hit the budget" instead of "is the answer correct," since hit-max-tokens and malformedness are strongly associated. They are retained in full in the raw records, counted in the malformed-rate diagnostics, and subjected to two explicit robustness controls below. This leaves **510 scorable candidates** (75.0%), of which 89.0% are correct — a real and substantial class imbalance, and a genuine descriptive property of this domain: once this model commits to an answer at proof depth 2–4, it is usually right.

Result. On the held-out split (171 scorable candidates across 56 tasks, base rate 0.889):

Feature set	Held-out AUROC
Shortcuts only (length, log-probabilities, hit-max)	0.5852
Hidden only	0.7261
Hidden + shortcuts	**0.7261**
Incremental (combined − shortcuts)	**+0.1409**
Paired task-clustered 95% CI on the increment	**[+0.0044, +0.2899]**
Headroom-normalized increment	0.3397

The interval excludes zero. Hidden states carry pre-answer information the best valid shortcuts do not, on a second domain and under the same strict protocol.

One caveat on how to read the combined arm. It matches the hidden-only AUROC exactly, but the two are not the same model: they select different ridge strengths (`l2` 2.0 against 4.0), their held-out predictions differ by up to 7.1×10^{-4}, and 8 of the 171 held-out candidates change rank between them. The AUROC coincidence is a tie in discordant-pair counts — 791 mis-ordered positive/negative pairs under each, over pair sets that are not the same — and not evidence that the shortcuts are redundant. What the fit does show is that the shortcut columns barely enter it: the selected model keeps 24 principal components out of 24,580 standardized dimensions, and those four columns retain **0.011%** of the fitted score's weight (largest per-component share 0.000228), so the projection attenuates them to near-nothing rather than finding them uninformative. On this domain the reported contrast is therefore in substance *hidden-only versus shortcut-only*, and it establishes that hidden states are the more informative of the two rather than that they add on top of shortcuts. The GSM8K arm is unaffected — there the combined fit (0.797) exceeds both the hidden-only (0.745) and shortcut-only (0.731) fits, so the nesting is doing real work — and the Horizon increment itself is unchanged either way, since it compares two AUROCs that are each computed correctly.

How this should and should not be compared with GSM8K. The comparison that carries weight is the *within-domain nested increment*, not the absolute scalar. Horizon Logic is a harder readout domain for this model, and the existing CoreContent evidence already shows logic to be its weakest of five domains (§4.6). The absolute AUROCs differ from GSM8K's and are not required to match:

Quantity	GSM8K	Horizon Logic (pooled)
Shortcut-only / hidden-only / combined AUROC	0.731 / 0.745 / 0.797	0.652 / 0.763 / 0.763
Incremental AUROC	**+0.066** [+0.021, +0.112]	**+0.111** [+0.056, +0.169]
Held-out basis	680 examples, 170 tasks	614 scorable, 201 tasks

Horizon Logic's shortcut baseline is markedly weaker than GSM8K's (0.652 versus 0.731 pooled): with proof depth fixed by construction, "how long the reasoning is" is a much poorer proxy for correctness than it is for open-ended arithmetic word problems. That is part of why the increment is nominally larger here. We do **not** read that as a stronger effect: the point-estimate comparison across domains is descriptive only.

Controls. Zero task crossing; zero answer-region exclusion violations; the gold value excluded by construction; a shuffled-label control at AUROC 0.508, indistinguishable from chance; a maximum leave-one-task-out influence of 0.029 on the combined AUROC, so no single task drives the effect; zero duplicate candidates (full list in Appendix I.2). Two further controls target the malformed-output concern directly. **First**, dropping the three held-out tasks with at least 75% malformed candidates (168 candidates remaining) leaves the increment essentially unchanged: combined 0.7301, shortcut 0.5888, increment **+0.1413** against the full-split +0.1409. The increment survives. **Second** — and reported because it is unflattering — a hidden-only classifier trained to predict *malformedness* rather than correctness reaches AUROC **0.7126**, close in magnitude to the 0.7261 correctness AUROC. The hidden state is doing real work on both questions, and the two are not cleanly separable from AUROC magnitude alone. What the first control rules out is the strongest form of the worry — that the correctness increment is a repackaged malformedness detector — since restricting to low-malformed tasks reproduces the same increment, and since the shortcut model, which has direct access to `hit_max_tokens`, the single most malformedness-correlated variable available, reaches only 0.585 and not 0.71. We state the nuance rather than smoothing it: these two signals likely share some underlying process-quality structure in this domain.

The original cohort's standing, stated plainly. The run above (170 tasks) was positive but imprecise: +0.141 with CI [+0.004, +0.290], a lower bound nearly at zero, and roughly nineteen effective held-out negatives. Worse, a post-publication control that handed the shortcut baseline a fifth feature — the per-task malformed-sibling count, deliberately non-pre-answer and therefore conservative — pushed that interval across zero (+0.122, [−0.014, +0.270]). On the original cohort alone, the result did not survive its strongest adversarial baseline. We report this because it is what motivated the extension, and because the reader should know the v2 estimate was fragile.

Powered extension and independent replication. We therefore generated a prospectively sealed, task-disjoint extension: 510 further tasks (offsets 170–680 of the same hash-ordered pool; the original run used offsets 0–170), same generator, prompts, sampling parameters, seed derivation, strict cut, and deterministic task-level split, with the analysis gated on first reproducing the published numbers exactly. The extension raises the held-out negative class from ≈19 to 84. Results:

- **New cohort alone** (the independent replication): +0.095, CI [+0.038, +0.157] under the published shortcut composite; **+0.091 [+0.035, +0.153] under the adversarial five-shortcut composite** including the malformed-sibling count. Both exclude zero on disjoint tasks the protocol had never touched.

- **Pooled** (best current estimate): **+0.111 [+0.056, +0.169]**; adversarial composite +0.107 [+0.053, +0.164]. Shuffled-label control 0.542; maximum leave-one-task-out influence 0.008; zero task crossings; splits reproduce the deterministic assignment exactly.

The calibration lesson is stated rather than hidden: the v2 point estimate (+0.141) was an overestimate from a wide first interval; the powered estimate is +0.111, and the replication cohort's own +0.095 is the most conservative current reading. What changed in kind, not degree, is robustness — the malformed-sibling baseline that broke the original cohort no longer threatens the result at either the replication or the pooled level.

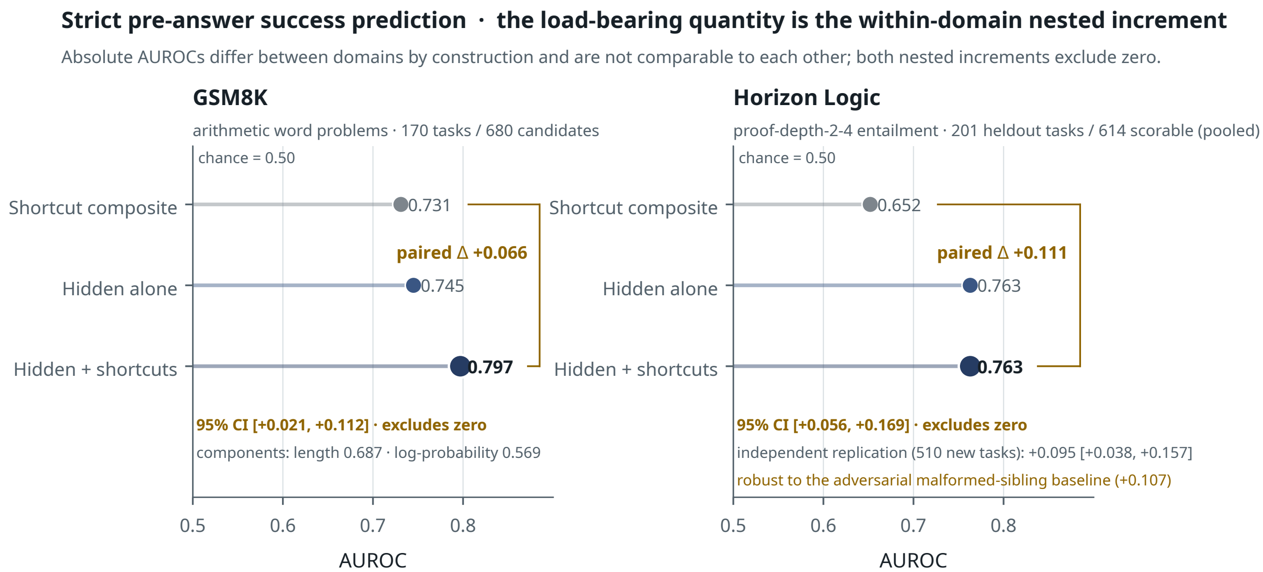


Figure 5. Strict pre-answer prediction on two domains. In each panel the load-bearing quantity is the within-domain nested comparison — what hidden features add to that domain's own best shortcut composite — not the absolute AUROC, which differs between domains by construction. GSM8K: +0.066, task-clustered 95% CI [+0.021, +0.112], 170 tasks / 680 candidates. Horizon Logic, pooled over the original cohort and the prospectively generated extension: +0.111, CI [+0.056, +0.169], 201 held-out tasks / 614 scorable candidates, 84 held-out negatives; the extension cohort independently replicates the increment (+0.095 [+0.038, +0.157]) and the result is robust to the adversarial malformed-sibling shortcut (+0.107). The two point estimates are not comparable to each other.

5.4 Scope

The effect is incremental and it is now measured on two domains. Both facts bound the claim, and neither undermines it.

Incremental is the right frame, not a weakness: length and log-probability are genuine predictors of success, they are included as explicit controls, and the hidden state's contribution is what it adds beyond them. A probe that merely re-read confidence would show no increment. This one does, with an interval excluding zero under the correct clustered test. The increment is also not merely a scoreboard quantity: converted into a frozen abstention policy, the same scores buy a validated selective-accuracy gain at equal coverage on both domains (§8.6, C1).

What that increment consists of is open. Candidate process-level correlates include the consistency of the intermediate calculation across loop iterations, the sharpness or stability of the evolving representation, and the convergence behavior of the loop trajectory. Separating these — and separating them from residual decoding correlates such as token-position variance not fully absorbed by the length control — is future work. This paper establishes that a shortcut-independent pre-answer signal exists, not which feature of the computation carries it.

Two domains is better than one and is not generality. The pre-answer protocol now has positive evidence on arithmetic word problems and on depth-controlled propositional entailment, with different verifiers and different reasoning structures; both intervals exclude zero, and the logic result now carries an independent task-disjoint replication and survives its strongest adversarial baseline. But two further candidate domains were tested and rejected at pre-flight for dataset reasons (SVAMP front-loads its answers; Hendrycks MATH degenerates into a length predictor once truncation is handled — §11), and nothing here speaks to domains without a deterministic verifier.

One checkpoint, and an attempted second. Cross-checkpoint pre-answer transfer remains unresolved. On Ouro-2.6B-Thinking — a prospectively sealed replication with the same tasks, prompts, sampling, seed derivation, strict cut, and (deliberately, sealed before generation) the same 448-token budget — the strict pre-answer increment was estimated too imprecisely to support either replication or absence: +0.027 under the adversarial composite, 95% CI [−0.13, +0.21]. The interval includes both a meaningful negative effect and an effect comparable to that observed on RLTT. The cause of the width is mechanical: at the sealed budget the Thinking variant's longer reasoning style truncates 45.6% of candidates into malformedness (RLTT: 25.0%), halving the scorable pool (130 held-out candidates, ≈20 negatives). The sealed artifact label `THINKING_PREANSWER_NULL` is retained for provenance, but "null" overstates the statistical conclusion: the estimate is **inconclusive, underpowered**. A separate exact-compute experiment found no benefit from tap-gated recurrent-depth allocation on Thinking (§8.6, C2); that experiment tests allocation utility rather than pre-answer readability and therefore does not update the pre-answer estimate.

That powered re-run has since been completed, and it sharpens the estimate without resolving it. On **900 fresh task-disjoint tasks** (offsets [1500, 2400), 3,600 candidates, 544 held-out scorable, **99 held-out negatives** against the pre-registered target of 80–100), sized by a simulation fixed before generation and analysed only after a Stage-A gate reproduced the pilot's published numbers to 10^{-9}, the new-cohort increment is **+0.042, 95% CI [−0.017, +0.104]** under the same adversarial composite. The sealed verdict is UNRESOLVED. The interval is roughly three times tighter than the pilot's ([−0.13, +0.21]) and the point estimate rose from +0.027 to +0.042, but it still contains zero: the run neither replicates the effect on Thinking nor excludes it. Three further facts are recorded rather than smoothed. The pooled estimate — a precision figure, explicitly *not* an independent replication — is +0.050 [−0.004, +0.106], with a lower bound essentially at zero. Practical equivalence was not established either: the 90% interval [−0.008, +0.093] exceeds the ±0.05 margin declared before generation, which the sealed power analysis had predicted would be unreachable at this sample size. And the design had **0.816 power to detect an assumed +0.095 increment** — the pre-registered power target, taken from the RLTT new-cohort value under the *published four-shortcut* composite (the matched adversarial figure there is +0.091). The observed 95% interval nevertheless still includes +0.095, so an RLTT-sized effect is **not formally excluded**: the run failed to replicate it, which is evidence against an effect that large without ruling it out, and it leaves smaller positive effects compatible with the data. On this checkpoint malformedness is also more readable than on RLTT (hidden-only malformed-vs-clean AUROC 0.841 against 0.713), though dropping the 48 held-out tasks that are at least 75% malformed leaves the increment slightly *higher* at +0.052, so the estimate is not an artifact of malformed-heavy tasks. We did not extend the budget post hoc, and no verdict was re-labelled after seeing the interval.

Finally, this result says nothing about whether the model *uses* the signal internally — a question we do not test. Part IV shows only what *our* interventions built on it achieve: decision-level gains, and no validated generative control. The claim here is narrower and cleaner: the information is present, externally readable, and available before the answer exists.

6. Generated-Branch Correctness and the Commitment Gap

Section 5 established that hidden states read the quality of the model's own computation. This section is the hinge on which the paper turns from readout to control. It shows that even when a quality signal is clearly *readable*, converting it into a **commitment** is a separate and much harder problem: generated-branch correctness is decodable from hidden features, and branch *survival* — keeping the correct branch alive in the pool — works very well, yet converting either into reliable forced commitment on *substance* has not been demonstrated. The expanded evidence sharpens rather than softens that statement. On an expanded pool, forced terminal selection does beat a properly matched random baseline — and decomposing where its margin comes from shows that most of it is avoidance of malformed, non-committing candidates rather than discrimination among well-formed ones. Readable, and even retainable, becomes selectable when the choice is about form; whether it becomes selectable when the choice is about content is, at this point in the story, the open question — a prospectively constructed pool answers it in §8.6. The rest of the paper is about that gap and how deep it goes.

6.1 The task

We generate pools of candidate branches for reasoning tasks, label each branch verifier-correct or not, and ask two questions of a hidden-feature reader (the S3B / S3B2 setting; Appendix E). First, **detection**: can the reader decode whether a given generated branch is correct? Second, **selection**: forced to commit to a single branch from a pool, can the reader reliably pick a correct one, against a properly matched random baseline?

6.2 Survival works: the branch-retention scaffold

Branch *survival* — keeping a correct continuation alive in the pool while pruning weak branches — is the one part of this pipeline that works well, and it is verified under a zero-crossing task-disjoint split.

Metric	Clean (task-disjoint) result
Stage oracle retention (DualAnchor)	**0.9697**
Terminal oracle retention	**1.0000**
Scaffold top-4 retention	**1.0000** (52 groups / 26 tasks)

These figures correct an earlier contaminated evaluation. The original 0.9848 stage-retention number was measured on a split with eight task IDs crossing the train/held-out boundary, and 26 of its 48 evaluation tasks were training-side; a clean re-run with zero crossings gives 0.9697 — a 1.5-point drop, the signature of a real effect surviving decontamination rather than an artifact dissolving (§3.7 documents the class of error, and §4.8 the audit tiers).

The survival claim does not rest on held-out accuracy alone. It is established causally: **ablating the layer-47 channel the survival tap reads collapses oracle retention from 1.0000 to 0.0417**. A leaked split cannot manufacture an ablation. The scaffold is reading something the branch dynamics genuinely depend on.

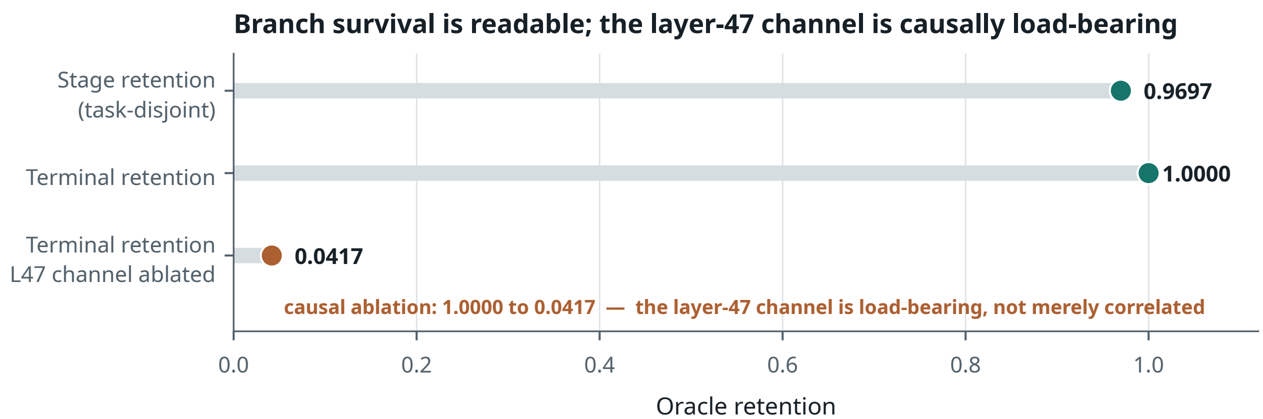


Figure 6. Survival works under a zero-crossing task-disjoint split, and the layer-47 locus is causally load-bearing for retention: ablating the channel the tap reads collapses retention to 0.0417. This does not identify the tap's exact scalar as the causal variable.

What survival does *not* deliver is commitment, and the rest of this section is about that gap.

6.3 Correctness is readable

Detection succeeds. An L2-regularized logistic reader over hidden features reaches **AUROC 0.7515** with pairwise accuracy **0.6835**; an expanded hidden-ridge reader reaches **AUROC 0.7755** with pairwise accuracy **0.7338**.[1] Metadata-only controls (features derived from surface properties rather than hidden states) remain weak, so the signal is in the hidden geometry, not in incidental artifacts. Generated-branch correctness is genuinely, non-trivially decodable — a fourth role-specialized readout to add to those of Section 4.

6.4 The original selection slice, and why it could not settle the question

Selection does not follow from detection. The first setting in which we asked the question directly is reported here as it stood, because it is the setting in which correctness is most cleanly decodable and because it defined the measurement problem that §6.5 finally has the power to answer. Its own verdict is unchanged and it is superseded, not corrected: the slice was too small to decide anything.

The oracle-present selection task comprises **N = 8 task groups** (drawn from a full pool of 160 candidates across 16 groups; groups with no correct branch cannot contribute an oracle-conditioned selection score). Under forced top-1 choice, the expanded reader selects a correct branch in **5 of 8 groups (0.625)**, and the weaker reader matches it — the substantially higher detection AUROC (0.7755) buys no improvement in forced-choice selection.

The baseline requires care, and two earlier figures were wrong. The eight groups contain ten candidates each, with correct-branch counts (7, 1, 3, 2, 4, 2, 8, 2). Uniform random choice within each group therefore succeeds with matched expectation **0.3625** (2.9 of 8 groups) — *not* the 0.5833 that earlier tables imported from the aggregation-mismatched S3B1

[1] The S3B2 detection figures (AUROC/pairwise) are pinned by the pending-items live-repo pass (`artifacts/reports/paper_verification/pending_items_resolution_20260703_214531.*`). The **0.5833** figure sometimes reported alongside them is **not** a matched S3B2 control, contrary to an earlier draft note: it is the macro-average over three oracle-present domains from the S3B1 corrected transfer, $(0.5_{\text{math}} + 0.75_{\text{reasoning}} + 0.5_{\text{logic}})/3 = 0.5833$ (`utilities/tests/manual/mpn_s3b1_loop_pool_transfer.py`), aggregated differently from the pool-weighted selection fraction in §6.4. It is retained only for continuity with prior tables and is explicitly not used as the matched baseline for the selection claim.

macro, and not the 0.625 a subsequent draft mistakenly asserted. The selector's 5/8 does **exceed** the matched-random point expectation. But the exact Poisson-binomial probability of at least five successes under random choice is $p = 0.087$, which does not clear significance at the 0.05 level: with eight groups, the test is underpowered.

The conclusion is therefore neither "selection works" nor "selection is at chance," but that **reliable forced selection is not established on this slice**: correctness is clearly decodable (pairwise ≈ 0.73, AUROC ≈ 0.78), the selector points in the right direction, and eight groups are too few to conclude anything firmer. High-margin abstention does not rescue forced top-1 either.

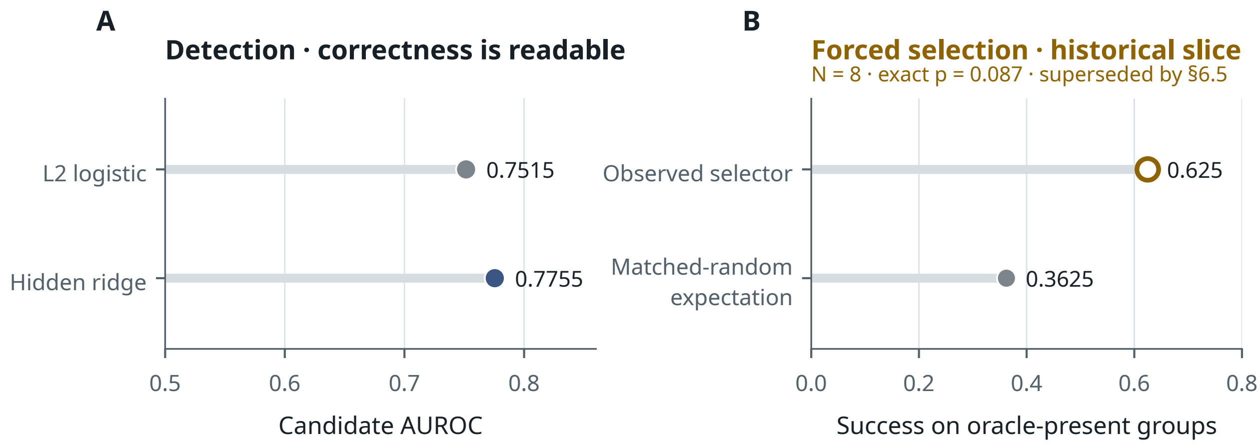


Figure 7. The commitment gap in its sharpest local form. Left (current): generated-branch correctness is decodable at AUROC 0.7755. Right (historical): forced top-1 selection on the original N=8 oracle-present groups exceeds the matched-random expectation without reaching significance (exact p = 0.087). The right-hand panel is retained as the historical slice that motivated the powered evaluation of §6.5, which supersedes it.

We report this slice because it is the setting in which correctness is most directly decodable, and because its detection/selection gap motivated first the survivor-set experiments — whose quantitative claims were withdrawn under audit — and then the powered evaluation of §6.5. The honest position on *this* slice is unchanged: eight groups establish neither a working terminal selector nor a quantified deficit.[2]

6.5 A powered terminal-selection evaluation, and what its margin is made of

The measurement problem of §6.4 was power. The evaluation reported here removes it, and the answer it returns is not the one the earlier sections anticipated: on an adequately powered pool, forced terminal selection **does** beat a properly matched random baseline, comfortably. Decomposing where that margin comes from is what turns a headline into a scientific claim, and the decomposition is unflattering enough that we treat it as part of the result rather than as a caveat appended to it.

The pool. The Horizon Logic generation of §5.3 supplies 170 task groups of four candidates each, task-disjoint by construction and reward-diverse by generation rather than by selection. Of the 59 held-out groups, 15 are all-correct and 5 all-wrong; these cannot support discrimination and are excluded from the endpoint, leaving **39 informative groups** — above the 25-group minimum this evaluation pre-registered, and an order of magnitude past the 2 reward-diverse tasks the clean survivor re-run had left. Unlike the pre-answer analysis of §5.3, malformed candidates are **not** excluded here: under forced terminal choice a non-committing candidate is a genuine failure mode and is scored as incorrect like any other. The selector reads a full-candidate ("terminal") pooled hidden feature over prompt plus complete generated text — no leakage constraint applies, because terminal selection scores completed candidates — and is the same low-capacity family used throughout (standardize → PCA → L2-logistic), with hyperparameters selected by task-grouped cross-validation on training and validation groups only.

The endpoint. The matched-random null is exact rather than nominal: each group i contributes its own success probability $p_i = c_i/n_i$, and the null distribution of the total is the Poisson-binomial over those 39 probabilities, all of which are preserved in the artifact.

[2] The matched-random baseline is analytic, not an artifact lookup: with per-group correct counts $(7, 1, 3, 2, 4, 2, 8, 2)$ out of ten candidates each, the expected random hit rate is $\frac{1}{8}\sum_i c_i/10 = 0.3625$, and the exact Poisson-binomial $P(\geq 5 \text{ hits}) = 0.0869$ (the probability of exactly five is 0.0717). Two prior figures are hereby corrected: the 0.5833 (an S3B1 three-domain macro, aggregated differently) and a later erroneous claim that matched random equalled the selector's 0.625.

Quantity	Value
Informative held-out groups	39
Observed forced top-1 successes	**34 / 39 = 0.8718**
Matched-random expected successes	23.0 / 39 = 0.5897
Paired difference	**+0.2821**
Exact Poisson-binomial $P(\geq 34)$	$\mathbf{2.44 \times 10^{-5}}$
Task-clustered bootstrap 95% CI on the difference	**[+0.1667, +0.3910]**
Pairwise ranking accuracy (122 pairs)	0.7623
Mean reciprocal rank / top-2 / top-3 oracle retention	0.9274 / 0.9487 / 1.0000

All integrity controls pass: zero task crossing between selector training and evaluated groups, zero duplicates, a shuffled-score control at exactly chance (paired difference 0.000), zero candidate-order-invariance failures on a twenty-group spot check, and no held-out-label tuning.

Where the margin comes from. A control selector using **no hidden state at all** — generated token count, whether the final-answer marker was found, whether the token cap was hit, and the pre/generated token ratio — reaches **35/39 (0.8974)**, a paired difference of +0.3077. It is nominally *better* than the hidden-state selector.

The two selectors are not statistically distinguishable, and this is worth stating formally rather than leaving at the bare counts. They are evaluated on the *same* 39 groups, so the comparison is paired and McNemar's exact test applies. Writing b for the groups the hidden selector wins and the shortcut selector loses and c for the reverse, the concordant groups cancel and $b - c = 34 - 35 = -1$, so $c = b + 1$ whatever the split. For every admissible value of b the exact two-sided McNemar p-value is **1.00**: with an odd number of discordant pairs split as evenly as one apart, half the binomial mass lies at or below $\min(b, c)$ by symmetry. The stored artifact preserves per-group matched-random probabilities and the totals but not the per-group outcomes, so b itself is not recoverable — and the conclusion does not depend on it. Access to the hidden state buys no detectable advantage over four surface features on this pool.

The reason is visible in the composition of the pool, which we can decompose exactly because every candidate's malformedness and correctness are recorded:

Stratum of the 39 informative groups	Groups
Contain at least one malformed (non-committing) candidate	**34 (87.2%)**
— of which: every well-formed candidate is correct, so avoiding the malformed one always wins	**26 (66.7%)**
— of which: also contain a well-formed but incorrect candidate	8 (20.5%)
Every candidate well-formed — a pure content-quality choice	**5 (12.8%)**

No malformed candidate is ever scored correct, so malformedness is a sufficient reason to reject and a selector that perfectly detected it would already win 26 of 39 groups outright. That is most of the observed margin, and it is exactly what a four-feature surface selector can compute. On the five groups where every candidate is well-formed and the choice is genuinely about content, the hidden-state selector picks correctly **5 out of 5** against a matched-random expectation of 3.75 — directionally encouraging, and far too small to carry any claim on its own. We report it descriptively and decline to test it.

The two conclusions this supports, stated separately. *Format-sensitive terminal selection is established on this pool.* Forced choice significantly exceeds an exact matched-random null under a task-disjoint, reward-diverse, adequately powered evaluation, and every control passes. That is a real result, and it is the first quantified terminal-selection result in this project. *Content-sensitive commitment among valid competing candidates is unresolved on this pool.* The hidden-state selector is not shown to add anything beyond malformed-avoidance on this domain: a hidden-free control matches it, most informative groups are decided by non-commitment, and the subset that would isolate content discrimination has five members. On this evaluation alone, the narrow claim "forced selection beats matched random" is earned and the broad claim "the model's hidden states let you pick the better answer" is not. Section 8.6 returns to the unresolved half with a pool prospectively constructed to isolate it — every candidate well-formed by construction — and resolves it.

What remains withdrawn. This result does not rehabilitate the earlier survivor-set figures, which measured something else on contaminated data and stay retracted without replacement (Appendix E.3): the best-survivor-versus-final-reward gap, and the integrated CoreContent-versus-DualAnchor comparison, whose clean re-run leaves nine tasks of which only two are reward-diverse. Nor does the present result transfer to them: it evaluates a domain-matched selector

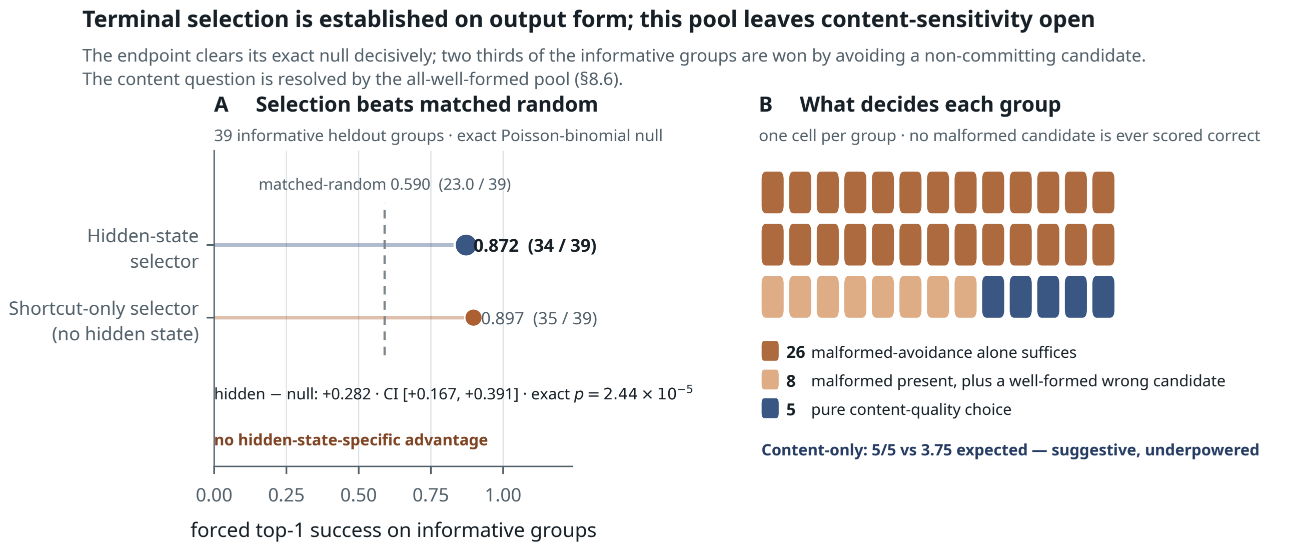


Figure 8. The expanded terminal-selection evaluation and its decomposition. (A) Observed forced top-1 against the exact Poisson-binomial matched-random expectation; the hidden-state selector clears it decisively, and a shortcut-only selector with no access to hidden state clears it by slightly more. (B) What each of the 39 informative groups actually requires: in 26 of them, avoiding the malformed candidate is by itself sufficient; 8 more contain both a malformed and a well-formed-but-incorrect candidate; only 5 are pure content-quality choices. The n = 5 subset (5/5 against 3.75 expected) is shown at its true size and is not a powered result.

trained on this pool, not the frozen S3B2/DualAnchor/CoreContent selectors out of domain, which were not re-scored here.

A measurement trap worth restating, since this evaluation is built to avoid it. Aggregate forced top-1 scores flatter themselves on tie-heavy pools, where many candidates are equally good and any choice scores well. That is why the endpoint here is computed only on informative groups, why all-correct and all-wrong groups are excluded rather than averaged in, and why the null is the exact per-group Poisson-binomial rather than a flat $1/n$. The malformed decomposition above is the same discipline one level deeper: the informative groups are themselves not all equally hard.

The connection to Section 3, stated as hypothesis. Forced top-1 is an *absolute, listwise* commitment, and §3.2 finds that preference is decoded more accurately relationally (0.5653) than pointwise (0.5418). That a model ranks pairs better than it scores singletons is a plausible mechanism for why comparison succeeds while absolute commitment on substance does not — and it is consistent with every listwise arbiter in the lineage failing to improve (Appendix E.3). We flag it as a hypothesis the current data motivates but does not test.

The scoped conclusion, as it stood on this evaluation: **survival is solved, detection is solved, format-sensitive commitment is established, and content-sensitive commitment is unresolved.** The original mixed-quality pool established format-sensitive selection but could not isolate substantive commitment. A prospectively constructed all-well-formed evaluation (§8.6) now resolves that question: terminal selection remains significantly above a matched-random null when every candidate is parseable and committed, establishing content-sensitive selection. A surface-only selector also performs above random there, and the hidden-state selector's numerical advantage over it is not statistically resolved at the present sample size — the two findings are reported separately in §8.6, and the second does not weaken the first.

6.6 Why this is the hinge

This section is where the paper turns, and the turn survives the audit even though several of its numbers did not.

The chain is: the model's hidden states let an external reader **keep the correct branch alive** (0.9697 retention, verified clean, and causally established by ablation) and **recognize a correct branch when they see one** (AUROC 0.7755). Under forced commitment, that reading converts into a real, significant selection gain — and on this pool the gain is carried by a property of the *output form*, not demonstrably of the content. At the time of the evaluation above, no mechanism in this project — not a listwise arbiter, not a tie-aware ranker, not a merged tap, not the S3B2 selector, and not the powered selector of §6.5 — had been shown to convert the readable signal into reliable commitment when the competing candidates are all well-formed and differ only in whether they are right. The all-well-formed evaluation of §8.6 shows that this conversion does in fact succeed when the pool is built to demand it (27/32 against a 64.8% matched-random expectation); what remains unresolved there is only whether hidden-state access adds increment beyond surface statistics.

That history is worth preserving because of where it moves the boundary. The favorable setting — an external reader, handed correctness labels to train on, asked only to choose among candidates the model already produced, with no demand on the frozen model whatsoever — is now a setting where substantive conversion is established. The demands that remain unmet are the harder ones: when the frozen model must itself be steered, branched, or given its compute budget by the readout, no validated gain has been shown. Sections 7 and 8 test exactly that: first by building machinery through which those signals *could* be acted on, validated by bit-exact identity, then by showing that no frozen intervention through that machinery produced a validated substantive capability gain under the conditions tested, and finally (§8.6) by locating the boundary precisely — between decision-level use of readable states and generative control over the underlying computation.

PART III **Executable Substrate**

7. Internal Branch/Carry/Prune Machinery

Before asking whether the readable signals can be *acted on*, we build the machinery that would make acting possible — and that machinery turns out to be the part of this project no audit touched.

This section describes an executable internal branching substrate for Ouro-RLTT: autoregressive, branch-specific key/value-cache carry across a 192-slot recurrent cache, validated through a six-level correctness ladder; a bit-exact suffix-recompute splice that cuts up to 88% of per-branch compute; and a live fork/carry/prune/reorder scaffold. Two things make it more than apparatus. First, it is validated by **exact identity and negative controls**, not by output plausibility — which is what makes the negative results of Part IV credible rather than attributable to a broken scaffold. Second, exact-equivalent branch-specific cache manipulation inside a *looped* transformer is not a port of standard incremental decoding, and we present it as a systems contribution usable independently of the introspection question this paper studies. We claim no capability gain here: the substrate is validated for *correctness and executability*, and Part IV is where we show that correctness is not enough.

7.1 What makes branch-carry hard in a looped model

The problem is not obviously hard until one tries it, so it is worth stating what the substrate must do that a standard incremental decoder does not.

Ouro's cache (`UniversalTransformerCache`) is indexed by **192 distinct slots per position** — slot $= u \cdot 48 + \ell$ for loop $u \in \{0..3\}$ and layer $\ell \in \{0..47\}$. Prefill populates all 192; each decode step appends one token to *every* slot; a batch reorder must permute the batch dimension of every populated slot. A **branch** is a distinct trajectory through this structure, and carrying one autoregressively means keeping its own `past_key_values`, `cache_position`, `attention_mask`, `position_ids`, `generated_ids`, and lineage aligned across every decode step, for every one of the 192 slots, without leaking into any sibling branch.

This is a different and strictly harder problem than the prompt-only layer carry used by our offline probes (which run with `use_cache=False` and can afford to recompute). Generation-time, branch-specific KV carry is what a *live* scaffold requires, and it is where the correctness burden actually lives: a single misaligned `position_ids` row or an incorrectly reordered slot produces plausible-looking text and silently invalid branches. We therefore validated it as a ladder of increasingly demanding correctness properties rather than by inspecting outputs.

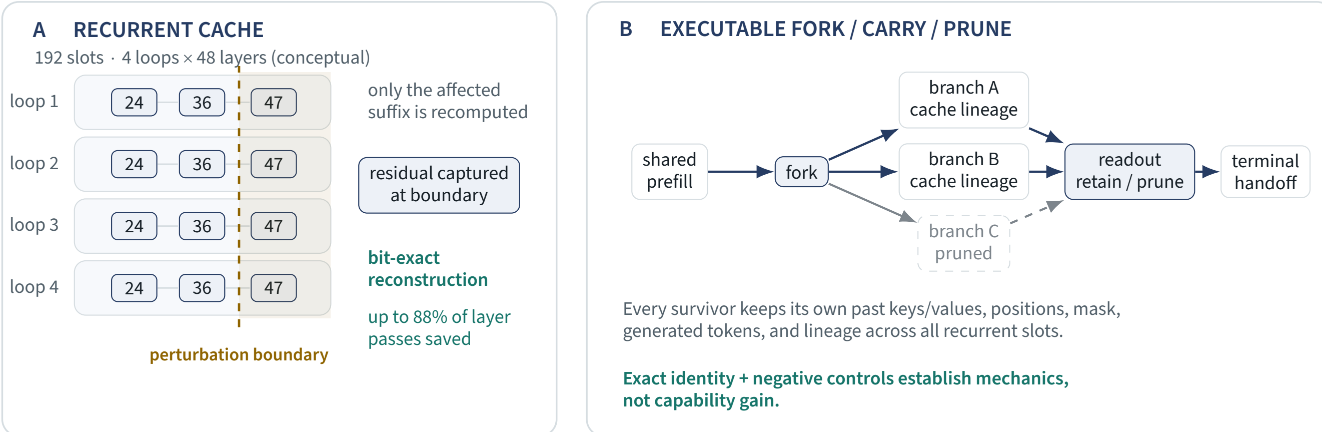


Figure 9. Left: the 192-slot recurrent cache (4 loops × 48 layers per position), with the tapped readout layers marked. Right: the live scaffold — one shared prefill, branch-specific cache lineages forked at a loop/layer boundary, carried, pruned, and handed off to terminal ranking.

7.2 A validation ladder, not a smoke test

The six validated levels are labelled V0–V5 to keep them distinct from the L1–L4 loop indices used elsewhere in the paper.

Level	Property established
V0 cached decode	matches full recompute (prefill bit-exact; decode within bf16 drift)
V1 token-boundary fork	$K = 2/4/8$ independent branch caches, **no cross-branch contamination**
V2 batched branches	batched ≡ independent ≡ full recompute
V3 prune / reorder	survivor subsets and order changes ($8 \to 4 \to 2$, $8 \to 3$, $4 \to 1$) keep lineage aligned
V4 current-token perturb	injection at layers 24/36/47, loop-targeted, carries correctly via the branch cache
V5 prompt-internal perturb	branch-specific cache required — **negative control**: withholding it diverges to RMS ≈ 3.0

All six pass. (A seventh level, V6, tested a first-attempt partial splice: its slot-boundary logic was valid but it delivered no compute saving, and it was superseded by the v2 splice of §7.3.) Two properties are worth drawing out. **Correctness is established by exact identity, not similarity**: a zero-perturbation fork reproduces the reference **bit-exactly at prefill (RMS 0)**, with only small bf16 drift during cached decode (RMS ≈ 0.05–0.2, max-abs < 1.0) — the expected numerical signature of cached-versus-recomputed key/value paths, not an error in the branch logic. And **the negative control matters as much as the positive**: forcing a branch to proceed *without* its proper cache lineage diverges to RMS ≈ 3.0, which is what tells us the carried cache is load-bearing rather than incidental. Correct carry gives exact reproduction; absent carry gives large divergence. A mechanism that only ever produced plausible text would have passed neither test.

Batched decode required one non-obvious fix: left-padding a batch of branches breaks unless each row is given explicit `position_ids` (RoPE's relativity makes a single-prompt left-pad shift harmless, but a *batched* one is not). We record it because it is exactly the class of bug that produces silently wrong branches rather than crashes.

7.3 The compute-saving splice, and the obstacle it had to clear

Naively, evaluating a perturbed branch means re-prefilling the whole prompt for that branch: K branches, K full prefills. The obstacle to doing better is specific and easy to miss. **The KV cache stores keys and values, but not the inter-layer residual stream.** A perturbed branch therefore cannot simply resume from the shared cache — the residual it would need to continue from was never stored, and reconstructing it appears to require the forward pass one is trying to avoid.

A first attempt (v1) validated the slot-boundary logic — the copy-affected cache reproduced the full cache bit-exactly — but delivered **no compute saving at all** (`PARTIAL_SPLICE_DIAGNOSTIC_ONLY`): knowing *which* slots change does not help if you must still re-prefill to obtain the residual they depend on. The saving required a second idea.

The solution is to capture, during a single shared-prefix prefill, the residual hidden state *at the perturbation boundary* (the output of loop u, layer ℓ). For an additive boundary perturbation the perturbed residual is then $H_{\text{boundary}} + \delta$ —

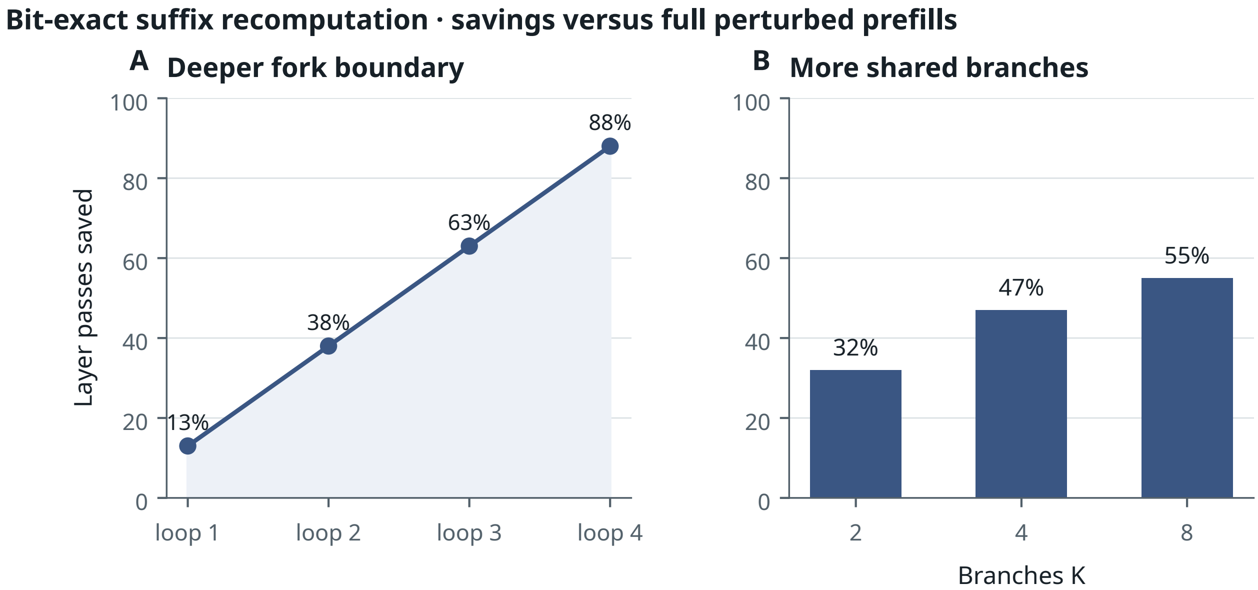


Figure 10. Compute saved by the bit-exact suffix-recompute splice, relative to K full perturbed prefills: by fork boundary (left — deeper forks recompute less) and by branch count at a fixed boundary (right). Savings amortize over K ≥ 2.

reconstructible **with no forward pass at all** — and only the *suffix* need be recomputed: the remaining layers of loop u, then loops $u + 1$ onward. The implementation is test-only orchestration over the model's layers, rotary embeddings, norm, and head: no weight edits, no permanent model surgery.

Establishing which cache slots this actually touches required an empirical result we did not anticipate: perturbing a layer's *output* leaves that layer's own boundary slot unaffected, so the first affected slot is $(u, \ \ell + 1)$, and the changed set is exactly the `downstream_only` prediction. Over-sharing an affected slot diverges — the negative control that pins the boundary.

The result is **bit-exact and cheap**. The spliced branch cache matches a full perturbed-prompt reference across all 192 slots, with prefill logits at RMS 0 and the continuation identical token-for-token — validated single-branch, multi-branch ($K = 2/4$, independent storage, no contamination), batched, and under prune/reorder. The savings grow with fork depth: recomputing only the affected suffix saves **13% / 38% / 63% / 88%** of per-branch layer passes for fork boundaries at loops 0–3, and **32% / 47% / 55%** at $K = 2/4/8$ branches (fork at loop 2, layer 24). Savings amortize over $K \geq 2$; at $K = 1$ there is nothing to share.

We position this carefully against a well-developed systems literature. Branch-forking and prefix-sharing over KV caches are standard: PagedAttention (Kwon et al., 2023) forks with copy-on-write pages, SGLang's RadixAttention (Zheng et al., 2024) shares prefixes across a radix tree of branches, and SpecInfer (Miao et al., 2024) verifies a token tree over a shared cache with tree attention. Reuse *across recurrent steps* is also known: depth-recurrent models attend to KV entries generated by the same projections at every iteration (Geiping et al., 2025; Zhu et al., 2025). A third, more recent line changes the cache itself: the Memory-Efficient Looped Transformer (MELT; Conchello Vendrell et al., 2026) replaces Ouro's per-loop KV entries with a single per-layer cache updated across loops by a learned gate, cutting cache memory from O(layers × loops) per token to O(layers) — a roughly four-fold KV reduction on an Ouro-1.4B-Thinking fine-tune at near-parity reasoning performance. What we have not found in this literature is the combination specific to this substrate — branch-specific carry plus a **residual-capture suffix splice** that reconstructs a branch perturbed *mid-computation* (at an interior loop/layer boundary) bit-exactly over the loop×layer recurrent cache, without the re-prefill that a stored-K/V-only cache would otherwise force. It is not a port of standard incremental decoding, and it is usable independently of the readout–control question this paper studies — which is why we present it as a contribution in its own right rather than as apparatus. We do not claim a performance advantage over these systems; the comparison is one of mechanism, and the substrate's role here is as a validated instrument, not a faster search. Two of our design commitments interact with MELT-style caches directly: our branch lineage and splice operate on the full loop×layer cache, so a gated shared cache would shrink the carried state four-fold but also collapses the per-loop KV distinction the splice's interior fork boundaries rely on; porting the substrate to a MELT-style cache is a concrete open engineering question. The *readouts* of §4–5 are unaffected in principle — they tap per-loop residual states, which MELT's cache compression preserves.

7.4 The live scaffold

These pieces compose into a live **branch/carry/prune/loop-back** scaffold: the system forks candidate branches at chosen boundaries, carries their caches forward, scores and prunes survivors, reorders the live set, and can loop back to earlier boundaries to re-explore. This is the apparatus required to *act* on process-quality readouts at run time — to branch where a readout is uncertain, prune branches a readout deems weak, and commit where a readout is confident. Having built it, we can ask the paper's central question sharply: given that the readouts exist (Part II) and the machinery to act on them exists (this section), does frozen intervention through that machinery actually help? Part IV's answer: at the decision level, yes; at the level of generative control, no.

7.5 A mechanism we built and did not keep: convergence hairs

Not every component of the scaffold survived. One is worth reporting because the reason it failed is informative, and because it is the kind of thing usually deleted from a paper rather than described.

The idea. Branches that fork from a shared prefix often reconverge — they drift apart, then settle back onto substantially the same continuation. If that can be *detected* mid-generation, the redundant branches can be **merged**, freeing search budget for genuinely distinct alternatives. We built this: a **convergence-hair** probe reading hidden states at layers 30 and 42, with a policy layer that would hard-merge branches judged to have converged.

The bar, set in advance. A merge policy is only safe if it does not silently discard the branch that would have been correct. We therefore required a policy to clear three conditions simultaneously before it could be used as a hard merge: terminal oracle retained ≥ 0.98; false-merge rate ≤ 0.05; and survivor reduction ≥ 0.10 — the last because a merge that saves nothing is not worth its risk.

No policy cleared it. The best-performing merge retained the terminal oracle on **0.9583** of tasks (below the 0.98 floor) while reducing survivors by only **0.0247** (against the 0.10 requirement). It was simultaneously too *aggressive* — losing oracle branches — and too *timid* — barely shrinking the pool. Conservative variants were safer and merged even less, which made them pointless. The verdict was to keep the probe as a **soft diagnostic** and never merge on it: `DUALANCHOR_CONVERGENCE_HAIRS_RS_STATUS = SCIENCE_BRANCH_GENERATION_WEAK`, hard merge not cleared.

Why this is not a footnote. First, it is a concrete instance of the paper's central asymmetry appearing *inside the substrate itself*: convergence was **readable** — the hairs found real diagnostic structure — and it was not **actionable**, because acting on the reading cost more oracle than it saved compute. Readable is not usable, one level down from where Part IV finds it.

Second, the discarded mechanism turned out to be a good instrument. In its demoted role the hair probe diagnosed the science-domain failure of §4.5: on chemistry and anatomy the branches converge, and they converge *to a no-good branch* — which is why no selector could rescue those tasks, and why the limit there is branch **generation** rather than branch evaluation. The component we could not use for control became the one that told us where the pipeline was actually broken. We report it because "we built X, X did not clear a pre-registered bar, and X was useful for something else" is a more honest and more useful thing to write down than a scaffold description with the failures pruned out.

7.6 Why a looped backbone: branches need iterative depth to diverge

Neither the branch machinery nor the readouts are, in principle, unique to looped transformers. A standard (non-looped) transformer could also fork at a token boundary and carry branch-specific key-value state, and process-quality signals could in principle be read from a standard model's activations. What the looped architecture supplies is not the *ability* to branch but the *time for branches to diverge.* Because the same weights are applied across four loop iterations, a perturbation injected at a loop/layer boundary propagates and differentiates over the subsequent iterations: a branch co-develops across the loop trajectory, and can be re-perturbed and pruned across iterations rather than being fixed at the moment of the fork. The four iterations give branches a shared-weight refinement budget — depth *in time* — within which to genuinely separate.

A standard transformer diverges too — a branch there develops through its remaining physical layers, across subsequent autoregressive token steps, and through ordinary cached decoding. What it lacks is the *specific* budget the loop supplies: repeated application of the *same* block stack at the *same* token position, i.e. recurrent-depth stages at which a branch can be re-perturbed and pruned at corresponding points of a shared-weight refinement. A non-looped model's within-position computation is traversed once; the loop adds passes over the same position that a branch can co-develop across. This is the sense in which our substrate is neither best-of-N nor Coconut: best-of-N draws independent one-shot samples with no shared evolving state, and Coconut superposes candidate steps within a single continuous thread; our branches are distinct trajectories that diverge along the loop dimension, which only a looped (or otherwise depth-recurrent) backbone provides; the representational advantages of such depth-recurrence are analyzed by Saunshi et al. (2025).

The scope of this argument is narrow. It is a claim about the *mechanism's* suitability, not a capability claim: the divergence itself is mechanically real (zero-perturbation forks stay identical at prefill while perturbed and no-carry branches diverge measurably; §7.2), but Part IV shows that in the *frozen* model this genuine divergence does not translate into reachability gains over matched sampling. The loop gives branches additional recurrent-depth opportunities to diverge; it does not, absent training, make that divergence useful for control. That gap is exactly the readout–control boundary, and the iterative depth argument is why we locate the fix in training-time integration (Section 12) rather than in a different frozen search procedure.

PART IV **Control**

8. Frozen Branching, Steering, and the Readout–Control Boundary

Parts II and III put two things on the table: readable process-quality signals, and an executable substrate through which those signals could in principle be acted upon. This section establishes the paper's load-bearing control result, with the evidence levels separated below and five sealed readout-to-gain conversions closing the section (§8.6). The result is not "readouts confer no gains" — two conversions at the decision level are established, calibrated abstention and content-sensitive terminal selection — and it is not "no form of selection works." It is the sharper statement the five verdicts jointly support: every tested *decision-level* use of the readable signal converts into a validated gain, and every tested route to *generative control* — steering, frozen branching, per-task allocation of recurrent depth, and minimally-trained direction binding — does not.

Whose failure this is. One clarification governs everything below, because the alternative phrasing is tempting and wrong. We do not show that "the model cannot use its own signal." The model is not consulting anything: it does not know the taps exist, and it plays no part in reading or acting on them. In every experiment here, *we* are the agent — an external probe reads the hidden states, and an external policy (a steering hook, a fork, a selector) attempts to act on what was read. The claim is therefore precise and narrower than the anthropomorphic version: **the readable signal converts into validated gains only when our external policy uses it to decide — to abstain, or to choose among finished candidates — and no intervention we constructed converted it into generative control over the computation itself.** Both the successes and the failures are ours; whether the model makes internal use of process-quality information is a question this paper does not test and cannot answer.

The intervention levels, with the finding at each:

Level	Intervention	Finding
Directional (§8.1)	write a readable direction into the residual stream	*established negative*: across seven methods the direction that reads success is not the direction that causes it (the *geometry*)
Branch-level (§8.2)	fork, carry, and generate a perturbed branch	*bounded screen* (four tasks): injected branches are real and divergent, but do not outperform K-matched ordinary sampling — the positive interpretation is removed, though the screen is too small to estimate a general deficit
Selective (§6.5, §8.3, §8.6 C4)	given the correct branch is in the pool, commit to one	*established, in stages*: format-sensitive selection on 39 informative groups (34/39 vs 23.0/39 matched-random, exact $p = 2.44 \times 10^{-5}$), then **content-sensitive selection** on a prospective all-well-formed pool (27/32 vs 64.8% matched-random, exact $p = 0.0086$); the hidden-vs-surface increment there remains unresolved
Allocative (§8.6 C2–C3)	let the readout set the model's own compute — loops per task, or mid-generation prune-and-reallocate	*sealed null / unpowered*: loop allocation is not signal-driven for tap or native gate on either checkpoint; the tournament pool could not differentiate policies (0.978 ceiling)

Level	Intervention	Finding
Trained-binding (§8.6 C5)	bind one writable direction to outcomes with a minimal LoRA pass	*bounded no-detected-gain result*: no detectable binding versus a shuffled control; both adapters cost unconditional accuracy

These are not five views of one finding; they concern different mechanisms, which is part of why the boundary is robust. We then test the geometric explanations that would unify the negatives, and find that neither the one-dimensional audit nor the multi-dimensional follow-up settles the question (§8.4). We call the resulting separation the **readout–control boundary** — decision-level use of readable states converts, generative control does not — and show it is *empirical*: not the consequence of a demonstrated linear-algebraic obstruction, but a robust pattern that survives the obvious attempts to explain it away, and one whose positive side is now itself established rather than assumed.

THE READOUT–CONTROL BOUNDARY: DECISION-LEVEL USE CONVERTS, GENERATIVE CONTROL DOES NOT

Readout	pre-answer success on two domains, survival, generated correctness, earlier layers as loops refine	ESTABLISHED POSITIVE
Abstention	hidden-state scores beat shortcut-only in 4/4 arms (Horizon hidden+shortcut, GSM8K hidden-only)	ESTABLISHED POSITIVE
Selection: form	forced top-1 34/39 vs an exact matched-random 23.0/39, $p = 2.44 \times 10^{-5}$	ESTABLISHED POSITIVE
Selection: content	27/32 vs 64.8% expected on an all-well-formed pool, $p = 0.0086$; hidden-vs-surface open	ESTABLISHED POSITIVE
Directional	seven steering/adapter methods yield no reliable signed capability gain	ESTABLISHED NEGATIVE
Branch-level	four-task fork comparison does not beat K-matched sampling; general deficit unestimated	BOUNDED SCREEN
Loop allocation	tap and native exit gate both fail a matched-histogram random control, on both checkpoints	SEALED NULL
Trained binding	sign-conditioned LoRA on one writable direction: $+0.008$ $[-0.036, +0.050]$ vs a shuffled control	NO DETECTED GAIN

Shaded rows are decision-level uses of the readable signal; unshaded rows are attempts at generative control. Also recorded: the matched-budget prefix-prune tournament could not be powered on this domain (0.978 any-of-6 ceiling); bounded LoRA changes parse/diversity but not net reachability; the one-dimensional two-null audit does not support simple span misalignment, and the multi-dimensional follow-up is unanswerable at the early loci — no matched writable tensors exist there.

Figure 11. Evidence status of the readout–control boundary. Process-quality readouts are established positive on two pre-answer domains and at progressively earlier physical depth; decision-level conversions — calibrated abstention and terminal selection, the latter now including content-sensitive selection on an all-well-formed pool — are established positive; directional steering is an established negative; the branch-level comparison is a bounded four-task screen; loop allocation and minimal trained direction-binding are, respectively, a sealed null and a bounded no-detected-gain result (§8.6). The bounded LoRA probes and the geometric audits are diagnostic rather than capability results.

8.1 Directional control fails: the write surface works, the direction does not

The most local intervention adds a readable direction back into the residual stream during generation and asks whether it steers behavior. The result is a clean dissociation, and getting the dissociation right matters more than the failure itself: **the write path is mechanically valid, and the directions are wrong.**

The write surface is validated, not assumed. Before concluding that steering fails, we established that steering is mechanically possible. Decoder-layer hooks are clean: zero-magnitude writes reproduce no-hook generation *exactly*; perturbation size scales predictably; perturbations propagate to later hidden states and to logits (surviving on the order of 32 tokens); and no CUDA/NaN/Inf instability appears within the safe envelope ($\alpha \leq 0.02$ effective RMS). Whatever fails here, it is not the plumbing — an important distinction, because "we tried steering and it didn't work" is otherwise indistinguishable from a broken hook.

Seven methods, one verdict. Each tested route produced an *unsigned* effect: outputs move, but not in the intended direction.

Method	Verdict
Raw readout direction (NoNorm)	`UNSIGNED_EFFECT`
Empirical success-mean difference	`EMPIRICAL_UNSIGNED_ONLY`
RMS-calibrated static direction	`RMS_UNSIGNED_ONLY`
Local outcome-score gradient probe	`GRADIENT_NO_BETTER_THAN_RANDOM`
Classifier-derived adapter	no reliable held-out control
Teacher-forced causal adapter	`LOCAL_LOGIT_CONTROL_ONLY` — improves logit margin under teacher forcing, `TEACHER_FORCED_ONLY` in free generation
Sequence-level (REINFORCE) adapter	`SEQUENCE_REWARD_IMPROVES` in training, `NO_ADAPTER_SPECIFIC_TRANSFER` held-out, and **`WORSE_THAN_RANDOM`** against a random-direction control

The last row deserves emphasis, because it is a stronger negative than "no gain." An adapter trained directly on sequence-level reward *did* improve that reward during training — and then, on held-out generation, performed **worse than a random direction of matched magnitude**. It did not merely fail to find a control direction; it confidently found the wrong one, and optimizing harder made it worse. The stopping verdict is `NO_FROZEN_BACKBONE_WRITE_PATH` / `FROZEN_BACKBONE_INFERENCE_STEERING_STATUS = CLOSED_UNDER_TESTED_METHODS`.

Why: three geometries that do not coincide. The direction that *reads* success, the direction along which successful and unsuccessful trajectories empirically *differ*, and the direction a learned adapter uses to exert local control are mutually near-orthogonal:

Direction pair	Cosine
Adapter control proxy vs. raw readout	−0.00055
Adapter control proxy vs. empirical success-mean difference	−0.00429
Raw readout vs. empirical success-mean difference	+0.10100

Compactly: *readout geometry ≠ empirical-success geometry ≠ local logit-control geometry.* The model's hidden states tell an external reader which trajectories are promising; the hooks can write into those states; and the direction that carries the reading is not the direction that causes the outcome. Two independently trained adapters converged on nearly the same direction (cosine +0.951) — they agree with *each other* and disagree with the signal they were built to exploit, which is what one expects if both are descending into the same non-steering local-control basin rather than discovering the outcome direction.

Two scope notes. This closure holds under the tested safe-α envelope and tested optimizers; it does not prove that *no* training method can ever steer Ouro, and Section 12 is about the training-time route it motivates. And this local linear diagnostic is *not* the global subspace-misalignment explanation tested but not supported in §8.4 — it is the narrower observation that the obvious linear write directions are not already usable control directions. Full method configurations in Appendix H.

8.2 Branch-level screen: the frozen-fork comparison under matched sampling

One level up, we fork rather than nudge: inject a perturbation at a loop/layer boundary, carry the branch forward through its own cache (Section 7), and let it generate an independent continuation. Mechanically this works — the substrate's bit-exact identity and lineage guarantees hold. In the bounded four-task comparison, injected branches did not outperform K-matched plain sampling. **Greedy** forks yield **zero** new-correct answers relative to the unforked baseline (divergence and reconvergence only). **Sampled** forks do occasionally reach new correct answers — the tempting result to report — but a **K-matched plain-sampling deconfound** dissolves it. Under matched conditions (4 tasks, 12 plain samples per task, temperature 0.7, top-p 0.95, 96-token sampling budget), plain sampling reaches an oracle of **0.750** while the sampled fork reaches **0.611** — the fork is −0.139 *below* matched sampling, not above it. The apparent gains are attributable to the sampling randomness the fork happens to introduce, not to the injection; drawing the same number of ordinary temperature samples does at least as well. A separate check confirms the branches are not cosmetic. **Hook-origin branches persist geometrically**: a perturbation injected mid-trajectory is still detectable in the hidden states at layer 47, and such branches *do* sometimes change the downstream outcome. The injection is producing genuinely distinct trajectories, not decorative noise that washes out — which is what makes the null informative rather than vacuous. What the frozen model lacks is not divergence but a way to *aim* it: the branches are real, they go somewhere different, and nothing in the frozen system can tell in advance which of them to prefer.

The scope of this claim is bounded by its size, and we state it as such. Four source tasks are a screen, not a powered null: the comparison removes the evidence for a frozen-fork *gain* (the sampled-fork advantage is explained by matched sampling, and greedy forks add no new correct solutions), but it is too small to estimate a general deficit. We record it as a bounded frozen-fork screen (Appendix G): the mechanism is valid, and in this four-task comparison injected branches do not outperform K-matched sampling.

This deconfound is also the direct answer to the objection that the branch/carry substrate is merely "best-of-N with extra steps." The K-matched comparison *is* best-of-N with oracle selection, and it is the baseline the frozen substrate is measured against — not a competitor the paper sidesteps. The honest conclusion is not that the branch machinery is useless, but that frozen inference-time branching does not yet beat the corresponding sampling baseline; whether trained branch control can is the question of Section 12.

A bounded training probe weighs against the "a little training would fix it" rejoinder, without closing it. A 300-step bf16 LoRA (30.3M parameters, 1.12% of the model, loss 0.34; deliberately not run to convergence) measurably changes behavior — branch **diversity** rises by +0.45 (2.17 → 2.62), coding **parse** improves 0.72 → 0.94, and math **oracle@K** improves 0.75 → 0.92 — yet net **reachability** is flat: macro positive-oracle@K moves 0.708 → 0.688, because logic (0.83 → 0.75) and reasoning (0.92 → 0.67) regress even as math and coding gain (math parse was already saturated at 1.0 → 1.0). Light training moves the surface statistics a readout cares about — diversity, parse quality — without moving net outcome reachability. Frozen readout does not confer control, and one shallow training pass does not either — a bounded probe against the trivial rejoinder, not a closure over light training generally.

8.3 Selection-level control succeeds on form, and — prospectively re-tested — on content

The third level grants the reader the correctness signal and asks only that it *choose*. This is the Section 6 material, which belongs equally here, and it is the one place where the boundary's shape has changed. It is now clear that the boundary is **not** "no external selection works." On the powered pool of §6.5, forced top-1 beats an exact matched-random null decisively (34/39 versus 23.0/39; exact $p = 2.44 \times 10^{-5}$; task-clustered 95% CI on the difference [+0.167, +0.391]). External selection can convert a readable property of the model's output into a real gain.

What that pool demonstrably converts, however, is surface validity. A shortcut-only selector that never touches a hidden state does as well or better (35/39); 34 of the 39 informative groups contain a malformed, non-committing candidate, and in 26 of them avoiding that candidate is by itself sufficient. The residual question — can the hidden state pick the correct answer among *well-formed* alternatives? — is exactly the question that pool cannot answer, having five such groups (on which the selector is 5/5 against 3.75 expected).

That residual question has since been answered prospectively (§8.6, experiment C4): on a pool constructed so that *every* candidate commits to a parseable answer, a selector trained on completed-candidate hidden features picks a correct candidate in 27 of 32 informative held-out groups (84.4%) against a 64.8% matched-random expectation (exact Poisson-binomial $p = 0.0086$; task-clustered 95% CI on the paired difference [+0.078, +0.305]). Content-sensitive selection is established. The remaining open question is narrower: a surface-only selector also beats random there (24/32), and the hidden selector's advantage over it (+0.09, 95% CI [−0.06, +0.25]) is not resolved at this sample size — so what hidden-state access adds *beyond surface statistics* in this setting is not yet established, and we keep the two claims separate.

8.4 Geometric explanations: one tested and unsupported, one unanswerable

The three intervention levels invite one tidy explanation. If the frozen branch mechanism can only write into a subspace U_{inj}, and the outcome direction d_{out} lies largely *outside* it, the frozen null would follow immediately — no intervention could push the computation along the direction that matters, however well that direction can be read. This would make "a signal that cannot be acted through" a literal geometric statement, and we tested it directly rather than asserting it.

Using exact-protocol regenerated S1/S3 frozen injection/carry deltas (regenerated from the protocol, not replayed from historical saved tensors — a caveat we preserve), we measured $\pi_{\text{inj}}(d_{\text{out}}) = 0.0183$. Taken alone this looks decisive: the outcome direction places 98.2% of its energy outside the writable span. It is not decisive, because a projection fraction is uninterpretable without the span's rank. The injection span has rank $k = 344$ in a $D = 24{,}576$-dimensional feature space, so a uniformly random direction already projects $k/D = 0.0140$ of its energy into the span in expectation. The observed 0.0183 is 1.31× that baseline — *above* random-subspace chance, not below it; against an empirical random-direction null (one million matched draws) the observed projection sits at the 99.99th percentile. The naive reading — outcome direction lies unusually *outside* the injection span — is therefore not supported by this null.

The appropriate null for the actual question is stricter: comparing d_{out} not to random directions but to **outcome-shaped shuffled-label** directions (directions with the same correlational structure as a real outcome axis but no true verifier information), the observed projection (0.0183) falls *below* their mean (≈ 0.0227; global shuffled mean 0.0230, domain-stratified 0.0227), at the low end of the shuffled-label null. The pending-items pass pins the null-audit draw counts (one million random-direction draws and ten thousand shuffled-label draws for the shuffled controls), so we keep

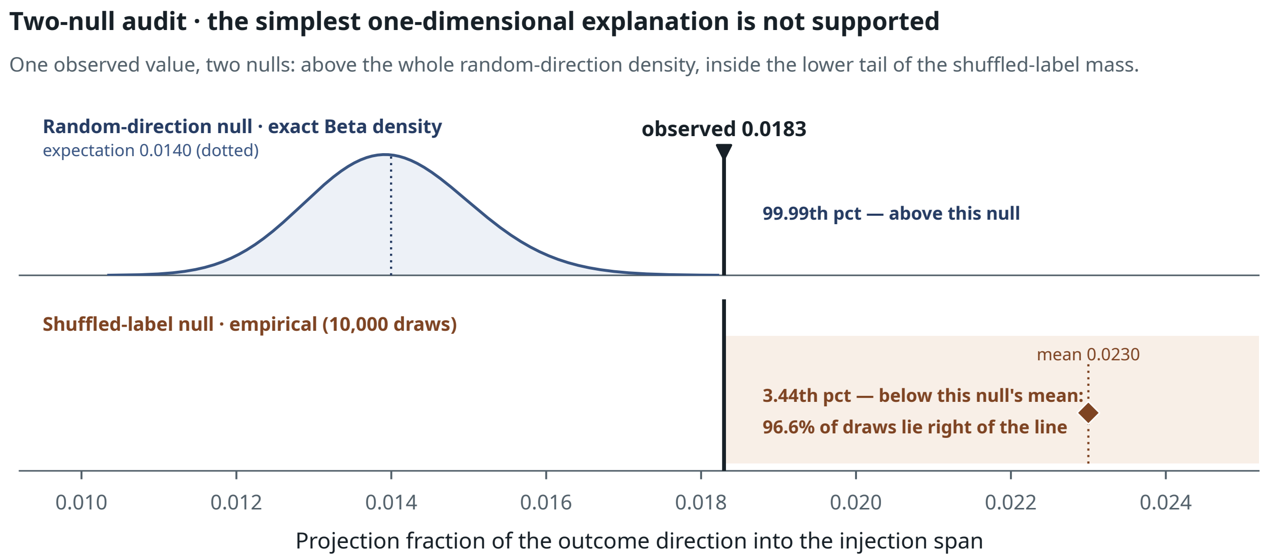


Figure 12. The two-null audit does not support the simplest one-dimensional span-misalignment explanation: the observed projection (0.0183) sits above the random-direction null and below the shuffled-label outcome-shaped null (weakly pointing the other way).

this phrasing as a low-end/null-tail observation rather than turning it into a load-bearing significance claim. This points weakly in the *opposite* direction from the random-null comparison: relative to generic label-correlated variance, the true success direction is *under*-represented in the injection span.

The two nulls disagree; the effect sizes are small (the gap between observed and shuffled-mean is $\approx$ 0.0044 of total energy); the estimate rests on a single one-dimensional projection from one regenerated delta bundle; and the injection span is effectively low-dimensional (participation ratio $\approx$ 5.06 despite nominal rank 344). A separate control confirms the injection span is statistically distinct from the natural sampling span — frozen forking is not merely resampling — but that answers a different question. We therefore do **not** treat this audit as evidence that the frozen null is explained by subspace misalignment. It enters the paper as a *simple explanation not supported by this audit*, not as support for the framing (Appendix J).

The multi-dimensional follow-up, and why it does not answer the question. A one-dimensional projection is a weak instrument, and the natural strengthening is a subspace-versus-subspace comparison: build a *readable* subspace (a bootstrap ensemble of probe directions), an *outcome-relevant* subspace (a bootstrap ensemble of between-class mean-difference directions, deliberately a different construction), and a *writable* subspace (the principal directions of the observed injection deltas), and measure principal angles against a rank-matched random-rotation null. The early loci identified in §4.4 made this newly worth doing: if readability moves earlier, perhaps the early loci are also better aligned with what an intervention can write. We ran it, at ranks 1, 3 and 5, over the early loci L3_8, L3_16, L4_8, L4_16 and the late references L4_24, L4_36, L4_47, against a 2,000-draw random-rotation null.

The intended comparison could not be made. No writable perturbation tensor exists at physical layer 8 or 16, at any loop, anywhere in this project: the injection-delta bundle's loci are exactly the cross product of loops 1–4 with layers 24, 36 and 47. There is therefore nothing to compare the early loci against, and no mathematically justified transport map that would move a late-layer writable direction into early-layer coordinates. We did not manufacture one, and we did not run a new intervention campaign to create the missing data. The verdict for the early-versus-late writable comparison is `INSUFFICIENT_MATCHED_LOCUS_WRITABLE_DATA`, and it should be read as *unanswered*, not as a negative result: nothing about alignment or misalignment at the early loci may be inferred from the absence of these tensors.

Three supporting diagnostics came out of the same run, and we report them as diagnostics rather than promoting them. First, readable and outcome-relevant subspaces overlap far beyond the random-rotation null at **every** locus tested, early and late alike — a genuine non-null finding that runs opposite to the story motivating the audit, since the early loci are not geometrically special and the tightest multi-rank overlap belongs to the terminal locus L4_47. It says outcome-readable structure is broad and multi-dimensional; it says nothing about writeability. Second, at the three late loci where a writable tensor does exist, neither the readable nor the outcome subspace aligns with it beyond the null at any rank — a clean empirical instance of *readable is not writable*, at the only loci where the comparison is possible. That is a caution about extrapolating readability to steerability at any locus; it is not an explanation of why steering fails, and §8.1's causal intervention results remain the actual evidence on that question. Third, the loop-to-loop rotation of the readable subspace at fixed physical layer 16 is largest at L1→L2 in the rank-1 angle and then declines, while the rank-3 mean angles stay nearly flat; the rank-1 ordering is directionally consistent with §4.4's frozen-transfer finding — but none of the three transitions clears the random-rotation null, so this is descriptive corroboration in the multi-dimensional geometry, not an independent statistical confirmation. Principal angles and nulls for all three are in Appendix J.2.

The net position on geometry is therefore more precisely stated than before and no more resolved. The earlier one-dimensional two-null audit did not support simple span misalignment. The attempted multi-dimensional early-locus comparison remained unanswerable because matched writable tensors were absent. The geometric mechanism of the readout–control boundary therefore remains unresolved, and closing it requires new writable data at the early loci — an intervention campaign, not a reanalysis (Appendix J).

8.5 Synthesis: the boundary is empirical, not geometric

Directional steering is an established negative. The bounded four-task branch screen removes evidence for a frozen-fork gain but cannot estimate a general deficit. Terminal selection resolved in two stages: on the mixed-quality pool the margin is explained by malformed-output avoidance, which a hidden-free shortcut selector captures equally well; content-sensitive selection is now established on the all-well-formed pool (§8.6, C4), and what remains unresolved there is whether hidden-state access improves over surface statistics. A bounded training pass moves diversity without moving net reachability. The simplest one-dimensional span-misalignment explanation is not supported by the rank-corrected audit, and the multi-dimensional version cannot be evaluated at the early loci for lack of matched writable data. The readout–control boundary is therefore an **empirical** pattern across multiple intervention types, not a demonstrated geometric obstruction.

Three simple explanations for the observed pattern were tested, which is what makes the boundary a result rather than an absence of one. First, it is not a mechanical implementation bug: zero-perturbation forks reproduce the reference at prefill and the suffix-recompute splice is bit-exact, so the branch/carry machinery preserves the intended computation to the relevant tolerance (§7.2–7.3). Second, it is not a sampling artifact: the K-matched deconfound shows the apparent sampled-fork gains are explained by sampling, and deterministic frozen forks produce no new correct solutions (§8.2). Third, it is not cleanly a linear-subspace mismatch: the rank-corrected projection audit returned conflicting random-direction and outcome-shaped shuffled-label nulls, supporting neither the "outcome outside the writable span" story nor its converse, and the multi-dimensional strengthening of that test cannot be run where it would matter most (§8.4). A fourth explanation is now ruled out on the selection side specifically: the null is not that external selection is impossible in principle, since forced selection does beat matched random when the discriminating property is one a surface reader can also see (§6.5).

We state the open mechanism plainly rather than paper over it. With the implementation and sampling confounds addressed, and the simplest geometric account not supported, the remaining bottleneck plausibly involves some combination of nonlinear propagation of perturbations through the loop, perturbation magnitude relative to the model's operating regime, decoding dynamics, loop-level instability, terminal selection policy, and — most importantly for what follows — the absence of any training-time objective that ties writable branch directions to verifier outcomes. The frozen model was never trained to make its injectable branches outcome-distinct, and nothing here suggests it would exhibit that alignment by accident. This motivates training-time branch-tournament integration, developed as future work (Section 12). A deliberately minimal training-time probe of its precondition — can a short adapter pass bind one writable direction to verifier outcomes at all — was run as part of the sealed programme reported next, and returned a bounded no-detected-gain result (§8.6, experiment C5); the full tournament-integration question remains open, and the frozen results locate it precisely enough to specify.

8.6 Five frozen conversions: where readouts become gains, and where they do not

Every intervention in §§8.1–8.2 targets the *write* surface: the residual stream, the weights, or the branch pool. Two actuator classes available to a deployed frozen system were never tested — the answer/abstain policy and the allocation of the model's own computation — alongside two questions the earlier evaluations left open (the §6.5 content residual, and the §8.5 training precondition). We closed all five with sealed, pre-registered protocols: fixed endpoints, verdict labels fixed before launch, task slices disjoint from every prior run and from each other, and matched-budget or shuffled controls gating every positive. Full designs, per-arm tables, run roots, and the two pre-data amendments are in Appendix J.

C1 — Selective prediction (abstention): positive, in both domains. Using only preserved predictions from the powered pre-answer cohorts, we asked whether hidden-state scores buy a better risk–coverage trade-off than shortcut-only scores at equal coverage. They do, in all four pre-registered arms. On Horizon Logic the tested contrast is the *combined* hidden-plus-shortcut score against shortcut-only, and the combined score improves the area under the selective-accuracy/coverage curve by +0.0124 (95% CI [+0.0048, +0.0207], pooled 4-shortcut arm; +0.0119 under the adversarial 5-shortcut composite; +0.0093 new-cohort-only). On GSM8K the per-row combined arm was not preserved, so that comparison is *hidden-alone* against the shortcut composite, at +0.0087 [+0.0003, +0.0181]. At 70% answer coverage on Horizon the gap is +4.2 points of selective accuracy (92.6% versus 88.4%). No raw-accuracy claim is made or implied — coverage 1.0 recovers the base rate by construction; the claim is the trade-off. This is a deployable reliability gain from the frozen readout, with the model untouched.

C2 — Tap-gated loop allocation: null on both checkpoints. If the loop-1 readout carries per-task difficulty signal, it should allocate a fixed budget of recurrent steps better than uniform or native policies. Fixed-depth generation tables ($d = 1..4$, $k = 2$, 180 fresh tasks per model) let every policy be evaluated as row selection at exactly matched compute. On both RLTT and the Thinking sibling the verdict is that allocation is not signal-driven: the tap policy never beats the 95th percentile of 100 random allocations with the same depth histogram, and neither does the model's own native exit gate (RLTT tap-versus-gate deltas span −0.06 to +0.04, all CIs crossing zero). Only the histogram matters — *which* tasks receive the extra loops is not being chosen by detectable signal, by either reader. The pools themselves are strongly depth-sensitive (RLTT fixed-depth success 0.38/0.84/0.86/0.80 across $d = 1..4$), so the null is not for lack of headroom.

C3 — Matched-budget prefix-prune tournament: pool cannot differentiate. A six-branch tournament that prunes to two survivors at an 80-token prefix by frozen DualAnchor score, evaluated against best-of-N baselines and a 500-replicate random-prune control over the same candidate table at exact token accounting, hit its pre-registered feasibility floor: 97.8% of tasks have at least one correct candidate among six draws, above the sealed 0.95 ceiling, so allocation policies cannot separate on this domain. Per the sealed plan no claim is made in either direction; the recorded internals (tournament 0.706 versus best-of-3 0.644, CI crossing zero; above the random-prune 95th percentile; prefix-score AUROC 0.616) are preserved in Appendix J for the harder-domain re-run this design now awaits.

C4 — All-well-formed terminal selection: positive; the §6.5 residual is resolved. Reported in §8.3: 27/32 informative held-out groups against 64.8% matched-random (exact $p = 0.0086$), on a pool where malformedness cannot carry the margin because every candidate commits by construction. The surface-only control also beats random (24/32), and the hidden-versus-surface increment is not resolved at this sample size (+0.09 [−0.06, +0.25]).

C5 — LoRA direction–outcome binding: not detected. The minimal trainable version of the §8.5 hypothesis: a LoRA pass (r=16, the §8 injection convention verbatim: one curated unit-RMS direction at physical layer 24, loop 1, last prompt token, $\alpha = 0.02$) trained with the injection *sign* carrying candidate correctness, against a twin adapter with coin-flipped signs. If binding were learnable this cheaply, injecting $+\alpha d$ at inference should raise verified success. It does not: the binding adapter's injection effect is +0.008 [−0.036, +0.050] on 90 task-disjoint problems, indistinguishable from the shuffled control (−0.011; difference-of-differences +0.019 [−0.044, +0.081]), while the frozen model's own injection delta re-measures at a non-significant +0.044, consistent with §8.1. Both adapters also cost −0.094 [−0.164, −0.019] of unconditional success against the frozen base — the sign-conditioned objective trains on verified-incorrect text, and the model learns the text more readily than the condition. The scope is deliberately narrow — one direction, one locus, an off-policy cross-entropy objective, at most 400 steps — so this is a bounded no-detected-gain result for the *cheapest* form of binding, not a null for training-time integration generally.

What the five verdicts say together. External frozen readouts now produce validated gains in two decision settings: selective prediction and terminal content selection. What remains unsupported is generative control — changing which solutions the model can reach through adaptive loop allocation, steering, or frozen branching. The readout–control boundary of this paper's title therefore sharpens rather than dissolves: it is not a boundary between reading and acting, since two conversions of readout into action-level gains are now established; it is a boundary between **decision-level use of readable states** — whether to answer, which existing answer to commit to — and **generative control over the underlying computation** — how much recurrence to spend, and what the model writes next. Every tested actuator on the decision side converts. Every tested route to generative control failed to produce a validated gain; the individual outcomes comprise an established steering negative, a bounded branch screen, a sealed allocation null, an unpowered tournament, and a bounded no-detected-gain binding result — each under protocols designed to detect exactly such gains.

PART V

Interpretation

9. Operational Proto-Introspection

We have deferred the paper's loaded term to this point on purpose. The reader has now seen the evidence: relational preference structure, role-specialized readouts, a readability that migrates earlier through the reused stack as the loop refines, pre-answer success prediction on two domains, a correctness signal whose conversion into forced selection was established first on output form and then, prospectively, among all-well-formed alternatives, an executable branching substrate, and a frozen control boundary tested at five intervention levels. Only now do we define the term those results collectively motivate, and defend it against the connotations it invites.

9.1 Definition

We say a hidden state is **operationally proto-introspective** if it contains externally readable information about the quality, stability, uncertainty, likely success or failure, or branch viability of the model's **own ongoing computation**, before external final judgment.

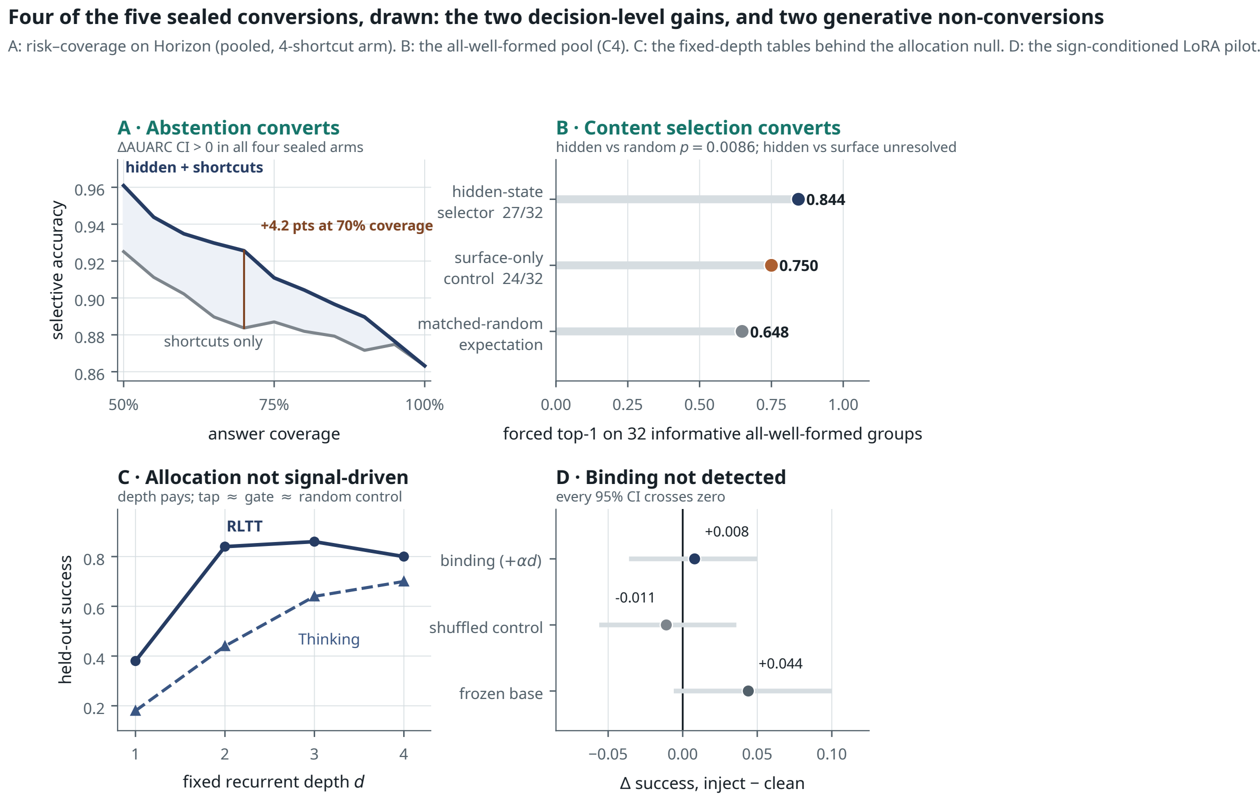


Figure 13. Four of the five sealed conversions, drawn. (A) C1: on pooled Horizon (4-shortcut arm) hidden-plus-shortcut scores dominate shortcut-only scores, worth +4.2 points of selective accuracy at 70% coverage; across all four sealed arms the pre-registered hidden-state-based comparison has a ΔAUARC interval excluding zero — hidden-plus-shortcut on Horizon, hidden-alone on GSM8K. (B) C4: on the all-well-formed pool the hidden-state selector clears the exact matched-random null (27/32, $p = 0.0086$); the surface-only control also beats random, and the hidden-vs-surface increment is unresolved. (C) C2: the fixed-depth tables show large returns to recurrent depth on both checkpoints (with RLTT's $d3 \geq d4$ non-monotonicity) — yet neither the tap allocator nor the native exit gate beats a matched-histogram random control at any budget, so *which* tasks get the depth carries no detectable signal. (D) C5: the sign-conditioned LoRA's injection effect is indistinguishable from its shuffled control, and the frozen re-measure reproduces §8.1's no-reliable-effect. C3, the prefix-prune tournament, is not drawn: its pool could not be powered (0.978 any-of-6 ceiling). Full tables for all five in Appendix L.

Three features of this definition do deliberate work. It is **operational**: it is a statement about what an external reader can recover from the state, not about what the model experiences or can say. It is about the model's **own ongoing computation**: the read object is the model's in-progress process trajectory, not the quality of some external artifact, which is why the pre-answer result (Section 5) is the definition's empirical anchor — the signal is available *before* the computation concludes and cannot be a reading of the finished answer. And it is **proto-**: a precursor to, and weaker than, the introspection the self-report literature studies, marking the gap rather than eliding it.

9.2 Why this is not merely probing

A natural deflation is that we have trained probes and dressed the results in psychological language. The grounds for rejecting it are specific. The signal is **pre-answer** (about ongoing computation, not finished output — the property that rules out the trivial reading, and the one §5 establishes), **role-specialized** (a structured set of distinguishable readouts — survivability, content quality, generated-branch correctness — not one competence scalar), **operationally consumed** (computed over the same cache a live branching scaffold manipulates, not analyzed offline), **causally load-bearing in at least one case** (ablating DualAnchor's layer-47 channel collapses oracle retention from 1.0 to 0.042 — the tap reads something the branch dynamics depend on), **depth-structured under recurrence** (the same information becomes linearly readable at progressively shallower physical layers as the loop refines, and the coordinates carrying it stabilize only after the first pass), and **coupled to a control boundary** (the same signals convert into validated decision-level gains while every tested route to generative control fails to produce a validated gain). No single one of these forces the interpretation, but their conjunction is more specific than "a probe works": it is a claim about *what kind* of information the model's process trajectory exposes and *where* it lives. Probing is the method; the object of study — externally readable process-quality structure in a model's own ongoing computation — is the contribution.

9.3 Why this is not self-report introspection

This is the distinction that most needs stating, because the field's term of art points elsewhere (Section 1.1). The self-report paradigm (Binder et al. 2024; Lindsey 2026; Comşa & Shanahan 2025) asks whether a model can **report** on its internal states and whether that report is causally grounded, and evaluates the report against criteria such as accuracy, grounding, internality, and metacognitive representation. Those criteria are defined over the *report*. We make no report claim at all: we never ask the model about its states, and our evidence is entirely about what an *external* reader recovers from *naturally-arising* activations. We therefore do not — and do not attempt to — satisfy the self-report criteria; our property is orthogonal to them, which is precisely why "proto-" is the honest prefix rather than "weak" or "partial." Where the self-report literature manipulates a representation and elicits a grounded report, we leave the representation untouched and elicit no report. The kinship is that both concern a model's relationship to its own internal states; the distance is that one is about articulable self-knowledge and ours is about external readability of process quality.

9.4 What we explicitly do not claim

We claim none of the following, and no result in the paper should be read as implying any of them: that the model is **conscious**, has **subjective experience**, or is **self-aware**; that the model **reports** on its states or possesses self-report introspection; that the model exercises **autonomous control** over its computation; or — importantly — that the model **uses** the readable signal internally. On the last point the paper's own evidence is the strongest disclaimer: every validated conversion of the signal into a gain (§8.6) is performed by *our* external policies — abstention and selection over the model's finished outputs — while every route by which the signal could influence the model's own computation failed to produce a validated gain, so we are in no position to claim it does so on its own. Operational proto-introspection is a readout-side property. It says the information is there and externally readable; it says nothing about the model having, using, or being able to talk about it.

9.5 Why looped models, and why the boundary matters

A scope note first, on what carries the framing and what does not.

The readouts qualify because of *what they read*, and this is easy to state imprecisely. The tap inputs contain hidden states rather than token IDs or decoded text. They are not reward models scoring finished text: they read the model's **hidden trajectory** — the loop states the model produces while computing — and nothing else. A tap therefore reads the model's own computation, from the model's own states, which is precisely the object the definition in §9.1 names.

Two of these readouts are additionally *forward-looking*, and they are the framing's strongest support because their timing forecloses the obvious deflation. The strict pre-answer probe (§5) reads a trajectory whose answer **does not yet exist** and predicts whether it will be correct; no part of the outcome is available to it, in code. The branch-survivability tap (§4, §6.2) reads an in-flight branch's states and predicts whether that branch will still contain a correct continuation — again, before the branch resolves, and, in DualAnchor's case, causally: ablating the channel it reads collapses oracle retention from 1.0 to 0.042. Both are readouts of unresolved computation, and both precede any external judgment of that computation. That is the property this paper names.

The remaining readouts — content ranking, generated-branch correctness — read the trajectories of computations that have already produced a candidate. They are process readouts on the model's own states, not text scorers, but they read a *completed* computation rather than one still in flight, and we treat them as supporting rather than anchoring evidence.

What the framing does **not** rest on is the branching substrate. The fork/carry/prune machinery of Section 7 is our construction, not the model's: it consumes readouts, it does not produce them, and Section 8 shows no frozen intervention through it produced a validated generative gain. Building a scaffold that reads a model's states is not evidence that the model introspects. The readouts are the evidence; the substrate is what allowed us to test whether reading becomes acting; the acting that succeeds (§8.6) is decision-level and external to it.

The evidence base is therefore a family of process-quality readouts on the model's own hidden states, with two forward-looking members, of which the strict pre-answer result is the cleanest. That result has now been reproduced under the same protocol on a second domain with a different verifier family and a different reasoning structure (§5.3), which is what an earlier version of this work named as the single experiment that would most change its standing. The replication is positive and independently reproduced on a prospectively generated task-disjoint cohort. It materially strengthens the framing without establishing generality, and a failure to reproduce the increment on further powered domains would still weaken the framing substantially — though the branch-survivability readout, the early-layer migration, and the control boundary would stand.

The property is natural to look for in looped transformers because repeated latent computation produces an internal trajectory — a sequence of intermediate states refining toward an answer — that a single-pass model does not expose in the same way. (We mean this precisely: the loop provides the trajectory *natively along depth*, by re-applying the same weights across iterations, whereas feedback approaches such as Coconut (Hao et al., 2024) can *induce* a latent trajectory in a non-looped model along the *sequence*, by spending token positions. The looped route is what makes the trajectory available at every position without consuming the output budget, and — per §7.6 — is also what gives injected branches the iterative depth to diverge.) The looped substrate is what makes "ongoing computation" a concrete, readable object.

This paper retracts a claim made here previously. A prior draft argued that the loop was necessary but not sufficient — that reasoning fine-tuning *installs* the readable content. The controlled replication of §3.5 does not support that: the linearly-readable relational direction is present across the whole Ouro lineage, base model included. What the loop supplies is the *trajectory* — an internal sequence of intermediate states, available at every position, with the iterative depth that lets injected branches diverge (§7.6) — not, on the current evidence, the readability of preference structure itself. Whether the *process-quality* signals that this paper's other readouts target (branch survivability, generated-branch correctness, pre-answer success) are similarly present in an untrained base model is an open question we have not tested; only the HH preference direction was replicated across backbones. Operational proto-introspection, as demonstrated here, is a property of a reasoning-trained looped model; which of its two adjectives is load-bearing remains unsettled.

One distinction keeps this consistent with the base-model literature. The Ouro authors argue their reasoning-trained latent states are *causally faithful* — perturbing an intermediate state changes the output (§7.2 of Zhu et al., 2025). Our readout–control boundary does not contradict this: causal faithfulness is a claim about *sensitivity* (the states are load-bearing, which is precisely why a readable signal exists at all), whereas our null is about *steerability* (the readable signal cannot be used by any frozen intervention we tested to move outcomes in a chosen direction). Our own machinery shows both: injected perturbations do change continuations (a no-carry branch diverges to RMS ≈ 3.0; §7.2), yet that sensitivity does not become controllable gain over sampling (§8). Sensitivity without steerability is, in fact, a compact restatement of the readout–control boundary.

And the control boundary is not a disappointing coda but a load-bearing part of the interpretation: it keeps the claim honest. A readout-only paper could be accused of over-reading a probe; a paper that *also* builds the machinery to act on the signal and finds that only decision-level action converts has, in effect, bounded its own claim from above. We even sought the clean geometric story that would have strengthened the frame — outcome direction outside the writable span — and reported honestly that the audit did not support it, and that its multi-dimensional successor could not be run where it would have mattered (Section 8.4). The interventions that *do* beat their baselines — calibrated abstention and terminal selection — operate on computations the model has already produced, and part of the selection margin is available to a surface reader; nothing that reaches into the computation itself has produced a validated gain. The result is a claim scoped by its own negative evidence: readable, readable earlier than the final pass, and usable to decide — not to steer; present, not exercised.

10. Synthesis

The paper's chain is short and each link is independently supported. Hidden states in a frozen looped transformer expose readable, role-specialized process-quality signals, and they are strong: branch survivability at 0.9697 oracle retention task-disjoint (with a causal ablation), content ranking at 0.6310, generated-branch correctness at AUROC 0.7755, and — most directly — prediction of the model's own eventual success before the answer exists, beyond surface shortcuts, on two domains (Sections 3–6). Recurrence organizes that readability in depth: the same candidate-quality information that is only weakly readable at physical layers 8 and 16 on the first pass is at parity with the established late-layer basis by the third, carried from the second pass onward by coordinates stable enough that a frozen tap transplants between loops without loss (§4.4). A real executable branch/carry/prune substrate exists through which those signals could be acted on, validated by bit-exact identity (Section 7). On the control side the evidence now divides. External frozen readouts produce validated gains in two decision settings: selective prediction, and terminal content selection — established on form on the powered pool, and on content on the prospective all-well-formed pool, with the hidden-vs-surface increment there kept separate as unresolved. What remains unsupported is generative control — changing which solutions the model can reach through adaptive loop allocation, steering, or frozen branching: directional steering is an established negative; the bounded four-task branch screen removes evidence for a frozen-fork gain but cannot estimate a general deficit; loop allocation fails a matched-histogram random control for tap and native gate alike; and two bounded training probes (a 300-step SFT LoRA, and a sign-conditioned binding pilot) do not establish a gain. The rank-corrected two-null audit does not support the simplest one-dimensional span-misalignment explanation, and its multi-dimensional successor is unanswerable at the early loci for want of matched writable data (Section 8).

The conjunction is the contribution. Each half alone would be unremarkable: readable probes are common, and the failure of frozen steering is unsurprising in isolation. What is not common is establishing both in the same system, with the signal demonstrably present, the machinery demonstrably correct, and the obvious confounds (sampling; surface validity) demonstrably controlled — so that the pattern cannot be dismissed as a weak probe, a broken scaffold, a sampling artifact, or a failure to try selection at adequate power. The through-line is an asymmetry that had to be *earned*: **readable becomes decidable — whether to answer, which answer to keep — but does not become generative control.** We interpret it as weak operational proto-introspection plus a readout–control boundary (Section 9) — the model's own ongoing computation is externally legible, progressively legible at shallower depth as it refines, and usable at the decision level, while every tested route into the computation itself failed to produce a validated gain.

This also suggests a use for the taps beyond diagnosis. If the readouts are treated cautiously, they become candidate *latent reward models*: small functions over hidden trajectories that can provide intermediate credit assignment for preference, content quality, survivability, or branch correctness before a final answer exists. The generative-control failures in Section 8 are the reason this remains future work rather than a result: a readable tap is not automatically a safe or sufficient reward signal, and must be anchored to external verifiers and audited for reward hacking.

10.1 Controls, failures, and surviving claims

Because this paper's central claim is an asymmetry rather than a single positive score, the negative results and controls are part of the evidence rather than afterthoughts. The table below summarizes the main checks that changed, limited, or falsified a stronger interpretation. All entries are controls or failures reported elsewhere in the paper or appendices, and every row's provenance is now pinned to a live-repo artifact.

Claim pressure-tested	Control or failure mode	Observed result	Surviving claim	Status
HH hidden states contain preference structure	Strict antisymmetrization of the fixed-order evaluator, **full 8,552-pair test set**	Fixed-order 0.9479 [0.9431, 0.9525]; strict antisym **0.6392** [0.6291, 0.6493]; symmetric/antisym magnitude ratio 1.50×	Preference structure is real; the fixed-order 95.2% is order-inflated and is discovery-stage only	**VERIFIED** on the full test set
Preference signal is relational, not pointwise	Matched pointwise probe on the identical pair-disjoint split	Relational linear 0.5653 vs pointwise linear 0.5418; paired Δ +0.0234 [+0.0132, +0.0334]	Relational decoding is more accurate, but preference IS available pointwise (0.5418 > chance) — the strong 'unavailable pointwise' claim is withdrawn	**VERIFIED clean**; the historical 21.75% was leak-inflated
Readable relational signal is installed by reasoning training	Controlled cross-backbone replication: reconstructed *historical* probe protocol plus the original evaluator, identical config across backbones	The reconstructed probe (row-level orientation split, leak-style by design and not comparable to §3.5's clean values) reads base/Thinking/RLTT identically, swap consistency 1.0; the original evaluator gives ~95% canonical on base too (95.0/95.0/94.5) with flat antisym (58.0/59.5/60.0); the historical base=24% does not reproduce and has no artifact	**Retracted.** Localization unestablished; the linear preference direction is present across the whole lineage including base — the clean values are §3.5's 0.5553/0.5653/0.5698	Failed replication, §3.5
Domain transfer is uniform	Science repair; source-specific breakdown; convergence-hair diagnostic	MMLU anatomy partially cleared (n=3); chemistry/physics/SciQ excluded, parse → 0.0; branches converge to a *no-good* branch (`CHEM_ANATOMY_NO_GOOD_CONFIRMED`)	Transfer is role/domain structured; the science limit is in branch **generation**, not evaluation — no selector can pick an oracle absent from the pool	**VERIFIED** (§4.5, §7.5)

Claim pressure-tested	Control or failure mode	Observed result	Surviving claim	Status
Specialist taps always beat generalists	**Task-disjoint domain-transfer study** (zero crossing IDs; 360–4,748 held-out groups; task-clustered CIs)	Code→coding **0.9528** vs HH→coding 0.6944; HH→alignment **0.6902** vs code→alignment 0.5609; reasoning specialist 0.7671 ≈ balanced generalist **0.7613**; random-20 HH→alignment 0.6038	Specialization pays where distinctions are hard (code, alignment) and is unnecessary where a general quality axis suffices (reasoning); training-set scale matters independently of domain	**VERIFIED clean** (§4.5); supersedes a contaminated small-N study
Pre-answer success signal is just a shortcut	Length, log-probability, and composite controls; task-clustered bootstrap	Hidden+shortcuts AUROC 0.797 vs shortcut composite 0.731; incremental +0.066, **task-clustered** CI [+0.021, +0.112], excludes zero; leave-one-task-out [+0.056, +0.071]	Hidden states add pre-answer information beyond simple shortcuts	**VERIFIED** under the correct clustered test; primary evidence
Pre-answer signal is specific to GSM8K	The same strict protocol on Horizon Logic; nested within-domain increment; a prospectively sealed 510-task task-disjoint extension; shuffled labels; leave-one-task-out; drop-high-malformed and adversarial malformed-sibling controls	Pooled (680 tasks): shortcut 0.652, hidden 0.763, combined 0.763; incremental **+0.111**, task-clustered CI [+0.056, +0.169]; extension cohort alone **+0.095** [+0.038, +0.157]; adversarial five-shortcut composite +0.107 [+0.053, +0.164]; 84 held-out negatives; shuffled-label 0.542; max single-task influence 0.008	The increment replicates on a second domain with a different verifier family, independently replicates on a disjoint cohort, and survives the strongest identified shortcut — which the original cohort alone did not, a calibration stated in §5.3	**VERIFIED, POWERED**; malformed-vs-clean separability (hidden AUROC 0.713) preserved as an unresolved nuance
Early-layer readability is an extraction or length artifact	Extraction path validated against the production cache; full-grid label shuffle at four cells; text-length baseline; 20 targeted tests	Locked policy 0.6867 cached vs 0.6850 fresh; shuffle floors 0.337–0.427 against real cells 0.410–0.633, with 8_L1 clearing its own shuffle by only +0.060 and 8_L4 by +0.195; length ranking macro 0.323	The loop trend and the L2-onward direction stability are real; first-pass readability at the shuffle-tested layer-8 cell is weak and barely above its own control	**VERIFIED clean** (§4.4); layers 8 and 16 confirmed as new loci
Generated correctness can be turned into top-1 choice	Forced selection after S3B2 refit; exact Poisson-binomial matched-random baseline	Hidden ridge AUROC 0.7755 / pairwise 0.7338; sel@oracle 5/8 = 0.625 on N=8 groups vs matched random 0.3625, exact P(≥5) = 0.087	Selector exceeds the matched-random point expectation but N=8 is underpowered: on this slice reliable forced selection is **not established**	Detection **pinned**; baseline computed exactly (two earlier figures — 0.5833 and 0.625 — corrected); superseded by the powered evaluation below
Forced terminal selection cannot be quantified	**Powered task-disjoint evaluation**: 39 informative groups; exact per-group Poisson-binomial null; shortcut-only control selector; shuffled-score and order-invariance checks	Observed **34/39 (0.872)** vs matched random **23.0/39 (0.590)**, paired diff +0.282, CI [+0.167, +0.391], exact p = 2.44e-05; shortcut-only selector **35/39**; 34/39 groups contain a malformed candidate, 26/39 decided by malformed-avoidance alone; 5 pure-content groups (5/5 vs 3.75 expected)	**Format-sensitive selection established**; **content-sensitive commitment unresolved** — the hidden state is not shown to add beyond malformed-avoidance here	**VERIFIED** on its own terms; the shortcut comparison is part of the result, not a footnote

Claim pressure-tested	Control or failure mode	Observed result	Surviving claim	Status
Survival scaffold solves terminal arbitration	Task-disjoint re-run of the branch-survival evaluation (zero crossing task IDs)	Survival verified clean (stage retention 0.9697, terminal 1.0000); the old survivor-set terminal figures **withdrawn** — that remainder has only 2 reward-diverse tasks	Survival works; substantive commitment on survivor pools stays unquantified, and the powered result above is a domain-matched selector on a different pool	Original evaluation had 8 crossing task IDs; §6.2, §6.5
Branching gains reflect internal fork control	K-matched plain sampling deconfound	Sampled fork gains are explained by matched sampling; greedy/deterministic fork does not add new corrects	No gain in the bounded four-task comparison; a general deficit is unestimated	**Bounded negative screen**; exact params recorded in artifact report
Branch/carry mechanics are invalid	Zero-perturbation and suffix-splice checks	Prefill fork is bit-exact; cached decode has small bf16 drift; suffix recompute splice is bit-exact across 192 slots	The substrate is mechanically real, but mechanics do not imply control	Verified by ledger; full detail in App F
Steering can use readout directions as control vectors	Seven steering/adapters methods	No reliable signed capability gain; local adapter directions can be near-orthogonal without solving global control	Readout is not reliable steering control	Artifact-backed
Light training fixes branch reachability	Bounded 300-step LoRA probe	Coding parse improves 0.72→0.94 and math oracle@K 0.75→0.92, but macro reachability 0.708→0.688 due to logic/reasoning regressions	Light training changes behavior/diversity but does not solve the boundary	Verified by ledger
Frozen null is explained by simple linear orthogonality	Rank-corrected two-null subspace audit	Observed projection 0.0183 is above random-direction chance but low vs shuffled-label outcome-shaped controls	The geometric explanation is ambiguous and not load-bearing	**Pinned**; random-direction and shuffled-label draw counts resolved
Early readable loci align better with writable directions than late ones	Subspace-vs-subspace principal angles at ranks 1/3/5 against a 2,000-draw random-rotation null; readable, outcome, and writable subspaces built three different ways	**No writable tensor exists at layer 8 or 16 at any loop**, so the intended comparison has no data; where it can be run (L4_24/36/47) readable and outcome subspaces sit at chance against writable, while readable↔outcome overlap is far beyond null at *every* locus and tightest at L4_47	The early-vs-late writable comparison is **unanswered**, not negative; readable↔outcome overlap is broad, not early-specific; “readable is not writable” holds where it is testable	Diagnostic only; nothing was transported across coordinate systems and no new intervention campaign was run
Readout gains vanish once malformedness cannot carry them	Prospective all-well-formed pool (every candidate commits by construction); exact per-group Poisson-binomial null; surface-only control selector; sealed power gate with pre-declared extension	**27/32 (0.844)** vs matched random 0.648, exact $p = 0.0086$, paired-diff CI [+0.078, +0.305]; surface-only control 24/32; hidden-vs-surface increment +0.09 [−0.06, +0.25]	**Content-sensitive selection established**; hidden-state increment beyond surface statistics unresolved at this n	**VERIFIED** (§8.6 C4); resolves the §6.5 residual

Claim pressure-tested	Control or failure mode	Observed result	Surviving claim	Status
Hidden scores are no better than shortcuts for deciding when to answer	Paired risk–coverage curves on preserved out-of-fold predictions, task-clustered bootstrap, adversarial shortcut composites	ΔAUARC positive with CI excluding zero in all four sealed arms (Horizon pooled/adversarial/new-only; GSM8K)	Hidden-state-based scores buy a real abstention gain over shortcut-only scores at equal coverage — hidden-plus-shortcut on Horizon, hidden-alone on GSM8K	**VERIFIED** (§8.6 C1)
Loop-1 readout can route recurrent compute per task	Fixed-depth tables, exact budget matching, native-exit-gate comparator, 100-draw random-histogram control, both checkpoints	Tap never beats the random-histogram p95 at any budget; neither does the native gate; pools strongly depth-sensitive	Per-task loop allocation is not signal-driven under the tested policies, for tap **and** native gate	**Sealed null** (§8.6 C2), both checkpoints
One writable direction can be bound to outcomes by light training	Sign-conditioned LoRA vs coin-flip-control adapter; six paired eval arms incl. frozen re-measure; zero-delta bit-identity gate	Binding delta +0.008 [−0.036, +0.050] ≈ shuffled control; both adapters cost −0.094 of unconditional success	Cheapest-form binding not detected; boundary holds against a minimal training-time attack; full integration untested	**Bounded no-detected-gain result** (§8.6 C5), scope: one direction/locus, ≤400 steps

The table is intentionally conservative. Several rows are negative: the stronger story fails, and the claim is narrowed. One row is newly positive and immediately narrowed by its own control. This is the pattern that makes the paper more than a collection of probes. The readout results survive shortcut, antisymmetry, transfer, extraction-path, and label-shuffle controls, while the control results survive matched sampling, steering, shortcut-selector, and subspace-explanation audits. What remains is not "the model can control itself," but a two-part statement the five sealed conversions of §8.6 make precise: the model's looped hidden states are externally legible — earlier in the stack than the final pass, and on more than one domain — and that legibility now converts into validated decision-level gains (abstention, content selection), while every tested route to generative control — loop allocation, steering, frozen branching, and the cheapest form of trained direction-binding — has failed to produce a validated gain.

10.2 Evidence-status map

For a reader who wants each major result's standing in one place:

Status	Results
Established positive	Strict GSM8K pre-answer increment (+0.066, task-clustered CI excludes zero); strict Horizon Logic pre-answer increment (pooled +0.111, CI [+0.056, +0.169]; independently replicated on a prospective task-disjoint cohort at +0.095 and robust to the adversarial malformed-sibling shortcut); task-disjoint branch survival (0.9697; L47 channel causally load-bearing); generated-correctness detection (AUROC 0.7755, grouped split); corrected content ranking (0.6310 vs 0.5525); recurrent early-layer readability shift (L4 − L1 between +0.188 and +0.212 at every tested layer; 8_L3 = 0.633, tied with L4_24 and L4_36) with L2-onward frozen cross-loop direction stability — replicated across the Ouro family (base, Thinking, 1.4B; frozen cross-checkpoint tap transfer at parity) and, as a depth trend, qualitatively on out-of-family Huginn; **format-sensitive** terminal selection on the expanded pool (34/39 vs 23.0/39, exact $p = 2.44 \times 10^{-5}$); **content-sensitive** terminal selection on the prospective all-well-formed pool (27/32 vs 64.8% matched-random, exact $p = 0.0086$; hidden-vs-surface increment separately unresolved); the selective-prediction gain (ΔAUARC positive with CI excluding zero in all four sealed arms, both domains); mechanical cache equivalence and the bit-exact splice
Established negative	No reliable signed steering under the seven tested methods; per-task loop allocation not signal-driven under the tested policies on either checkpoint (tap and native exit gate both fail the random-histogram control); no detectable direction–outcome binding from the minimal sign-conditioned LoRA (scope: one direction, one locus, ≤400 steps)
Bounded negative screen / positive interpretation removed	In the four-task comparison, frozen injected branches do not beat K-matched sampling and greedy forks add no new-correct answers; the screen is too small to estimate a general deficit
Unresolved	The incremental value of hidden-state access over surface statistics in content selection (+0.09, CI [−0.06, +0.25] on the all-well-formed pool); mid-generation prune-and-reallocate at matched budget (the tournament pool's 0.978 any-of-6 ceiling exceeded the sealed feasibility floor; suggestive internals recorded, no claim); multi-dimensional readable/writable alignment at the early loci (no matched writable tensors exist there); the mechanism of the readout–control boundary; domain generality beyond the two pre-answer domains; Thinking strict pre-answer transfer — **unresolved after a powered replication attempt** (900 fresh tasks, 99 held-out negatives: +0.042, 95% CI [−0.017, +0.104], sealed verdict UNRESOLVED; three times tighter than the pilot's [−0.13, +0.21] and still crossing zero, with practical equivalence also unestablished at the ±0.05 margin. The design had 0.816 power to detect the pre-registered +0.095 target, but +0.095 itself lies inside the observed interval, so an RLTT-sized effect is not formally excluded — the run failed to replicate it rather than ruling it out. Distinct from the Thinking loop-allocation entry under Established negative, which tests allocation utility and does not update this estimate); whether training-time integration beyond the minimal binding pilot crosses the boundary
Withdrawn / corrected	Leaked preference magnitudes (0.845, 0.2175); the "preference unavailable pointwise" claim; the fixed-order 95.2% as a relational result; training-stage localization (base=24%); contaminated survivor-set terminal-selection and domain-transfer magnitudes

Diagnostic-only	Convergence hairs; the one-dimensional subspace audit; the broad readable↔outcome subspace overlap; bounded-LoRA surface changes; the small-n S3B2 cross-role transfer failure; the historical MATH-origin observation

11. Limitations

We state limitations directly; several are already integrated at the point of each claim, and we consolidate them here rather than confess them at the end.

- **Two pre-answer domains, one checkpoint.** The central introspective result (Section 5) has positive evidence on GSM8K and on Horizon Logic, under the same strict cut and the same nested within-domain comparison, and the Horizon result now carries an independent, prospectively sealed task-disjoint replication (+0.095 [+0.038, +0.157]) that also survives the adversarial malformed-sibling shortcut — the original 170-task cohort alone did not, which is stated in §5.3 rather than smoothed over. That is a genuine strengthening, and it is still not generality: both domains use deterministic verifiers, nothing here speaks to domains without one, and pre-answer generality across checkpoints remains open after a powered attempt: the 900-task Thinking replication returned +0.042 [−0.017, +0.104], sealed verdict UNRESOLVED (§5.4). The design had 0.816 power to detect the pre-registered +0.095 target, but that value still lies inside the observed interval, so an RLTT-sized effect is not formally excluded: the run failed to replicate it, and establishes neither the effect nor its practical absence at the ±0.05 margin. Smaller positive effects remain compatible with the data; resolving those would need roughly a five-fold larger cohort (an approximate scaling, not a pre-registered figure). The separate Thinking depth-allocation null does not bear on it — that experiment tests allocation utility, not pre-answer readability. In the logic domain a hidden-state classifier separates malformed from well-formed candidates at AUROC 0.713, close to the correctness AUROC; the increment survives dropping high-malformed tasks and the malformed-sibling baseline, but the two signals are not claimed to be fully disentangled.
- **The recurrent-depth trend is family-replicated, not universal.** The §4.4 pattern is supported across four Ouro checkpoints and, as a depth trend only, on one out-of-family depth-recurrent architecture (Huginn), where its absolute level is much lower and no comparable first-pass rotation was detected. This does not establish universality across recurrent architectures, and the earlier-in-physical-depth formulation remains specific to Ouro's reused-stack design.
- **The early-layer result is readout-only, and bounded.** §4.4 identifies loci that are readable, shallow, and leave substantial downstream depth. It tests neither controllability nor any capability gain, and no intervention was performed there; the loci are candidate readout surfaces, not demonstrated steering sites. It is measured on one target (CoreContent candidate quality), on a bounded subset (120 held-out groups per domain), under one pre-registered pooling convention, and it is exploratory across fourteen cells — the finding rests on the consistent first-to-later-loop increase and the L1-versus-L2+ stability split, not on any single cell clearing significance. 195 `hendrycks_math` task IDs spanning splits were excluded from the subset, which flags a dataset-hygiene fix for any future rebuild. And the frozen CoreContent direction does **not** transfer to generated-branch correctness (AUROC 0.354–0.475 at every cell), so the stability result is about one target's coordinates across loops, not a universal quality axis.
- **The 95.2% fixed-order evaluator figure is not a relational result.** It was substantially reliant on a canonical-ordering prior, collapsing to **0.6392** strict antisymmetrized accuracy on the full 8,552-pair test set (§3.3). We report it only as fixed-order, discovery-stage accuracy. The strongest *clean* preference readout is the antisymmetrized nonlinear evaluator at 0.6392; the linear relational probe reads 0.5653.
- **Terminal selection: content-sensitivity is established, hidden-specificity is not.** The original expanded pool established the format-sensitive form (34/39 vs 23.0/39, $p = 2.44 \times 10^{-5}$, with a shortcut-only selector at 35/39 and malformed-avoidance carrying most groups). The prospective all-well-formed pool (§8.6 C4) resolves the content question affirmatively — 27/32 vs 64.8% matched-random, $p = 0.0086$ — but its own surface-only control also beats random (24/32), and the hidden-versus-surface increment (+0.09, CI [−0.06, +0.25]) does not exclude zero at this sample size. On Horizon-domain pools, well-formed outputs evidently carry surface correlates of correctness; what hidden-state access adds beyond them is the open question, not whether content selection exists. Both results are also domain-matched: selectors were trained on their own pools, and neither is evidence that the frozen S3B2/DualAnchor/CoreContent selectors would perform comparably out of domain, which was not tested. The older survivor-set figures remain withdrawn without replacement.
- **Neither geometric audit resolves the boundary.** The exact-protocol one-dimensional subspace audit (Section 8.4) is ambiguous under a two-null analysis and does not explain the frozen null. The multi-dimensional successor was run and could not answer the question it was built for: no writable perturbation tensor exists at physical layer 8 or 16 at any loop, so the early-versus-late writable comparison has no data, and we did not manufacture it by transporting late-layer directions into early-layer coordinates. The supporting diagnostics that did emerge — broad readable↔outcome

overlap at every locus, and chance-level readable↔writable alignment at the three late loci where it can be measured — are diagnostic, not causal, and the second of them concerns the *available* injection-delta geometry rather than steerability in general. The readout–control boundary is empirical; we do not have a mechanistic account of it.

- **Frozen results, plus one minimal training probe.** All control findings are for the frozen backbone under tested methods and magnitudes, with one exception: the sign-conditioned LoRA binding pilot (§8.6 C5), which is a probe of the cheapest form of direction–outcome binding — one curated direction, one locus, an off-policy cross-entropy objective, at most 400 steps, at $\alpha = 0.02$ where the direction's own bank validation used 0.005. Its no-detected-gain result constrains that cheapest form only. We have run no full training-time integration (no S3A tournament, no S3C), and make no claim about what such training would yield.
- **No substantive generative-capability, control, report, or consciousness claims.** We establish no capability gain from frozen branching, steering, or compute allocation, no autonomous control, no self-report, and nothing about subjective experience or awareness. The interventions that do beat their baselines — forced selection and calibrated abstention — operate at the decision level, on computations the model has already produced. The scope is a readout-side property, its two validated decision-level conversions, and the boundary at generative control.
- **Retracted claims, and two systematic causes.** Five figures reported in this project — including three in the prior published paper (Kirin, 2026a) — were distorted (four inflated, one deflated below chance): four by source-item leakage across the train/test split, and one (the 95.2% fixed-order evaluator) by a presentation-order prior that no split check would catch. One further claim did not reproduce at all. The relational linear probe (84.5% → 0.5653), the pointwise linear probe (21.75% → 0.5418), CoreContent v2 (0.6691 → 0.6310), and branch survival (0.9848 → 0.9697) were each corrected by splitting on *source items* rather than constructed rows; the fixed-order evaluator's 95.2% was separately inflated by a canonical-ordering prior (→ 0.6392 antisymmetrized); and the training-stage localization (base 24%) has no surviving artifact and does not reproduce. The terminal-selection magnitudes that had rested on the contaminated branch-survival split are withdrawn without replacement (§6.5). We retract the strong claim that preference is *unavailable* pointwise (it is decodable at 0.5418, above chance) and the claim that reasoning fine-tuning *installs* the readable signal. The corrective protocol and both mechanisms are stated in §3.7, and the correction of record for the prior paper is the erratum to arXiv:2604.09870. What survives is a smaller, cleaner set of results: preference is decoded more accurately relationally than pointwise (+0.0234), the antisymmetrized nonlinear evaluator reads 0.6392 on the full test set, and the full training pipeline modestly raises the linear signal (+0.0145, RLTT vs base).
- **The readouts do not require a looped architecture.** A non-looped SFT transformer reads candidate quality at 0.5680 under a task-disjoint split (§4.7). The readout side of this work is therefore not a property of recurrence. We also do not claim looping is irrelevant: the Ouro-vs-control comparison is not architecture-controlled (different family, corpus, objective, width, tap geometry), so the 6.3-point gap is unattributed. A causal architecture study would need matched looped/non-looped models trained identically.
- **The corrected CoreContent coding figure is mutant-only.** The corrected task-disjoint coding value (0.8956) is measured against deterministic mutants; a corrected task-disjoint *relevance* evaluation has not been run, because the relevance negatives were generated inside the old splits and were not regenerated. The stored-split finding that the coding tap is a corruption detector rather than a relevance judge (0.94 vs 0.58) is retained as qualitative, not re-quantified.
- **Two candidate second domains were rejected at preflight, and the one that worked shows why.** SVAMP proved unusable because its answers are front-loaded (parser and label checks disagreed), and Hendrycks MATH, while supplying genuine long reasoning, produced ~21/22 correct among parseable generations — most failures were non-commitment or truncation, so dropping truncations removes the negative class while labelling truncation as failure degenerates the task into a length predictor. Horizon Logic succeeds where those failed precisely because proof depth is a *controlled* variable and the verifier is deterministic, which is what makes the malformed class separable from the incorrect class rather than confounded with it — but the same property leaves it with a small negative class. A third domain will need a protocol chosen with that trade-off in view, not simply more compute.
- **Provenance items.** The strict pre-answer result, the full-set antisymmetry audit, the S1 K-matched decimals, the two-adapter convergence cosine, and the steering seven-method closure are now verified against live-repo artifacts. The math-transfer origin figures (§3.6) remain unarchived and are retained only as non-load-bearing origin motivation.

What would falsify or strengthen this framing. We state the conditions under which the paper's claims should be revised, since a framing worth defending should be one whose failure modes are namable. The proto-introspection framing would be *weakened* if a third powered pre-answer domain removed the hidden-state incremental gain; if the Horizon Logic increment failed to survive at a larger negative-class size, or were shown to be a repackaged malformedness detector by a control stronger than the two we ran; if S3B2 generated-branch correctness collapsed under grouped or task-held-out splits; if the early-layer loop trend failed to reproduce on a second readout target; or if the strict pre-answer effect failed to replicate under an independent implementation. (Three candidate weakeners have already been tested and did *not* materialize: the full 8,552-pair antisymmetrization audit confirmed rather than eliminated the relational preference result; the task-clustered bootstrap confirmed rather than dissolved the GSM8K pre-answer increment; and the second-domain replication came back positive rather than null.) It would be *strengthened* by a third powered pre-answer domain that preserved the incremental gain; by a content-selection pool on a domain whose well-formed outputs carry weaker surface correlates, which would resolve the hidden-versus-surface increment the all-well-formed pool leaves open; by a trained branch-control objective that converts readout into actionability (crossing the boundary rather than describing it, and going beyond the minimal binding pilot); or by replication of the readout and boundary in a different looped or depth-recurrent architecture. A properly pair-split antisymmetric replication across the Ouro lineage — larger than the one reported in §3.5, and with the original probe weights preserved — would settle the training-stage question that the current evidence leaves open.

12. Future Work

The frozen boundary specifies its own next step. Because control fails not for lack of readable signal but plausibly for lack of a training-time objective tying writable branch directions to outcomes, the natural intervention is **training-time branch-tournament integration** (S3A): training the model so that its injectable branch directions become outcome-distinct, aligning the branch-control manifold with the already-readable outcome geometry. We frame this as the motivated next experiment, not a result; the §8.6 C5 pilot tested only its cheapest precondition — one direction, one locus, off-policy cross-entropy — and its no-detected-gain result narrows the easy version without touching the full hypothesis. External evidence makes the hypothesis concrete: Coconut (Hao et al., 2024) is a clean case in which latent-space exploration became *usable* precisely because it was trained into the model rather than bolted onto frozen inference — consistent with the boundary we observe, where reading and acting on analogous signal in a frozen model confers no *generative* gain. Coconut demonstrates only the positive half (trained latent exploration is usable); the frozen-fails half is supplied by our own results, not by Hao et al., who trained latent reasoning from the start and did not test a frozen variant. The two halves together — their trained success and our frozen-control results — are the pattern S3A is designed to reproduce for internal branch control.

Architectural developments also widen the experimental space. MELT (Conchello Vendrell et al., 2026) makes loop count memory-free at inference by sharing one gated per-layer cache across iterations; since our readouts tap per-loop residual states rather than the cache, a MELT-style model would allow the §4.4 loop-trend question to be asked at reasoning depths Ouro's $O(\text{loops})$ cache makes expensive — does readability keep improving past four iterations, or saturate? — while porting the branching substrate to a gated shared cache is the corresponding open engineering problem (§7.3).

12.1 Taps as latent reward models

A natural next use for the tap family is **reward learning over latent computation** rather than only over completed text. Standard reward models usually score final responses, or occasionally explicit process traces written in tokens (Lightman et al., 2023). The readouts in this paper score a different object: hidden-state trajectories, pairwise candidate differences, and branch states inside the looped computation. This makes them plausible auxiliary rewards for branch-tournament RLTT. A DualAnchor-like tap can reward retaining branches that still contain an oracle continuation; a CoreContent-like tap can reward content-quality improvements; an S3B-style tap can reward generated-branch correctness; and a strict pre-answer success tap can provide early credit before the answer string reveals the target value. In this interpretation, the taps do not replace task verifiers or preference labels. They densify them, turning sparse final supervision into intermediate signals over the latent trajectory that produced the answer.

This use case is also where the readout–control boundary becomes practically important. The same results that make taps attractive as reward features make them dangerous as unanchored objectives. S3B2 shows that a correctness signal can be decodable while forced selection remains weak; the frozen branching and steering results show that externally readable directions are not automatically usable as control directions; and the orthogonality audit shows that a simple linear geometric story is not enough to explain the boundary. A training system that uses taps as rewards should therefore keep external verifiers as the final authority, use grouped heldout tasks to check generalization, calibrate tap margins before converting them into reward, and include explicit reward-hacking controls such as shuffled labels, metadata-only baselines, adversarial branch pools, and ablations where the tap reward is withheld. The intended proposal is not "optimize the tap and trust it," but rather **verifier-anchored latent reward shaping**: use taps to assign credit inside the model's computation while retaining external correctness, preference, or safety evaluations as the arbiter.

In the context of alignment and monitoring, this also makes the taps useful as *diagnostic reward models*. A tap can mark where in the latent trajectory a branch becomes low-quality, unstable, self-contradictory, or likely to fail, even when the final answer is still recoverable. That opens a path to process-level datasets built from hidden trajectories rather than only text completions: reward the trajectory for preserving correct branches, penalize premature collapse or false pruning, and train the model to make branch states more separable before final decoding. The present paper stops short of that training step. Its contribution is to show that such latent reward features are readable and structured, and that frozen inference-time generative control is not enough to exploit them.

12.2 Other uses for taps: monitors, routers, and data engines

The reward-learning use case is the central one, but it is not the only plausible role for taps. More generally, a tap is a small, auditable sensor over a latent trajectory. That makes it useful anywhere a system needs to decide what to do with an ongoing computation before the final answer is available. One use is **adaptive compute allocation**: a pre-answer success tap could decide whether to stop, continue looping, branch, ask for a verifier, or defer to a stronger model. The most direct form of this has already been tested and failed — §8.6's C2 found per-task loop allocation not signal-driven, for the tap and for the model's own exit gate, on both checkpoints — so we propose it only for the actuators C2 did not cover (deferral, verifier calls, escalation to another model), not for setting recurrent depth. Another is **candidate routing**: survivability and correctness taps can cheaply filter large branch pools before expensive external verification. A third is **active data selection**: branches with high disagreement between a tap and an external verifier are precisely the examples that should be added to the next training or calibration round.

A further use is **hidden-state monitoring**. Textual chain-of-thought monitoring is useful when reasoning is externalized, but recent safety work emphasizes that monitorability is fragile: relevant computation may be absent from the visible chain, compressed into latent state, or deliberately made hard to read. Hidden-state taps are not a replacement for chain-of-thought monitors, but they are a complementary instrument: they can look for process-quality, inconsistency, branch collapse, or likely failure in the latent computation itself. This connects the present work to representation-engineering and activation-monitoring methods, where simple directions or population-level representations have been used to detect or influence high-level properties such as truthfulness, refusal, and safety-relevant behaviors. The important difference is role and timing: our taps are trained on looped process trajectories and are intended to read the quality of an unfolding computation, not only a static representation of a completed prompt.

A related use is **AI-control monitoring**. In control protocols for untrusted agents, monitors decide when to defer, audit, resample, or block actions. A hidden-state tap could provide an additional monitoring channel: not "what did the model say?" but "what did the computation look like while producing it?" This could be useful for detecting low-confidence success, suspiciously unstable branches, or cases where the final answer looks clean but the latent trajectory resembles known failure modes. The same adversarial caveat applies even more strongly here than in reward learning. If a model can observe or optimize against the tap, the tap can become a target. Any safety use would need heldout taps, randomized monitor ensembles, adversarial evaluations, and protocols that deny the untrusted policy direct access to the monitor internals.

Finally, taps can serve as **scientific measurement tools**. They let us ask when a capability becomes readable during training, which loop/layer loci carry it, whether a signal transfers across domains, and whether a proposed intervention changes the internal trajectory before it changes the output. In this paper, that measurement role is already visible in the controls-and-failures ledger (§10.1). Future work could turn it into a standard diagnostic suite: before claiming that a new latent-reasoning method improves reasoning, measure whether its hidden states become more separable for survivability, correctness, and pre-answer success, and whether those signals remain calibrated under distribution shift.

12.3 Taps and recursive improvement

Taps are also relevant to recursive self-improvement, but only in a limited, verifier-anchored sense. A recursively improving system needs to choose among proposed data mixtures, prompts, branch policies, training objectives, verifier designs, or model edits. Final-output evaluation alone is expensive and sparse; a tap can provide a cheap first-pass critic over the internal computation produced by each proposal. In such a loop, taps could filter proposals, assign dense credit to promising latent trajectories, flag internal degradation before it shows up in aggregate benchmarks, and prioritize which candidate changes deserve external evaluation.

This is not a claim that taps enable autonomous RSI. The results of this paper argue against that interpretation. Readable process-quality signals do not become frozen generative control, and a system trained to maximize tap scores directly would be vulnerable to Goodharting the tap. The safe RSI-adjacent role is therefore evaluator-layer support: taps can make improvement loops more sample-efficient by identifying which internal trajectories are worth checking, while external verifiers, heldout tasks, adversarial branch pools, and tap-withheld audits remain the authority. In short, taps could be part of the critic and monitoring layer of a recursive-improvement system; they are not the self-improvement engine.

12.4 The experiment queue, in priority order

Ordered by how much each would change the paper's standing rather than its polish:

1. **A content-only terminal-selection pool.** *Done — resolved affirmatively (§8.6 C4).* The pool was built exactly as specified here (malformedness screened at generation time, never by correctness), and content-sensitive selection is established: 27/32 vs 64.8% matched-random, exact $p = 0.0086$. The successor item this spawns: the same design on a domain whose well-formed outputs carry weaker surface correlates of correctness, to resolve the hidden-versus-surface increment (+0.09, CI [−0.06, +0.25]) that the Horizon pool cannot.
2. **A third powered pre-answer domain, chosen against the trade-off §11 names.** Two domains do not establish generality. The useful next domain is one with a deterministic verifier *and* a substantial negative class, reducing the class-imbalance problem encountered in the existing domains.
3. **Writable data at the early loci.** §8.4's multi-dimensional geometry audit could not answer its own question because no injection-delta tensor exists at physical layer 8 or 16. A bounded intervention campaign that produces matched perturbation deltas at the loci §4.4 identifies would make the early-versus-late readable/writable comparison possible for the first time — and would simultaneously be the first *causal* test of whether those loci are actionable at all, which the readout-only study deliberately does not address.
4. **Training-time branch integration (S3A).** The route the frozen boundary motivates (§12 opening). A selector at scale, with a training-time objective binding writable branch directions to verifier outcomes, is the direct test of whether the boundary is crossable at all. *Its cheapest precondition has now been probed and returned a bounded no-detected-gain result (§8.6 C5: sign-conditioned LoRA, one direction, one locus, ≤400 steps — no detectable binding, and both adapters cost unconditional accuracy).* The full version — on-policy, multi-direction, reward-driven — remains open and is now better specified by what the pilot ruled out.
5. **The early-layer loop trend on a second readout target.** §4.4 is measured on candidate-quality ranking, and the frozen direction does not transfer to generated-branch correctness. Whether the *loop trend itself* — as opposed to the particular direction — reproduces for branch survivability or pre-answer success is untested, and the small-n S3B2 corroboration is layer-specific at best.
6. **A properly pair-split antisymmetric replication across the Ouro lineage**, with the probe weights preserved, to settle the training-stage question §3.5 leaves open (the observed RLTT − Base effect is real but small, and no single stage is attributable).
7. **Read/write robustness gates** to characterize the steering envelope more finely than the tested safe-α band, and a **minimal independent replication** of the pre-answer and boundary results.

We name the longer-horizon integrated system (Jormungandr) only as a direction, with no capability claim attached.

13. Conclusion

We asked whether the intermediate states of a frozen looped transformer carry readable information about the quality of the model's own ongoing computation, and whether that readability confers control. The answers are yes and no.

Hidden states expose role-specialized process-quality signals that low-capacity taps recover from frozen representations and that a live branching scaffold consumes: branch survivability at 0.9697 oracle retention under a task-disjoint split (with the layer-47 locus causally load-bearing — ablating that channel collapses retention to 0.0417), content ranking at 0.6310 across five domains, generated-branch correctness at AUROC 0.7755, and — the paper's primary result — a strict pre-answer probe that predicts the model's own eventual success before the answer exists, adding significant information beyond length and log-probability shortcuts on **two** domains: +0.066 on GSM8K and +0.111 pooled on Horizon Logic, both intervals excluding zero under a paired task-clustered bootstrap, with no single task driving either; the logic increment is independently replicated on a prospectively generated task-disjoint cohort and survives an adversarial malformed-sibling baseline that the original cohort alone did not. The recurrent structure organizes this in depth as well as in time: candidate-quality information that a matched tap recovers only weakly at physical layers 8 and 16 on the first pass is at parity with the established late-layer basis by the third, and from the second pass onward it is carried by coordinates stable enough that a frozen tap transplants between loops without measurable loss — a pattern that replicates across the Ouro family, whose frozen taps transfer between checkpoints at parity, and whose depth trend reappears — with no comparable first-pass rotation detected — in the out-of-family depth-recurrent Huginn. A same-class candidate-quality readout also appears in a **non-looped** SFT transformer (0.568 task-disjoint), showing that this form of process-quality readability does not require recurrence. We report that against our own framing's interest. We then built the machinery through which such signals could be used, validated by bit-exact identity, and ran the conversions. External frozen readouts now produce validated gains in two decision settings: selective prediction — hidden-state scores beat shortcut-only scores on the risk–coverage curve in all four sealed arms across both domains (hidden-plus-shortcut on Horizon; hidden-alone on GSM8K, whose combined arm was not preserved) — and terminal content selection, established first on output form (34 of 39 informative groups against an exact matched-random expectation of 23.0, with malformed-avoidance carrying the margin) and then, on a prospectively constructed pool where every candidate is well-formed, on content (27 of 32 against 64.8% expected, exact $p = 0.0086$; whether hidden-state access adds increment beyond surface statistics there remains unresolved). What remains unsupported is generative control — changing which solutions the model can reach: directional steering is an established unsigned negative; in a bounded four-task screen, injected branches do not outperform matched sampling; per-task allocation of recurrent depth is not signal-driven, for our tap or for the model's own exit gate, against a matched-histogram random control on two checkpoints; and a minimal LoRA pass detects no binding of a writable direction to outcomes. The simplest one-dimensional geometric explanation for this pattern is not supported by a two-null audit, and its multi-dimensional successor cannot be evaluated at the newly identified early loci because no matched writable perturbation data exists there. We name the readable, decision-usable property operational proto-introspection, defined narrowly against the self-report introspection literature rather than as an instance of it, and take the readout–control boundary — between decision-level use of readable states and generative control over the underlying computation, not any capability, control, or awareness claim — to be the paper's central contribution.

Every load-bearing current quantitative claim in this paper is reported under the audited protocol (§3.7): source-item-disjoint splits with zero-crossing integrity checks, and antisymmetrized evaluation for fixed-order pairwise scorers. Historical or diagnostic quantities with incomplete provenance are explicitly marked and are not load-bearing. The audit that produced that protocol — two evaluation traps and five corrected figures, three of them in the prior published paper — is stated in this paper at the level of detail needed to apply it elsewhere, and the prior paper's correction of record is the erratum to arXiv:2604.09870. The corrected story is less dramatic, and the surviving effects are real.

The most important open question is whether training-time integration can cross a boundary that no frozen intervention we built could. The most important open caveats are that strict pre-answer generality across checkpoints remains unresolved on Thinking, and that the incremental value of hidden-state access over surface statistics in all-well-formed content selection is not yet established.

APPENDICES

Complete reproducibility record

> Public mirror: `github.com/VykosMolt/Branching-Looped-Transformer`, which carries the code, the documentation, and the experiment records cited below; the reproducibility commit and per-result artifact paths are in Appendix K. Where a figure has been superseded by a corrected re-run, the appendix reports the corrected value and marks the original withdrawn rather than deleting it.

Appendix A — Hidden-state extraction and feature formats

Feature construction uses hidden trajectories from the frozen looped backbone. The canonical feature basis taps layers **24, 36, and 47** across four loop states L1–L4 at width 2048, giving $D = 3 \times 4 \times 2048 = 24{,}576$ when concatenated. Role-specific variants use final-loop L4, mean-over-loops, L1/L4 fusion, or full loop-concatenation; the main text reports which variant is used where. The 24/36/47 basis is the general-purpose default; one role is an exception — the locked CoreContent terminal selector prunes layer 47 as dead weight and uses a 2-channel 24+36 tap (Appendix E.2.2), which slightly outperforms the 3-channel version on real negatives. Layer 47 remains load-bearing for DualAnchor's looped survival (its perturbation is not diagnostic-only there; Appendix E.2.1), so the basis is not uniformly reducible — 47 matters for survival and not for terminal content ranking.

§4.4 extends this basis downward to physical layers 8 and 16 across all four loops, and finds that the early layers reach parity with 24/36/47 by the third recurrent pass; the canonical three-locus basis therefore records where the signal sits after refinement completes, not where it first becomes readable.

The 24/36/47 basis comes from the historical locus program summarized in `evaluator-locus-summary.md`. That program began with pairwise locus and loop ablations, passed through normalization/bias decomposition and all-layer cached probes, and only then settled on 24/36/47 after the v10 Thinking-vs-RLTT loop-geometry analysis. The goal was to preserve a small mid/late/final trajectory: a mid-depth anchor (24), a late integration anchor (36), and the terminal/pre-output boundary (47). Full all-layer extraction would have been much more expensive and would have made every downstream audit harder to repeat; the three-locus basis retained the useful loop-localized readout signal while keeping tiny taps and repeated controls tractable. In the DualAnchor architecture-looped branch-selection line, the same loci were promoted from a static feature basis into a twelve-stage schedule over the four loop iterations: `L1_24 -> L1_36 -> L1_47 -> L2_24 -> L2_36 -> L2_47 -> L3_24 -> L3_36 -> L3_47 -> L4_24 -> L4_36 -> terminal L4_47`.

For the original evaluator, hidden states were captured by a forward hook on the Ouro model body rather than relying on generic `output_hidden_states` plumbing, because the local Ouro wrapper exposes the loop trajectory through model-specific outputs. The HH-RLHF data, the extraction procedure, and the ~5M-parameter evaluator head follow the setup of Kirin (2026a); we reuse it unchanged except for the role-specific feature variants introduced here. Supporting: `interfaces-and-tools.md`, `evaluator-locus-summary.md`, `chronological-evaluator-summary.md`, Kirin (2026a, arXiv:2604.09870), hidden-state extraction scripts.

Appendix B — Pairwise evaluator and HH flip/antisymmetry audit

Evaluator architecture. Fixed-order pairwise evaluator: token attention pooling per loop state, normalized candidate differences, projection to 512 dimensions, a two-layer GRU across the four loop states, and a nonlinear scorer (full detail in Appendix C). The historically selected epoch-2 checkpoint preserves the fixed-order accuracy peak: 83.3% → 95.2% → 62.4% across epochs 1/2/5 (Kirin 2026a). The audit shows that this peak combines genuine relational signal with a maximal transient presentation-order prior and must not be described as a clean generalization peak; later epochs overfit.

Full-test-set antisymmetry audit (8,552 pairs). The evaluator's canonical-order accuracy reproduces (0.9479, 95% CI [0.9431, 0.9525]; the historical 8,141/8,552 = 0.9519 lies inside this interval), but its **strict antisymmetrized accuracy is 0.6392** (95% CI [0.6291, 0.6493]).

Measurement	Full 8,552-pair result
Fixed-order (canonical) accuracy	0.9479
Swapped-direction accuracy	0.1954
Strict antisymmetrized accuracy	**0.6392**
Strict sign-flip rate	0.2475
Normal/flipped score correlation	−0.9247
Both orders prefer the first argument	75.25%
Symmetric (order) component, mean	+1.2649
Symmetric / antisymmetric magnitude ratio	1.50×

All 8,552 test indices present exactly once; no duplicate, missing, or non-finite rows. The evaluator is *not* degenerate — swapped scores remain strongly anti-correlated with canonical ones — but the positive first-position offset dominates the sign for most pairs. Pooling and normalization ablations (§3.3) confirm that attention pooling is not the cause and that the trained difference-LayerNorm *restrains* rather than creates the order effect.

Linear probes. The L-BFGS difference probe scores $w^{\top}(h_A - h_B)$ with no bias, hence exact antisymmetry. Clean pair-disjoint result: **0.5653**. The historical 84.5% was inflated by orientation-row leakage (§3.7) and is retracted.

Swap-protocol training-metric deflation. Documented in Kirin (2026a): the antisymmetry-enforcement protocol masked the pairwise model's capability across seven consecutive runs, with the deflated training metric inversely correlated with test performance.

Supporting: `flip-test-interpretation.md`, `chronological-evaluator-summary.md`, full-HH audit artifacts (`full_hh_antisymmetry.json`, `full_hh_antisymmetry_rows.csv`), Kirin (2026a, arXiv:2604.09870).

Appendix C — Tap architectures and training details

The paper uses the term *tap* for small readout heads trained on frozen hidden-state features. This is deliberately narrower than the original evaluator. The Kirin evaluator used token attention pooling for each loop state, normalized candidate differences, projected them to a 512-dimensional hidden space, ran a two-layer unidirectional GRU over the four loop states, concatenated the GRU output with the final-loop projection, and passed the result through a nonlinear scorer. That architecture was useful as a discovery tool because it could integrate information across the loop trajectory, but subsequent locus and swap-control work showed that the GRU was repeatedly weak or mildly counterproductive relative to simpler exact-antisymmetric heads. It remains a control/escalation path, not the default tap architecture.

The simplest comparison taps are `AntisymLinear` and `AntisymLinearNoNorm`. Both form a pairwise difference

$$\Delta h = h_A - h_B,$$

and then apply a bias-free linear readout so that swapping candidates flips the score sign. The normalized variant scores

$$s_{\mathrm{LN}}(A, B) = w^{\top}\,\mathrm{LN}(h_A - h_B),$$

whereas the NoNorm variant scores

$$s_{\mathrm{raw}}(A, B) = w^{\top}(h_A - h_B).$$

Both are structurally antisymmetric: $s(B, A) = -s(A, B)$ up to numerical tolerance and the exact details of the symmetric preprocessing. The difference is interpretive. `AntisymLinear` suppresses raw scale and norm effects and asks whether the *direction* of the pairwise difference carries the label; it is the safer comparator for hard near-miss distinctions where magnitude can be a shortcut. `AntisymLinearNoNorm` preserves raw magnitude and often behaves more like a scalar utility readout; it is useful when quality is relatively transitive or when easy-prune/objective distinctions are carried by feature scale. We keep both families as complementary probes rather than treating either as universally superior.

This distinction is part of the evaluator-to-tap transition. The large GRU evaluator first revealed the existence of a loop-trajectory signal, but its capacity and fixed-order training made strict antisymmetry auditing mandatory. The tap family was designed so that the comparison rule itself is auditable: if a head says A beats B, the same head must say B loses to A. Success under that constraint is therefore stronger evidence for readable hidden geometry than success by a large order-sensitive evaluator. Tap head sizes, optimizer/effective-batch settings (EBS = 32), epoch selection and overfitting behavior, DualAnchor and CoreContent head definitions, and pointwise-vs-pairwise probe setups (the clean pair-disjoint pointwise result is 0.5418; the historical 21.75% was pair-leaked, §3.7) are tracked in the supporting training scripts and ledgers. Supporting: tap training scripts, `chronological-evaluator-summary.md`, `evaluator_pairwise.py`, Kirin (2026a, arXiv:2604.09870).

Appendix D — Domain transfer and role-separation tables

Clean (task-disjoint) domain-transfer study. Deterministic split, **zero task IDs crossing** the boundary; held-out sets from 360 coding groups to 4,748 alignment groups; task-clustered bootstrap intervals. This is the reportable study; it supersedes the contaminated one below.

Tap trained on	Evaluated on	Top-1	Pairwise	Reading
Code	coding	**0.9528**	**0.9650**	strong in-domain code specialization
HH (general)	coding	0.6944	0.8727	general head transfers, substantially weaker
HH (general)	alignment	**0.6902**	0.6831	alignment specialization is real
Code	alignment	0.5609	0.5538	code specialization does not replace preference
Reasoning	reasoning	0.7671	0.8870	reasoning specialist
Balanced (all-core)	reasoning	**0.7613**	0.8827	generalist matches the specialist — no reasoning tap needed
Random-20 HH subset	alignment	0.6038	0.5968	data-scale effect, independent of domain

Summary: specialization pays where the within-domain distinction is hard (code, and to a lesser degree alignment) and is unnecessary where a general quality axis suffices (reasoning). Training-set *scale* is a separate axis from domain match.

Superseded (contaminated) study — retained for transparency, not for use. An earlier domain-transfer table reported code-trained 0.875/0.833 vs HH 0.750/0.600 on strict-clean code; HH all-200 0.855 vs code-trained 0.535; reasoning 0.960/**0.986**; HH→code transfer 0.500/0.571; and random-20 means 0.655/0.640. Those evaluations were small-N (tournament counts in the tens; the 0.986 rests on 25 tournaments) and pre-dated the task-disjoint discipline of §3.7 — at least one branch dataset had task IDs appearing on both sides of the split. **They are withdrawn.** The qualitative direction survived the clean re-run; the magnitudes did not. We list them here so that readers of earlier drafts can locate what changed.

Science: partial repair, source-specific failure. Source-specific repair partially cleared MMLU **anatomy** (held-out positive-oracle 0.333, parse 1.0, n = 3 tasks) while **chemistry, physics, and SciQ** stayed excluded, with parse rates collapsing to 0.0. The **convergence-hair** diagnostic (§7.5) locates the cause: on chemistry and anatomy the branches converge to a *no-good* branch (`CHEM_ANATOMY_NO_GOOD_CONFIRMED`) — the model does not generate a correct branch for a tap to find, so the limit is in branch *generation*, not branch *evaluation*. Reasoning, not science, became the headline scope.

Layer/loop localization. Canonical layers 24/36/47 with L1–L4 loop variants; layer 47 carries the largest loop spread while 24/36 are more loop-converged (§4.3, Appendix A).

Cross-loop early-layer matrix (§4.4).

- *Data.* Bounded deterministic subset of the corrected CoreContent v2 dataset: 250 train / 60 validation / 120 held-out groups per core domain (1,250 / 300 / 600 groups; 8,592 candidates), on the existing task/prompt-disjoint split with **zero** task-ID crossings asserted; 195 `hendrycks_math` task IDs spanning splits excluded.
- *Features.* One frozen forward per candidate (`use_cache=False`, four UT steps, no early exit) at layers {8, 16, 24, 36, 47} × loops {L1–L4}, mask-valid mean pooled, stored `[5,4,2048]` fp16.
- *Tap and selection.* `AntisymLinearNoNorm`, locked pairwise training procedure, 36-point grid selected on validation macro top-1 only.
- *Uncertainty.* Task-clustered bootstrap, 10,000 draws, seed 20260720, identical draws for both members of every paired comparison; matched chance (tie-aware, analytic) macro 0.2818 [0.2784, 0.2855].

§4.4 reports the local-refit point estimates and their task-clustered intervals. The paired differences against the final-loop layer-24 refit — the contrast that defines the `EARLY_READABLE` label — are tabulated below; the four bold cells qualify (above chance and within 0.03 of the 24_L4 refit).

Cell	Δ vs 24_L4 refit	95% CI	Cell	Δ vs 24_L4 refit	95% CI
8_L1	−0.2083	[−0.2567, −0.1600]	16_L4	**+0.0100**	[−0.0150, +0.0350]
8_L2	−0.0500	[−0.0900, −0.0100]	24_L1	−0.1900	[−0.2350, −0.1466]
8_L3	**+0.0150**	[−0.0217, +0.0500]	24_L2	−0.0500	[−0.0867, −0.0133]
8_L4	**+0.0033**	[−0.0267, +0.0350]	24_L3	−0.0083	[−0.0383, +0.0217]
16_L1	−0.1783	[−0.2250, −0.1317]	36_L4	+0.0283	[−0.0017, +0.0583]
16_L2	−0.0350	[−0.0750, +0.0050]	47_L4	+0.0017	[−0.0417, +0.0433]
16_L3	**−0.0017**	[−0.0333, +0.0283]			

The three loop contrasts all exclude zero — 8_L4 − 8_L1 = +0.2117 [+0.1633, +0.2600], 16_L4 − 16_L1 = +0.1883 [+0.1450, +0.2350], 24_L4 − 24_L1 = +0.1900 [+0.1466, +0.2350] — and the best early cell is statistically tied with the strongest reference: 8_L3 − 36_L4 = −0.0133 [−0.0467, +0.0200].

§4.4 also reports the nine →L4 transplants. The nine intermediate transfers complete the matrix (Δ is the paired difference against the target-local refit; ρ is Spearman score correlation on identical held-out candidates):

Transfer	Target top-1	Δ vs target refit	ρ	Label
8: L1→L2	0.4667	−0.1016	0.690	ROTATED
8: L1→L3	0.4000	−0.2333	0.548	ROTATED
8: L2→L3	0.6100	−0.0233	0.895	**STABLE**
16: L1→L2	0.4717	−0.1116	0.845	ROTATED
16: L1→L3	0.4817	−0.1350	0.771	ROTATED
16: L2→L3	0.6117	−0.0050	0.954	**STABLE**
24: L1→L2	0.4833	−0.0850	0.845	ROTATED
24: L1→L3	0.5233	−0.0867	0.771	ROTATED
24: L2→L3	0.6100	0.0000	0.918	**STABLE**

Every ROTATED transfer's Δ interval excludes zero; no STABLE transfer's does. Pairwise sign agreement runs 0.713–0.837 for ROTATED and 0.915–0.967 for STABLE transfers. Controls, in full: label-shuffle at four representative cells (8_L1 0.3500, 8_L4 0.4267, 16_L2 0.4033, 24_L4 0.3367); text-length ranking baseline macro 0.3233 (best-signed 0.3700); extraction-path validation against the production cache on 2,434 identical held-out candidates (per-cell **mean** cosine 0.9982–0.9991 at layers 24/36/47 — the per-candidate minimum is much lower and is not claimed as a bound — with the locked policy scoring 0.6867 cached versus 0.6850 fresh); 20/20 targeted tests pass, covering scoring equivalence against the locked scorer, candidate/label alignment, split integrity, frozen-head immutability, deterministic refit, validation-only grid selection, and bootstrap clustering. Exploratory secondary readout on S3B2 (16 tasks, 160 branches, leave-one-task-out): locally refit pooled AUROC rises with loop at layer 16 (0.597 → 0.654 / 0.651) but not at layer 24 (L1 0.635 > L2 0.560), with layer 8 weak throughout (0.490–0.577); the frozen CoreContent direction transfers at 0.354–0.475 pooled AUROC at every cell, i.e. at or below chance.

V3 family and out-of-family replication runs (2026-07-26). Pre-registered plans sealed before extraction (`family_xloop_v3_20260726/EXPERIMENT_PLAN.md`, `huginn_probe_v3_20260726/EXPERIMENT_PLAN.md`). Family: the identical sealed 2,150-group subset (sha-verified copy), tap class, 36-point grid, validation-only selection, and bootstrap seed 20260720 re-run on ByteDance/Ouro-2.6B (base, pinned 1ed0425), Ouro-2.6B-Thinking (pinned f1edd81), and Ouro-1.4B (24 layers; capture layers mapped proportionally to {4, 8, 12, 18, 23}; its own tokenizer and fresh taps). Early-cell refits per checkpoint are given in §4.4; every L4−L1 loop contrast excludes zero on every checkpoint; frozen RLTT taps score within 0.028 macro top-1 of each 2.6B sibling's local refits with Pearson/Spearman 0.97–0.99. Huginn: tomg-group-umd/huginn-0125 (bf16), one frozen forward per candidate at eight recurrence steps, states captured at the last recurrent-core block, masked-mean pooled [1, 8, 5280]; the sealed pairwise machinery runs on the step axis unchanged. Step-wise refits 0.332/0.320/0.303/0.362/0.383/0.378/0.402/0.438; step-8−step-1 +0.107 [+0.058, +0.157]; step-1-sourced taps transfer at parity through step 4 and lose 0.093 [0.045, 0.142] by step 8. Artifacts: `family_xloop_v3_20260726/<model>/{train_eval_results.json, bootstrap_stats*.json, crossmodel_rltt_transfer.json}`, `huginn_probe_v3_20260726/{train_eval_results.json, bootstrap_stats_v3.json}`, each with `SHA256SUMS` and `ENVIRONMENT.json`.

The complete local-refit matrices (heldout macro top-1; chance 0.2818; identical subset and procedure):

Cell	RLTT	base 2.6B	Thinking 2.6B	Cell	RLTT	base 2.6B	Thinking 2.6B
8_L1	0.4100	0.4150	0.4083	24_L1	0.4283	0.4200	0.4233
8_L2	0.5683	0.5533	0.5667	24_L2	0.5683	0.5667	0.5633
8_L3	0.6333	0.6083	0.6233	24_L3	0.6100	0.6100	0.6133
8_L4	0.6217	0.6300	0.6133	24_L4	0.6183	0.6367	0.6267
16_L1	0.4400	0.4300	0.4217	36_L4	0.6467	0.6400	0.6317
16_L2	0.5833	0.5683	0.5783	47_L4	0.6200	0.5883	0.6083
16_L3	0.6167	0.6333	0.6183				
16_L4	0.6283	0.6483	0.6233				

Ouro-1.4B (24 layers; mapped cells; its own tokenizer and fresh taps):

Layer	L1	L2	L3	L4
4	0.4650	0.5250	0.6017	0.6033
8	0.4617	0.5550	0.6100	0.6083
12	0.4733	0.5783	0.6233	0.6100
18 (ref, L4 only)	—	—	—	0.5683
23 (ref, L4 only)	—	—	—	0.5600

Huginn-0125, recurrent-core boundary (chance 0.2818): step-wise local refits and step-1-sourced frozen transfers (Δ against the target-step refit, 95% CI from the sealed clustered bootstrap):

Step	1	2	3	4	5	6	7	8
Local refit	0.3317	0.3200	0.3033	0.3617	0.3833	0.3783	0.4017	0.4383
L1→step Δ	—	+0.017	+0.017	−0.018	−0.060*	−0.023	−0.037	−0.093*

(*CI excludes zero: L1→5 [−0.110, −0.012]; L1→8 [−0.142, −0.045]. All other transfer intervals include zero. Step-4-sourced taps transfer forward at ≤0.027 loss throughout.)

Supporting: cross-loop early-layer tap run (2026-07-20); task-disjoint domain-transfer re-run (2026-07-13); `domain-transfer-ledger.md`, `core-domain-tap-audit.md`, `science-reasoning-repair.md`, `evaluator-locus-summary.md`, `chronological-evaluator-summary.md`.

Appendix E — DualAnchor / CoreContent / S3B details

E.1 Pre-DualAnchor survival scaffold

Before DualAnchor, the branch-survival line passed through fixed-composite and selection-only scaffold experiments. These are included because they show the same survival/selection split before the later DualAnchor naming and architecture-looped baseline.

Scaffold / policy	Main retention result	Failure mode / caveat	Status
`fixed_composite_conservative_top4`	oracle retention 0.931; false-prune 0.069; avg survivors 3.873	**measured pre-audit**, on the same split family later found to have crossing task IDs; treat as historical, not as a clean estimate	`SURVIVAL_READY` (verdict stands; magnitude unaudited)
old-context/coding subset	retention 1.000; coding false-prune 0.000	subset-specific; pre-audit; not a general selector	supporting diagnostic
selection-only Phase 2 prototype	fixed top4 oracle retention 0.9514; false-prune 0.0486; avg survivors 3.9109	terminal-reward figures **withdrawn** (contaminated split; see E.3)	`SURVIVAL_READY_FINAL_ARBITER_WEAK` (verdict stands; magnitudes do not)

These results are the pre-DualAnchor ancestor of the later selection wall. They support the claim that survival and terminal arbitration separated early: top-k pruning could retain oracle branches at high rates, while the final arbiter remained weak. Supporting: `branch-generation-and-survival.md`, `current-state.md`.

E.2 DualAnchor, CoreContent, and S3B2

E.2.1 DualAnchor — genesis and construction. DualAnchor is the branch-survival / retention family (scores, prunes, and forwards survivors during the looped branch/prune search), not a terminal correctness selector. It is the two tap identities `MIX_CODE_REASONING` and `MIX_OBJECTIVE_ALL` — the strongest existing content/action readouts — with branch-validity grafted on by weight-space transplant, so that two "dual-anchored" taps carry content and branch-viability together rather than being split across separate content, branch, and bridge taps. The design reached its locked form through an incremental lineage, each step with its own diagnostic verdict:

Step	Verdict	What it established
old-anchored branch-valid taps v1	`OLD_ANCHORED_BRANCH_TAP_USEFUL`	weight-space transplant adds branch/bridge gain without dropping content/code performance
two-tap branch selector v1	`TWO_TAP_BRANCH_SELECTOR_READY`	the two taps score cached branch-survival groups with high retention (heldout retention 0.9825, false-prune 0.0175)
fresh-dataset comparison v1	`TWO_TAP_MATCHES_OR_BEATS_OLD`	on fresh same-content domains, new two-tap ≈ beats old (0.8818 vs 0.8800; 192 tasks / 742 candidates / 550 pairs)
HH-RLHF comparison v1	`TWO_TAP_MATCHES_OR_BEATS_ON_HH`	on 512 HH pairs, modestly beats old (0.6152 vs 0.5977)
layer-native two-tap v1	`DOMAIN_READY_BRANCH_GAP`	native 24/36/47 taps preserve domain behavior but retain branch gaps
branch-gap repair v1	`DUALANCHOR_BRANCH_GAP_REDUCED`	repair narrows the gap without a clean fixed-bundle pass
architecture-looped v3	`READY_WITH_TERMINAL_DEFER`	all-loop repeated branch/prune survival works; terminal forced commitment is the weak point (magnitudes later withdrawn, E.3)

Architecture-looped v3 diagnostic (48 tasks; 24 reasoning / 24 science; 3,454 rows): stage oracle retention **0.9697** (task-disjoint, zero crossing task IDs; supersedes a contaminated 0.9848 measured with 8 crossing IDs and 26/48 training-side tasks), terminal oracle retained **1.0000**, scaffold top-4 retention **1.0000** (52 groups / 26 tasks). Terminal-selection figures from the contaminated evaluation (forced top-1 oracle 0.9167, reward-diverse 0.6364, rewards 0.2625/0.3167) are **withdrawn**; the clean remainder has only 2 reward-diverse tasks of 9. An L47 ablation confirms earlier-loop L47 perturbation is load-bearing, not diagnostic-only: disabling it collapses oracle retention from 1.0000 to 0.0417. The locked policy is confidence-gated top-1 with defer / top-k handoff rather than unconditional forced top-1 — the survival-without-terminal-selection pattern of §6.

E.2.2 CoreContent — genesis and construction. CoreContent is the content / terminal-selection family: it ranks candidates *within* the handed-off survivor set, using the same digest-and-compare engine as the Section 3 preference evaluator (frozen forward, mean-pooled loop states, antisymmetric linear tap over layernorm($\text{state}_i - \text{state}_j$)), generalized from HH pairs to five domains. Its construction is the inverse of DualAnchor's — data-driven, not transplant:

- **v1 (tap crafting):** every crafted content tap *lost* to the broad-objective baseline `mixedhead_MIX_HH_OBJECTIVE`. Diagnosed cause: starved per-domain data (reward-diverse groups: coding 30, reasoning 5, math 66, logic 80, alignment 200).
- **v2 (dataset expansion + refit, 2026-06-04/06):** the starved domains were expanded 27–520× (reward-diverse groups: coding 1,733, reasoning 2,600, math 3,200, logic 2,199, alignment ~25,993; sources include MBPP/APPS/HumanEval, GSM8K/Hendrycks/SVAMP, LogiQA, ARC/OpenBookQA/CommonsenseQA/ StrategyQA, HH / UltraFeedback / SHP / PKU), features re-extracted (64 shards, 4.87 GB, leakage found and fixed), and the same small taps refit. A crafted tap then beat the baseline on untouched held-out data:

Corrected (task-disjoint) result — the reportable figure. Under a deterministic task-disjoint split (seed 20260711; 23,054 train / 2,948 validation / 2,831 held-out tasks; **zero** task IDs crossing splits):

Quantity	Value
Corrected held-out macro top-1	**0.6310**
Broad-objective baseline (`mixedhead_MIX_HH_OBJECTIVE`)	0.5525
Historical stored-split figure (superseded)	0.6691
Task IDs crossing splits, stored / corrected	195 / 0

Corrected per-domain held-out top-1: coding **0.8956** (n=182), math **0.6409** (298), reasoning **0.5985** (269), alignment **0.6143** (2,564), logic **0.4057** (212).

Important scope note on the corrected coding figure. The corrected coding value (0.8956) is **mutant-only**: every corrected held-out coding group contains a canonical solution plus deterministic mutants, and none contains wrong-problem candidates. A corrected task-disjoint *relevance* evaluation has **not been run** — the relevance negatives were generated inside the old stored splits and were not regenerated after task-disjoint reassignment (evaluating the corrected policy against that stale cache gives 0.5165, but every corrected held-out target draws at least one donor from outside its split, so the number is not publication-clean). The 0.8956 must therefore **not** be read against the historical 0.583 as a matched comparison.

Historical stored-split diagnostics (superseded; retained for the qualitative finding). The following per-domain table was measured on the contaminated stored split and is reported only because it establishes a qualitative property that the corrected run does not re-measure:

Domain / negative type	CoreContent v2	Mixed baseline	Δ
coding constructed mutants	0.935	0.613	+0.32
coding real wrong-problem relevance	0.583	0.502	+0.08
math constructed perturbations	0.657	0.586	+0.07
logic real distractors	0.460	0.353	+0.11
reasoning real distractors	0.681	0.677	+0.00
alignment real preference pairs	0.612	0.534	+0.08

Honest limitations. On the stored split, the macro headline was roughly *half* a constructed-negative artifact: restricted to real-negative domains (reasoning/logic/alignment) the edge over the baseline was **+0.063**, with CIs barely disjoint. The coding tap learned *corruption detection*, not prompt relevance: ~0.94 against syntactic/semantic mutants but **0.58** against real, compiling solutions to *other* problems, and retraining it *with* wrong-problem negatives did **not** close the gap (0.593 → 0.583), indicating prompt-code relevance is not linearly accessible in the pooled features — an intrinsic ceiling, not a data artifact. This corruption-detector-not-relevance-judge characterization is the qualitative finding we retain; the corrected split does not refute it, but neither does it currently re-quantify it, and we say so rather than implying otherwise. Finally, layer 47 proved **dead weight for this role**: a 2-channel 24+36 tap equalled or beat the 3-channel tap and was slightly better on real negatives, so the locked content selector is `CoreContent_v2_blockwise_pruned_24_36`. At branch-selection time the realistic candidate pool resembles the wrong-problem regime more than the mutant regime, so the mutant-heavy figures should be read as an upper bound on deployed selection quality.

Non-looped control (Appendix E.2.2b). The identical CoreContent protocol on MiniCPM-2B-sft-bf16 (40 distinct blocks, no loops; revision `4ec16344…`, weight SHA-256 `0b0c993a…`; layers 24/36, mask-valid mean pooling, task-disjoint split, 0 crossing IDs) gives held-out macro top-1 **0.5680** and macro pairwise **0.7237** (per-domain top-1: coding 0.9121, alignment 0.6927, reasoning 0.5204, math 0.3893, logic 0.3255). Establishes: readable candidate-quality structure does not require a looped architecture. Does not establish: any causal attribution of the Ouro-vs-control gap to looping (the models differ in family, width, corpus, objective, and tap geometry). See §4.7.

E.2.3 S3B2 (generated-branch correctness refit). L2 logistic AUROC **0.7515** / pairwise **0.6835**; expanded hidden-ridge AUROC **0.7755** / pairwise **0.7338**; metadata-only controls weak; high-margin abstention non-rescuing. Selection: **5/8 = 0.625** on N = 8 oracle-present task groups (full pool 160 candidates / 16 groups). The **0.5833** figure sometimes cited as the matched baseline is *not* a matched control — it is the S3B1 three-domain macro $(0.5+0.75+0.5)/3$; a pool-weighted matched-random over the same eight groups (correct counts 7,1,3,2,4,2,8,2 out of ten candidates each) gives matched random **0.3625**, with exact Poisson-binomial $P(\geq 5) = 0.087$ — so the selector exceeds the random point expectation but N=8 is underpowered and reliable selection is not established *on this slice* (§6.4). This slice is superseded for the terminal-selection question by the powered evaluation of E.3.1 below, which uses a different pool; the S3B2 detection figures are unaffected and stand. Supporting: `artifacts/reports/paper_verification/pending_items_resolution_20260703_214531.*`, `dualanchor-architecture-baseline.md`, `dualanchor-tap-evolution.md`, `content-selection-taps.md`, `corecontent-dataset-expansion-v2.md`, `core-domain-tap-audit.md`, `terminal-selection-and-arbiters.md`, `current-state.md`.

E.3 Terminal selection and the final arbiter

Terminal selection — choosing one final output from the survivor set — was the persistent weak point, and several arbiter designs were tried before the program settled on defer rather than a solved selector. The lineage and its verdicts:

Stage	Status	Interpretation
selection-only prototype v1	`SURVIVAL_READY_FINAL_ARBITER_WEAK`	good branches survived; final selection lagged best survivor
final arbiter v1 (`listwise_softmax`)	`FINAL_ARBITER_WEAK_BUT_USEFUL`	beat majority and fixed-top-1, missed the 0.75 target
final arbiter v1.1 (`tie_aware_rank_listwise`)	`NO_IMPROVEMENT`	rank-heavy / tie-aware training did not clear readiness
merged weight taps v1	`FINAL_ARBITER_IMPROVES_ONLY`	weight-space merged tap helped the final choice but did not replace the selector
merged tap integration v1.1	`DOMAIN_FALLBACK_USEFUL_BUT_REASONING_LIMITED`	domain-gated fallback improved CV macro; reasoning remained the blocker
DualAnchor architecture-looped v3	`READY_WITH_TERMINAL_DEFER`	survival ready; terminal confidence weak on the hard slice

Status of the quantitative figures. The arbiter lineage's *verdicts* stand — every terminal arbiter closed weak or `NO_IMPROVEMENT`, and the program's locked policy is confidence-gated top-1 with defer. Its *numbers* do not. The selection-only prototype figures (top-4 retention 0.9514, best-selected reward 0.9453, final reward 0.6672), the architecture-looped terminal figures (forced top-1 oracle 0.9167, reward 0.2625 against best 0.3167, reward-diverse 0.6364), and the integrated comparison (reported as CoreContent pairwise 0.6584 against DualAnchor forced top-1 0.3787) were all measured on splits with task IDs crossing the train/held-out boundary, and the integrated comparison additionally mislabeled a macro top-1 as a pairwise accuracy and used CoreContent candidate groups rather than real branch-survivor pools. All are **withdrawn**. A zero-crossing re-run (2026-07-13) verifies survival (stage retention 0.9697, terminal 1.0000, scaffold top-4 1.0000) but leaves only 2 reward-diverse tasks of 9 — too few to quantify terminal selection in either direction. On that remainder, forced top-1 oracle retention is 0.8889, forced top-1 reward ≈0.0000, best terminal reward 0.0222; on the clean actual-survivor subset CoreContent top-1 is 0.7778 against DualAnchor forced top-1 0.8889, a descriptive reversal that is itself underpowered. We report no terminal-selection magnitude.

The confidence work concluded `TERMINAL_WEAK_ON_HARD_SLICE` / `TERMINAL_CONFIDENCE_ONLY`: aggregate terminal top-1 looks strong only because many tasks are tie-heavy; on reward-diverse and positive-plus-reward-diverse slices, unconditional top-1 is not reliable. The locked rule at terminal `L4_47` is therefore: (1) score terminal candidates pairwise with both DualAnchor taps; (2) evaluate forced top-1 as diagnostic only; (3) collapse only if the confidence gate fires; (4) otherwise keep or defer the terminal survivors. No arbiter in this line is a steering module. The integrated terminal test that had been reported here (CoreContent-v2 0.6584 against DualAnchor forced-top-1 0.3787) is withdrawn for the reasons above and did not replicate on clean survivors. The composed pipeline (DualAnchor survival → CoreContent ranking → survivor handoff) remains the system's architecture; what is no longer claimed is a quantified advantage for either component at the terminal stage. Supporting: `terminal-selection-and-arbiters.md`, `history/bg-run-notes/terminal-arbiters/*`, `history/bg-run-notes/survival-selection/bg_selection_only_phase2_prototype_v1.md`, `branch-training-logic-expansion.md`, `current-state.md`.

E.3.1 The powered terminal-selection evaluation (§6.5). None of the withdrawn figures above is rehabilitated by this run, which uses a different pool and a domain-matched selector. Pool: the Horizon Logic main generation (170 task groups × 4 candidates), split 80 / 31 / 59 by deterministic task hash. Of the 59 held-out groups, 15 are all-correct and 5 all-wrong and are excluded from the forced-choice endpoint, leaving **39 informative groups**. Malformed candidates are retained and scored as incorrect (unlike the pre-answer analysis of §5.3), because non-commitment is a genuine failure mode under forced terminal choice. Feature: full-candidate ("terminal") pooled hidden state over prompt plus complete generated text, one additional forward per candidate through the canonical extractor. Primary selector: standardize → PCA → L2-logistic (`k_pca` 16, `l2` 4.0, grouped-CV AUROC 0.7184 on train+val only). Control selector: shortcut-only (generated-token count, found-final-marker, hit-max-tokens, pre/generated token ratio), same fit family, no hidden state.

§6.5 gives the endpoint. The secondary metrics it omits: pairwise ranking accuracy 0.7623 for the hidden selector against 0.7951 for the shortcut control (122 pairs each); MRR 0.9274 against 0.9359; top-2 oracle retention 0.9487 for both; top-3 retention 1.0000 against 0.9487; and candidate-level AUROC over all held-out candidates with malformed included, 0.7873. The matched-random null is exact rather than nominal: each group contributes its own $p_i = c_i/n_i$, and the null is the Poisson-binomial over those 39 probabilities, stored per group in the run artifact.

Controls: zero task crossing between selector training and evaluated groups; 0 duplicate groups or candidates; frozen checkpoint unchanged throughout; shuffled-score control at paired difference exactly 0.000; candidate-order invariance with 0 failures on a 20-group spot check; no held-out-label tuning (grouped CV on train+val only); exact per-group matched-random probabilities stored; per-group scores deterministically re-derivable from the saved coefficients and the preserved terminal feature pool. Sacrifice logged: the frozen S3B2 / DualAnchor / CoreContent selectors were **not** re-scored under Horizon Logic's distribution, so this run says nothing about their out-of-domain behavior.

Appendix F — Branch/carry/prune and KV-cache implementation

Cache structure. `OuroForCausalLM`, bf16, eager attention; `total_ut_steps = 4`, `num_hidden_layers = 48` → **192 distinct cache slots**, with `cache_slot = current_ut * num_hidden_layers + layer_idx` (slot audit: `SLOT_MAPPING_CONFIRMED`). Each (loop, layer) owns a distinct KV slot; prefill populates all 192; a decode step appends one token to every slot; `reorder_cache` permutes the batch dimension of every populated slot.

Why this is not prompt-only carry. Prompt-only layer carry (used by the offline probes, which run `use_cache=False`) is a strictly easier problem. Generation-time branch-specific carry requires each branch to maintain its own `past_key_values`, `cache_position`, `attention_mask`, `position_ids`, `generated_ids`, and lineage, aligned across every autoregressive decode step.

Equivalence standard. Prefill is **bit-exact** (RMS 0). Cached decode shows small bf16 drift (RMS ≈ 0.05–0.2, max-abs < 1.0) from cached-versus-recomputed key/value paths; top-1 token sequences match except at model-intrinsic argmax near-ties. The splice (below) is bit-exact end-to-end.

v1 validation ladder (levels are labelled V0–V6 to keep them distinct from the L1–L4 loop indices used elsewhere) (`autoregressive_kv_branch_carry_v1`, 2026-06-01; status `PROMPT_INTERNAL_BRANCH_CACHE_VALID`):

Level	Result
V0 cached decode	matches full recompute (prefill bit-exact; decode within bf16 drift)
V1 token-boundary fork	K = 2/4/8 independent branch caches; no cross-branch contamination
V2 batched branches	batched ≡ independent ≡ full recompute
V3 prune / reorder survivors	8→4→2, 8→3, 4→1; lineage aligned
V4 current-token perturb	layers 24/36/47, loop-targeted; carries via branch cache
V5 prompt-internal perturb	branch-specific cache required; **negative control** RMS ≈ 3.0 without it
V6 partial splice (v1)	slot-boundary logic valid but **diagnostic only — no compute saving**

Supporting: `PADDING_MASK_SAFE` (left-padded batched decode requires explicit per-row `position_ids`), DualAnchor integration smoke `BRANCH_PRUNE_CARRY_SMOKE_VALID`, failure analysis `NO_MAJOR_FAILURES`.

v2 suffix-recompute splice (`partial_cache_splice_v2`, 2026-06-04; status `PARTIAL_SPLICE_COMPUTE_SAVING_VALID`, upgrading v1's V6 from diagnostic-only):

- *Obstacle.* The KV cache stores K/V but **not** the inter-layer residual stream, so a perturbed branch cache naively forces a full re-prefill.
- *Solution.* During a minimal shared-prefix prefill, additionally capture the residual hidden at the perturbation boundary (output of loop u, layer ℓ). For an additive boundary perturbation, reconstruct $H^{\text{perturbed}}_{\text{boundary}} = H_{\text{boundary}} + \delta$ with **no forward pass**, and recompute only the suffix (loop u, layers $\ell + 1..$; then loops $u + 1..$). Implemented as test-only orchestration over `model.model.layers / rotary_emb / norm / lm_head` — no weight edits, no permanent model surgery.

- *Hook timing (empirical).* Perturbing a layer *output* leaves that layer's boundary slot unaffected; the first affected slot is $(u,\ \ell+1)$, and the changed set equals the `downstream_only` prediction. Over-sharing an affected slot diverges (negative control).
- *Equivalence.* The spliced branch cache is **bit-exact** versus a full perturbed-prompt reference: all 192 slots, prefill logits RMS 0, continuation bit-for-bit. Validated single-branch, multi-branch (K = 2/4, independent storage, no contamination), batched + prune/reorder, and left-padded.
- *Measured compute saving* (baseline = K full perturbed prefills; splice = one shared prefill + K suffix recomputes), per-branch layer passes saved: loop 0 **13%**, loop 1 **38%**, loop 2 **63%**, loop 3 **88%**. K-scaling at (loop 2, layer 24): K=2 **32%**, K=4 **47%**, K=8 **55%**. Amortized over $K \geq 2$.

What cannot be claimed (recorded in the source notes and honored here): production readiness; steering of any kind; any claim beyond the validated levels. The substrate is a correctness result, not a capability result.

Supporting: `kv-cache-branch-carry.md`, `bg_autoregressive_kv_branch_carry_v1.md`, `autoregressive_kv_branch_carry_validation_note.md`, `bg_partial_cache_splice_v2.md`, `branch-generation-and-survival.md`, `phase1-controller-and-routing.md`.

Appendix G — S1 frozen-fork / K-matched sampling deconfound

Protocol. A branch is forked at a chosen loop/layer boundary, carried forward through its own KV cache (Appendix F), and allowed to generate an independent continuation. The question is whether the *injection* adds reachability — i.e. reaches correct answers the unforked baseline does not — or whether any apparent gain is attributable to the sampling randomness the fork happens to introduce.

Deconfound. The comparison is against **K-matched plain sampling**: drawing the same number of ordinary temperature samples and taking the oracle over them. If matched sampling reaches an equal or higher oracle, the fork adds nothing.

Parameter	Value
Tasks	4
Plain samples per task	12
Temperature	0.7
Top-p	0.95
Sampling budget	96 tokens (long reference: 160)

Results.

Condition	Oracle
Plain sampling (K-matched)	**0.750**
Sampled fork	**0.611**
Fork − sampling	**−0.139**
Greedy (deterministic) fork, new-correct answers	**0**

The sampled fork lands *below* matched sampling, and deterministic forking produces no new correct answers at all. Verdict: `FROZEN_FORK_CLOSED__SAMPLING_EXPLAINS_SCREEN__S3_IS_LEVER` — frozen branch injection adds no demonstrated reachability beyond matched stochastic decoding in this bounded test.

Scope. Four tasks is a small screen; the load-bearing claim is the *direction* of the deconfound (fork does not exceed matched sampling, greedy fork adds nothing), not the precise decimal. Supporting: S1 frozen-fork artifacts (2026-06-17), `artifacts/reports/paper_verification/pending_items_resolution_20260703_214531.*`, `current-state.md`.

Appendix H — Steering / adapters closure

Validated write surface. Before any steering claim: decoder-layer hooks are mechanically clean. Zero-magnitude writes reproduce no-hook generation exactly; perturbation size scales predictably; perturbations propagate to later hidden states and to logits (surviving ~32 tokens); no CUDA/NaN/Inf instability inside the safe envelope. Safe-magnitude envelope: $\alpha \leq 0.02$ effective RMS.

Seven methods, all unsigned.

Method	Verdict
Raw readout direction (NoNorm)	`UNSIGNED_EFFECT`
Empirical success-mean difference	`EMPIRICAL_UNSIGNED_ONLY`

Method	Verdict
RMS-calibrated static direction	`RMS_UNSIGNED_ONLY`
Local outcome-score gradient probe	`GRADIENT_NO_BETTER_THAN_RANDOM`
Classifier-derived adapter	no reliable held-out control
Teacher-forced causal adapter	`ADAPTER_IMPROVES_LOGIT_MARGIN` (teacher-forced) → `TEACHER_FORCED_ONLY` → `LOCAL_LOGIT_CONTROL_ONLY`
Sequence-level (REINFORCE) adapter	`SEQUENCE_REWARD_IMPROVES` (training) → `NO_ADAPTER_SPECIFIC_TRANSFER` (held-out) → **`WORSE_THAN_RANDOM`**

Summary verdicts: `BG_SEQUENCE_LEVEL_ADAPTER_VERDICT = NO_FROZEN_BACKBONE_WRITE_PATH`; `FROZEN_BACKBONE_INFERENCE_STEERING_STATUS = CLOSED_UNDER_TESTED_METHODS`.

Geometry. Readout geometry ≠ empirical-success geometry ≠ local logit-control geometry:

Direction pair	Cosine
Adapter proxy vs. raw NoNorm readout	−0.0005530
Adapter proxy vs. empirical success-mean difference	−0.0042941
Raw NoNorm readout vs. empirical success-mean difference	+0.1010016
Sequence-level adapter vs. teacher-forced adapter (independent training runs)	**+0.9510944**

The two independently trained adapters converge on nearly the same direction while both remaining nearly orthogonal to the readout and empirical-success directions — consistent with both descending into the same non-steering local-control basin rather than discovering the outcome direction.

Scope. The closure holds under the tested safe-α envelope and tested optimizers. It does not prove that no training method can steer Ouro; it closes the simple frozen-backbone inference-time path and motivates the training-time route of §12. What was explicitly *not* established: reliable action steering, a production write path, or a trained steering corridor.

Supporting: `steering-and-adapters.md`, `artifacts/reports/probes/bg_steering_suite_2026-05-18/summary.md`, `artifacts/reports/probes/bg_steering_suite_2026-05-18/analysis.md`, `bg_causal_intervention_adapter_2026-05-18`, `current-state.md`.

Appendix I — Strict pre-answer audit details

I.1 GSM8K (§5.2).

Data. GSM8K (Cobbe et al., 2021): **170 tasks**, 4 sampled solutions each, **680 examples**; 407 positive / 273 negative by external checker. Raw per-example features, predictions, and labels are preserved at `artifacts/reports/proto_introspection/within_domain_recapture.pt`, so no regeneration is required to re-audit the cut.

The strict pre-answer cut. Enforced in the extraction code, not by post-hoc filtering. Excluded by construction:

1. **Answer region** — all tokens from the point the solution begins committing its final numeric answer onward. Loop states over this span never enter the feature vector.
2. **Gold value** — the reference answer is not present in the extraction path in any form. The probe has no channel through which to match against it.
3. **Correctness label** — produced by an external checker after generation; used solely as the probe's training target, never as an input feature.

The probe therefore sees only the loop trajectory of a computation that has not yet produced an answer.

Features and controls. Hidden features are pooled loop states over the pre-answer span. Two shortcut controls are computed on the same span: solution **length** and token **log-probability**. The composite control is length + logprob.

Results. Per-feature AUROC: hidden 0.745 [0.707, 0.783]; length 0.687; logprob 0.569; length + logprob **0.731**; hidden + all **0.797**. Incremental gain **+0.066** (0.065790 exactly).

Interval estimation. Because the 680 examples are nested within 170 tasks, an i.i.d. bootstrap over examples is anti-conservative. The reported interval is a **paired task-clustered bootstrap**: each of 10,000 draws (seed 20260710) resamples 170 task IDs with replacement, retains all four candidates per sampled task, computes both AUROCs on that draw, and records their difference. Result: 95% percentile CI **[+0.021, +0.112]**, bootstrap mean +0.0658, SD 0.0235, zero one-class draws — **excludes zero**. A candidate-level bootstrap gives the narrower [+0.032, +0.100] and is reported only as a diagnostic. Leave-one-task-out re-estimation moves the increment within [+0.056, +0.071] (max change 0.0098), so no single task drives the result.

Provenance note. The original June interval ([+0.017, +0.114]) was described in its report as a task/group bootstrap, but the paired clustered-delta routine was not preserved and the original draws were not saved, so its execution path is not code-auditable. The interval above was recomputed from the preserved raw predictions and supersedes it. Supporting: `artifacts/reports/paper_verification/fable_hardening_checks_20260710_175017/preanswer_task_clustered_ci.json`.

I.2 Horizon Logic (§5.3). Task source: the project's synthetic propositional-entailment set, category `synthetic_propositional`, proof depth 2–4, with a deterministic truth-table verifier. Prompt: the existing multiple-choice template, options mapped to letters, requiring an explicit `FINAL ANSWER:` commitment line. Generation: 170 tasks × k = 4, `max_new_tokens = 448`, temperature 0.7, top-p 0.95, seed 20260724; 680 candidates in 4,368.9 s. A 24-task pilot at `max_new_tokens = 320` revealed a 39.6% malformed rate concentrated at proof depth 3 (61% versus 27% at depth 2) and motivated the one permitted generation-config adjustment; the pilot's own tasks are retained as a diagnostic-only artifact and are **excluded** from every reported number, so that a single frozen token budget covers the analysis pool.

Strict cut. Text strictly before the first case-insensitive match of `FINAL\s*ANSWE`, computed at the text level and re-tokenized through the canonical extractor — the same convention as the GSM8K recapture. Enforced checks: 0 answer-region exclusion violations (no scorable candidate has its whole answer region swallowed); the gold letter and gold text never enter any prompt or feature-extraction string, and are used only post hoc for labelling; mean pre-answer length 149.0 tokens; 622 of 680 candidates contain a marker.

Features and shortcuts. Canonical pooled `[3,4,2048]` hidden state (layers {24, 36, 47} × loops {L1–L4}) at the pre-answer cut. Shortcuts: pre-cut generated token count, mean pre-cut token log-probability, minimum pre-cut token log-probability, hit-max-tokens indicator. No early-layer secondary variant was evaluated — the canonical extractor supports only {24, 36, 47} — and that omission is logged rather than silently dropped.

Composition. 680 candidates; 454 raw successes; 170 malformed (25.0%); 212 hit max tokens (31.2%); depth histogram {2: 460, 3: 192, 4: 28}; 510 scorable (75.0%) at a scorable success rate of 0.8902. Task-level split by deterministic `sha256(task_uid)`: 80 / 31 / 59, zero crossings. Held-out: 171 scorable candidates across 56 tasks, base rate 0.8889 — roughly 19 negatives.

Hyperparameters and secondary quantities. §5.3 gives the AUROCs and the increment. Selected by task-grouped 5-fold CV on train+val only: hidden (`k_pca` 24, `l2` 4.0, CV AUROC 0.5913), shortcut (`k_pca` 16, `l2` 2.0, CV 0.6108), combined (`k_pca` 24, `l2` 2.0, CV 0.5915). Bootstrap mean of the increment +0.1395 over 2,000 rounds; accuracy at threshold 0.5 for the combined model 0.8830.

Controls beyond those stated in §5.3. Split membership is a pure function of the task hash, so permuting task IDs cannot change it; 0 duplicate (task, candidate) pairs; the held-out split is deterministic and was opened once; raw held-out predictions are preserved with task IDs, labels, and all three score vectors. The two malformed-artifact controls are numbered 11 and 12 in the run record and are reported in full in §5.3.

V3 power extension (run 2026-07-26). Pre-registered plan sealed before generation (`horizon_power_v3_20260726/EXPERIMENT_PLAN.md`, including the fixed verdict labels and the commitment to report a null as a null). Three shards of 170 tasks at offsets 170/340/510 of the same hash-ordered pool; the generation wrapper changes only the output directory of the sealed generator and refuses any offset overlapping the original slice. Analysis gate: the published cohort's numbers must reproduce to 10^{-9} from the sealed shards before any new data is touched (passed at machine zero). Extension composition: 2,040 candidates, malformed rate 24.7%, 1,533 scorable; pooled held-out 614 scorable across 201 tasks, 84 negatives. Verdict: `REPLICATED_AND_ROBUST_TO_MALFORMED_SIBLING_SHORTCUT`. Endpoints: extension-only +0.0953 [+0.0376, +0.1569] (4-shortcut) and +0.0910 [+0.0345, +0.1531] (adversarial 5-shortcut incl. malformed-sibling count); pooled +0.1114 [+0.0563, +0.1689] and +0.1074 [+0.0528, +0.1643]; shuffled-label 0.5415; LOTO max influence 0.0081; v2/v3 task sets disjoint by construction and asserted. Artifacts: `horizon_power_v3_20260726/{auroc_results_v3.json, raw_predictions_heldout_v3.json, SHA256SUMS}`.

Thinking-checkpoint attempt (run 2026-07-26). Sealed before generation (`thinking_preanswer_v3_20260726/EXPERIMENT_PLAN.md`, with script hashes, fixed verdict labels, and the explicit commitment to keep the 448-token budget regardless of observed class balance; one recorded infrastructure amendment — a cache-API compatibility shim in a local copy of the upstream modeling code, weights untouched). Same task slice as the RLTT main run (identity asserted), same prompts, sampling, seeds, and strict cut; candidates generated by Ouro-2.6B-Thinking (pinned f1edd81). Composition: 680 candidates, malformed rate 0.4559 (RLTT reference 0.2500) — the Thinking variant's longer reasoning truncates at the sealed budget — 370 scorable, 130 held-out, ≈20 negatives. Verdict: `THINKING_PREANSWER_NULL`; within-Thinking increments +0.0168 [−0.1440, +0.2061] (4-shortcut) and +0.0273 [−0.1285, +0.2106] (adversarial 5-shortcut). The sealed label is retained verbatim for provenance, but "null" overstates the statistical conclusion: interpretation bound at seal time, and maintained here, is that intervals this wide neither replicate nor exclude the RLTT-sized effect — the estimate is inconclusive and underpowered, and the paper's status for this result is *unresolved* everywhere it is cited. Artifacts: `thinking_preanswer_v3_20260726/{thinking_preanswer_results.json, raw_predictions_heldout.json}`.

Powered Thinking replication (run 2026-07-27/28). Sealed before generation (`thinking_preanswer_power_v5_20260727/EXPERIMENT_PLAN.md`), with the sample size fixed by a formal simulation rather than a heuristic: 500 replicate draws per cell resampling the pilot's own held-out predictions through the exact task-clustered estimator, giving power 0.816 at N = 900 for an assumed true increment of +0.095 — the pre-registered power target, which is the RLTT new-cohort increment under the published four-shortcut composite; the matched adversarial five-shortcut figure is +0.091. The observed interval includes +0.095, so the run does not formally exclude an RLTT-sized effect. The plan fixed the ±0.05 smallest effect of interest on operational grounds (the smallest increment that would change the cross-checkpoint conclusion), the four-way verdict order `NEGATIVE_TRANSFER` → `POSITIVE_REPLICATION` → `PRACTICAL_EQUIVALENCE` → `UNRESOLVED`, the malformedness controls, and a recorded warning that the ±0.05 margin was unreachable at any feasible N — so a true null would land `UNRESOLVED` rather than demonstrate equivalence. Protocol matched the pilot exactly, including the 448-token budget; slice offsets [1500, 2400), disjoint from every prior run. Analysis ran once, gated on reproducing the pilot's published increments to 10^{-9} (passed, max $|d| = 0$). Composition: 900 tasks, 3,600 candidates, malformed rate 0.434 (pilot 0.456), 2,037 scorable, 544 held-out scorable, **99 held-out negatives**. Primary (new-only, adversarial five-shortcut composite): **+0.0423, 95% CI [−0.0166, +0.1038]**, 90% CI [−0.0083, +0.0932]; verdict **`UNRESOLVED`**, `PRACTICALLY_SMALL` false, equivalence condition not held. Secondary pooled precision estimate — never described as independent replication — +0.0503 [−0.0044, +0.1059]. Sealed controls: dropping the 48 held-out tasks that are ≥75% malformed *raises* the increment to +0.0520; the hidden-only malformed-vs-clean AUROC is 0.8406, higher than the 0.713 seen on RLTT-Horizon, so correctness and malformedness are less separable on this checkpoint and we do not claim otherwise. Artifacts: `thinking_preanswer_power_v5_20260727/{EXPERIMENT_PLAN.md, sizing_simulation_v5.json, thinking_power_results.json, raw_predictions_heldout_v5.json}`.

Appendix J — Orthogonality / null audit

The hypothesis under test. If the frozen branch mechanism can only write into a subspace U_{inj}, and the verifier-outcome direction d_{out} lies largely outside it, the frozen-control pattern would have a simple geometric explanation. The rank-corrected audit does not support that one-dimensional span-misalignment account; it enters the paper as a tested but unsupported explanation, never as proof of either that account or its converse.

Setup. Exact-protocol regenerated S1/S3 injection/carry deltas (regenerated from the protocol, not replayed from historical tensors — a caveat we preserve). Ambient dimension $D = 24{,}576$; injection-span rank $k = 344$; participation ratio ≈ 5.06, so the span is effectively far lower-dimensional than its nominal rank.

Quantity	Value
Observed projection $\pi_{\mathrm{inj}}(d_{\mathrm{out}})$	**0.018296**
Random-direction baseline k/D	0.013997
Ratio observed / random	**1.307**×
Percentile vs. random-direction null (10^6 draws)	99.9907 ($p_{\mathrm{right}} = 9.3 \times 10^{-5}$)
Shuffled-label null, global mean (10^4 draws)	0.022990 (observed at percentile 3.44)
Shuffled-label null, domain-stratified mean	0.022702 (observed at percentile 0.0)

Why the two nulls disagree. Against *random directions*, the outcome direction projects into the injection span **more** than chance (1.31×, 99.99th percentile) — so the naive reading, "the outcome direction lies unusually outside the writable span," is not supported by this null. Against *outcome-shaped shuffled-label* directions (same correlational structure, no true verifier information), it projects **less** than their mean — pointing weakly the opposite way. The gap between observed and shuffled-mean is ≈ 0.0044 of total energy: small.

Verdict. Ambiguous and fragile. The estimate rests on a single one-dimensional projection from one regenerated delta bundle, and the two nulls point in opposite directions. A separate control confirms the injection span is statistically distinct from the natural sampling span (frozen forking is not merely resampling), but that answers a different question. We therefore do **not** treat subspace misalignment as an explanation of the frozen null.

J.2 The subspace-vs-subspace follow-up, and why it does not close the question. The principal-angle analysis listed as future work in earlier versions has now been run, and its primary question is unanswerable with the data this project holds. *Inputs, read-only:* the frozen cross-loop feature set of §4.4 (hashes verified before and after the audit), and the exact-protocol S1/S3 injection-delta bundle, whose `delta_locus` perturbation vectors exist **only** at loop {1–4} × layer {24, 36, 47} — twelve late loci, 32 samples each. *Constructions:* a readable subspace from a task-resampled bootstrap ensemble (N = 40/locus) of standardize → PCA → L2-logistic directions mapped back to the 2048-dimensional feature space, normalized, stacked, and reduced by SVD; an outcome-relevant subspace from a deliberately different construction, a bootstrap ensemble (N = 100/locus) of standardized between-class mean-difference directions; and a writable subspace from the SVD of the centered observed injection deltas. *Null:* rank-matched random subspaces in the same ambient space, 2,000 draws per rank (rank-1 mean 88.99°, p05 87.53°; rank-3 mean 88.21°, p05 87.44°; rank-5 mean 87.66°, p05 87.06°).

Comparison	Loci	Result
Readable ↔ outcome	L3_8, L3_16, L4_8, L4_16, L4_24, L4_36, L4_47 (+ L1_16, L2_16)	Enriched vs null at **every** locus and rank; rank-1 angles 27.3–37.9°. Tightest multi-rank overlap at the terminal locus **L4_47** (rank-1 27.27°, rank-3 mean 47.83°, rank-5 mean 58.69°) versus 66–69° rank-3 elsewhere. **Not early-enriched.**
Readable / outcome ↔ writable	L4_24, L4_36, L4_47 only	At chance at every rank: readable↔writable mean 87.73–89.38°, outcome↔writable 87.69–89.61°, against null p05 87.06–87.53°. `enriched_vs_null` false in all 18 cells. Verdict `READABLE_OUTCOME_OVERLAP_WITHOUT_WRITABLE_ALIGNMENT`.
Early-vs-late writable alignment	—	**`INSUFFICIENT_MATCHED_LOCUS_WRITABLE_DATA`**: no writable tensor exists at layer 8 or 16 at any loop. Not manufactured, not transported across coordinate systems, and no new intervention campaign was run to create it.
Loop-to-loop readable rotation (layer 16 fixed)	L1→L2, L2→L3, L3→L4	Rank-1 angles 30.89°, 16.63°, 11.16°; rank-3 means 57.09°, 56.84°, 58.25°. The rank-1 angle is largest at L1→L2 and then declines; the rank-3 means are nearly flat. Descriptively consistent with §4.4's frozen-transfer result, but `rotation_beyond_null` is **false** for all three, so this is corroboration in the multi-dimensional geometry, not an independent confirmation.

Sacrifices logged. The label/task-permutation null for the readable and outcome ensembles ran at 25 rounds at one locus (L4_47, rank 3; mean angle 59.54°) rather than the full 2,000, because each round refits a 40–100-member bootstrap ensemble; the primary random-rotation null, which every verdict above is referenced against, ran at full scale. CCA-based geometry was not attempted and rank-10 subspaces were not computed; both were marked optional, and ranks 1/3/5 are reported in full at every locus.

Net position. The one-dimensional audit did not support simple span misalignment; the multi-dimensional audit could not answer the early-locus question for want of matched writable data; the diagnostics it did produce concern readable↔outcome breadth and late-locus readable↔writable non-alignment, neither of which explains the control failure. The geometric mechanism of the readout–control boundary remains unresolved, and closing it requires new writable data at the early loci (§12.4, item 3).

Supporting: `s1_s3_exact_injection_orthogonality_null_audit_2026-06-17.*`, helper `tools/s1_s3_exact_injection_null_audit_2026_06_17.py`; subspace geometry audit (2026-07-24).

Appendix K — Artifact and reproducibility index

Repository state. Reproducibility is pinned to commit `e4776dd41a85cad699ac36f309b5986ab48bd171` of the working repository. Canonical project state: `current-state.md`. The public mirror is `github.com/VykosMolt/Branching-Looped-Transformer`; artifact paths below map onto it by dropping the `artifacts/` prefix, so `artifacts/reports/<run>/` is `results/<run>/` there. Model weights, extracted feature tensors, and datasets are excluded from the mirror, but every published run directory ships the `SHA256SUMS` covering its full contents, so the omitted files remain enumerable and verifiable.

Models.

Role	Identifier	Revision / hash
Base	`ByteDance/Ouro-2.6B`	`1ed04250da1a9936042725d302e81c8fa2ab5abd`
Reasoning-SFT	`ByteDance/Ouro-2.6B-Thinking`	`f1edd81e7ac41355db670500ceaf204e0f73af68`

Role	Identifier	Revision / hash
RL-trained (primary)	Ouro-RLTT research weights provided by Jonathan Williams; locally converted checkpoint at `.../models/ouro_rltt_local`	local shard hashes recorded; upstream revision unavailable. **These weights are not publicly released**, so the RLTT results here cannot be reproduced end-to-end without them; every derived artifact (features, predictions, taps, statistics) is preserved and hash-verified so the analysis layer remains fully auditable.
Non-looped control (§4.7)	`openbmb/MiniCPM-2B-sft-bf16`	rev `4ec16344ac13e6ef5010aeecaa533369ac8eb53c`; weights SHA-256 `0b0c993ace78c5983373948c636b0e587fcf1ac6f2e0f980bf7d735fe7dc52f8`
Evaluator checkpoint	`artifacts/checkpoints/evaluator/pairwise_epoch2.pt`	SHA-256 `3630c2092eca8db13239f763bc9c212f4b673866e47f811c3095efc57409ec96`

Environment. `transformers==4.54.1` (pinned); PyTorch `2.12.0.dev20260407+cu128`; bf16 forward, fp32 features, eager attention; no quantization. Hardware: NVIDIA RTX 5070 Ti Laptop GPU.

Seeds. Pair selection / split: `20260711`. Task-clustered bootstrap (§5.2): `20260710`. Domain-transfer task-disjoint split (§4.5): `20260711`. Cross-loop early-layer subset, training grid, and task-clustered bootstrap (§4.4): `20260720`. Horizon Logic generation, analysis, terminal selection, and geometry audit (§5.3, §6.5, §8.4): `20260724`. Linear-probe reconstruction: `42`. Bootstraps use 10,000 draws throughout except the 2026-07-24 programme, which uses 2,000 rounds for its paired task-clustered intervals and 2,000 draws for the geometry random-rotation null.

Primary artifacts by result.

Result	Artifact
Strict pre-answer GSM8K (§5.2)	`artifacts/reports/proto_introspection/within_domain_recapture.pt` (raw features, predictions, labels); clustered CI in `.../fable_hardening_checks_20260710_175017/preanswer_task_clustered_ci.json`
Cross-loop early-layer localization (§4.4)	`artifacts/reports/cross_loop_early_layer_taps_20260720/` — `train_eval_results.json` (all 14 refit cells and 18 transfers), `bootstrap_stats.json` (task-clustered intervals and paired deltas), `split_integrity.json`, `extraction_controls.json`, `length_baseline.json`, `s3b2_secondary.json`, `features/`, `predictions/`, `code_tests/test_results.json`, `SHA256SUMS` (100 files)
Strict pre-answer Horizon Logic (§5.3)	`artifacts/reports/paper1_v2_overnight_20260724/horizon_logic/` — `auroc_results.json`, `raw_predictions_heldout.json`, `cut_integrity.json`, `split_integrity.json`, `task_manifest.json`, `horizon_generation_main_off0.pt` (+ index), `pilot_diagnostic_only/`
Powered terminal selection (§6.5)	`artifacts/reports/paper1_v2_overnight_20260724/terminal_selection/` — `terminal_results.json` (endpoints, exact per-group `per_group_p`), `group_manifest.json` (per-candidate success/malformed, from which the 26/8/5 decomposition is recomputed), `terminal_pool.pt`
Subspace geometry audit (§8.4, Appendix J.2)	`artifacts/reports/paper1_v2_overnight_20260724/subspace_geometry/` — `geometry_results.json`, `subspace_definitions.json`, `subspace_bases.pt`, `feature_manifest.json`
2026-07-24 programme root	`artifacts/reports/paper1_v2_overnight_20260724/` — `MASTER_RESULTS.md`, `MASTER_RUN_MANIFEST.json`, `SHARED_ARTIFACT_AUDIT.md`, `ENVIRONMENT.json`, `SHA256SUMS` (39 files)
Full 8,552-pair antisymmetry audit (§3.3)	`.../remaining_todos_resolution_20260710_183700/full_hh_audit/full_hh_antisymmetry.{json,csv}`
Powered pair-disjoint probes (§3.2, §3.5)	`.../powered_clean_probe_and_corecontent_20260711/hh_{base,thinking,rltt}_40000_pair_disjoint_results.json`; paired bootstrap `powered_hh_paired_bootstrap.json`; pointwise `hh_thinking_40000_pointwise_pair_disjoint_results.json`

Result	Artifact
CoreContent task-disjoint refit (§4.6)	`.../powered_clean_probe_and_corecontent_20260711/ouro_corecontent_task_disjoint_results.json`
Non-looped control (§4.7)	`.../nonlooped_architecture_control_20260710_235252/` (report + `corecontent_control_results_task_disjoint.json`)
Domain-transfer re-run (§4.5)	`artifacts/reports/probes/paper_v44_task_disjoint_rerun_2026-07-13/tier2_domain_transfer.json`
Branch survival re-run (§6.2)	`.../paper_v44_task_disjoint_rerun_2026-07-13/tier1_branch_survival_selection.json`; integrated survivors `integrated_true_survivors.json`
S3B2 detection/selection (§6.3–6.4)	`.../final_engineering_expansion_2026-06-17/s3b2_generated_branch_correctness_expanded_2026-06-17.md`
KV-cache / splice (§7)	`kv-cache-branch-carry.md`; `bg_autoregressive_kv_branch_carry_v1.md`; `bg_partial_cache_splice_v2.md`
Steering closure (§8.1)	`artifacts/reports/probes/bg_steering_suite_2026-05-18/{summary,analysis}.md`; `bg_causal_intervention_adapter_2026-05-18`
Orthogonality null audit (§8.4)	`s1_s3_exact_injection_orthogonality_null_audit_2026-06-17.*`

Verification tooling. `tools/paper_verification/` — including `powered_hh_pair_disjoint_probe.py`, `powered_hh_pointwise_pair_disjoint_probe.py`, `paired_bootstrap_powered_hh.py`, `rerun_ouro_corecontent_task_disjoint.py`, `nonlooped_transformer_control.py`, and `utilities/tests/manual/rerun_paper_v44_task_disjoint_audits.py`.

Integrity verification of the added runs. Both added experiment roots — `cross_loop_early_layer_taps_20260720` and `paper1_v2_overnight_20260724` — ship a `SHA256SUMS` covering every file in their root; they verify **100/100** and **39/39** respectively at the time of writing. Both were produced against the same pinned commit and model checkpoint as the rest of the paper, and both were opened read-only when their results were integrated. The geometry audit of Appendix J.2 additionally re-verified the cross-loop artifact's hashes before and after running against it.

Known provenance gaps (stated rather than hidden): the original ~84.5% linear-probe weight vector was not preserved (the cross-backbone replication of §3.5 uses a reconstruction); the historical base-24% localization run has no surviving artifact (§3.5, retracted); the original June pre-answer CI's bootstrap code and draws were not preserved, so the interval reported here is a fresh recomputation from the raw predictions (§5.2, Appendix I); the math-transfer origin figures (§3.6) are unarchived and non-load-bearing; and the base-backbone extraction in the powered probe run completed without writing its shard manifest, though shard and row counts are independently verified.

Appendix L — The frozen-conversion programme: sealed designs and full tables

Five experiments, run 2026-07-26/27 after the v3 results were frozen, each with a sealed plan (`EXPERIMENT_PLAN.md` in its run root) fixing endpoints, controls, budgets, and verdict labels before launch. Task slices are non-overlapping offsets of the hash-ordered Horizon pool ([680,860) C2, [860,1080) plus a pre-declared extension [1350,1500) C4, [1080,1260) C3 and C5 training, [1260,1350) C5 evaluation); C1 consumed only preserved out-of-fold predictions and touched no model. Verdicts are quoted verbatim.

C1 — Selective prediction (`selective_prediction_v3_20260726/`). Endpoint: selective accuracy at coverage $c \in \{0.50, \ldots, 1.00\}$ and ΔAUARC (combined vs shortcut ranker), paired task-clustered bootstrap, 2000 rounds.

Arm	n	ΔAUARC	95% CI	At 70% coverage
Horizon pooled, 4-shortcut	614	+0.0124	[+0.0048, +0.0207]	92.6% vs 88.4%
Horizon pooled, adversarial 5-shortcut	614	+0.0119	[+0.0044, +0.0196]	—
Horizon new-cohort-only	443	+0.0093	[+0.0006, +0.0184]	—
GSM8K out-of-fold (hidden vs shortcut)	680	+0.0087	[+0.0003, +0.0181]	—

C2 — Tap-gated loop allocation (`depth_alloc_v3_20260726/`), verdict `ALLOCATION_NOT_SIGNAL_DRIVEN` on both checkpoints. Fixed-depth tables (k=2, 180 tasks/model, offsets [680,860)); policies select rows at exactly matched budget; gate: beat the 95th percentile of 100 random allocations at matched depth histogram. Fixed-depth held-out success — RLTT: 0.38 / 0.84 / 0.86 / 0.80 across d=1..4; Thinking: 0.18 / 0.44 / 0.64 / 0.70. Neither the loop-1 tap allocator nor the native exit gate cleared the random-histogram control at any budget (RLTT tap-vs-gate deltas −0.06 to +0.04, all CIs crossing zero). Note the RLTT $d3 \geq d4$ ordering, consistent with trained-depth non-monotonicity.

C3 — Matched-budget prefix-prune tournament (`tournament_v4_20260727/`), verdict `POOL_CANNOT_DIFFERENTIATE` (sealed feasibility floor: any-correct-of-6 ceiling 0.978 > 0.95). 180 tasks, offsets [1080,1260); B=6, prune to S=2 at P=80 prefix tokens by frozen DualAnchor locked channels; simulated with best-of-3/best-of-4 and a 500-replicate random-prune control over one candidate table at exact token accounting. Recorded internals, no claims attached: tournament 0.706, best-of-3 0.644 (Δ +0.061 [−0.011, +0.128]), best-of-4 0.661, random-prune p95 0.639; prefix-score AUROC 0.616; mean tokens/task 834 / 760 / 1007 (spend guard triggered). Two pre-data amendments recorded in the plan: a best-of-4 bracket + spend guard (added after a synthetic smoke test exposed the early-termination budget asymmetry), and B=6 drawn as two seeded rounds of three (single-batch k=6 exceeds 12 GB).

C4 — All-well-formed terminal selection (`wellformed_terminal_v4_20260727/`), verdict `WF_CONTENT_SELECTION_ESTABLISHED · HIDDEN_MATCHES_SHORTCUT`. Pool: resample until four well-formed candidates per task (form-filtered only; correctness never consulted), 219-task primary cohort plus a 150-task extension sealed *before any selector was fit* (the primary cohort tripped the pre-registered 25-group power floor: candidate-level success among well-formed outputs is 0.856, so most groups are all-correct). Pooled: 369 tasks, 110 held-out groups, 32 informative. Hidden selector 27/32 (0.844) vs matched-random 0.648, paired diff +0.195 [+0.078, +0.305], exact Poisson-binomial $p = 0.0086$; surface-only control 24/32 (0.750); hidden-vs-surface +0.09 [−0.06, +0.25]; candidate-level AUROC 0.805. Per-group detail (sizes, matched-random p_i, both selectors' outcomes) is stored in the results artifact. One primary-cohort task was lost to a transient out-of-memory collision with an unrelated process and is recorded in the run's error list.

C5 — LoRA direction–outcome binding pilot (`lora_s3a_pilot_20260727/`), verdict `BINDING_NOT_DETECTED`. Two adapters (PEFT LoRA r=16, all-linear, ≤400 steps, bs 1 × accum 8): *binding* (hook $+\alpha d$ active while learning verified-correct candidates, $-\alpha d$ for incorrect; §8 injection convention verbatim — curated unit-RMS L24 direction, loop 1, last prompt token, $\alpha = 0.02$) and *control* (coin-flipped signs). Training data: the C3 candidate table. Zero-delta bit-identity gate passed. Evaluation: 90 task-disjoint tasks (offsets [1260,1350)), K=4, paired seeds, six arms.

Arm	Success
binding + inject $+\alpha d$	0.422
binding, clean	0.414
control + inject	0.425
control, clean	0.436
frozen + inject	0.553
frozen, clean	0.508

Δ_{bind} +0.008 [−0.036, +0.050]; Δ_{ctrl} −0.011 [−0.056, +0.036]; difference-of-differences +0.019 [−0.044, +0.081]; frozen injection delta +0.044 [−0.006, +0.100], consistent with §8.1's no-reliable-effect. Side-finding: both adapters cost −0.094 [−0.164, −0.019] of unconditional success against the frozen base — the sign-conditioned objective trains on verified-incorrect text, and the model learns the text more readily than the condition. Scope caveats: one direction (whose bank validation used $\alpha = 0.005$, half an order below the sealed 0.02), one locus, off-policy cross-entropy rather than reinforcement, ≤400 steps.

References

Arditi, Andy, Oscar Obeso, Aaquib Syed, et al. 2024. *Refusal in Language Models Is Mediated by a Single Direction*. https://arxiv.org/abs/2406.11717.

Bai, Yuntao, Andy Jones, Kamal Ndousse, et al. 2022. *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*. https://arxiv.org/abs/2204.05862.

Betley, Jan, Xuchan Bao, Martín Soto, Anna Sztyber-Betley, James Chua, and Owain Evans. 2025. *Tell Me about Yourself: LLMs Are Aware of Their Learned Behaviors*. https://arxiv.org/abs/2501.11120.

Binder, Felix J., James Chua, Tomek Korbak, et al. 2024. *Looking Inward: Language Models Can Learn about Themselves by Introspection*. https://arxiv.org/abs/2410.13787.

Cobbe, Karl, Vineet Kosaraju, Mohammad Bavarian, et al. 2021. *Training Verifiers to Solve Math Word Problems*. https://arxiv.org/abs/2110.14168.

Comşa, Iulia, and Murray Shanahan. 2025. *Does It Make Sense to Speak of Introspection in Large Language Models?* https://arxiv.org/abs/2506.05068.

Conchello Vendrell, Victor, Arnau Padres Masdemont, Niccolò Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. 2026. *Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models*. https://arxiv.org/abs/2605.07721.

Fu, Tingchen, Yupeng Hou, Julian McAuley, and Rui Yan. 2024. *Unlocking Decoding-Time Controllability: Gradient-Free Multi-Objective Alignment with Contrastive Prompts*. https://arxiv.org/abs/2408.05094.

Geiping, Jonas, Sean McLeish, Neel Jain, et al. 2025. *Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach*. https://arxiv.org/abs/2502.05171.

Hao, Shibo, Sainbayar Sukhbaatar, DiJia Su, et al. 2024. *Training Large Language Models to Reason in a Continuous Latent Space*. https://arxiv.org/abs/2412.06769.

Hendrycks, Dan, Collin Burns, Saurav Kadavath, et al. 2021. "Measuring Mathematical Problem Solving with the MATH Dataset." *Advances in Neural Information Processing Systems: Datasets and Benchmarks*. https://arxiv.org/abs/2103.03874.

Kirin, Jan. 2026a. *Relational Preference Encoding in Looped Transformer Internal States*. https://arxiv.org/abs/2604.09870.

Korbak, Tomek, Mikita Balesni, Elizabeth Barnes, et al. 2025. *Chain of Thought Monitorability: A New and Fragile Opportunity for AI Safety*. https://arxiv.org/abs/2507.11473.

Kutasov, Jon, Chloe Loughridge, Yuqi Sun, et al. 2025. *Evaluating Control Protocols for Untrusted AI Agents*. https://arxiv.org/abs/2511.02997.

Kwon, Woosuk, Zhuohan Li, Siyuan Zhuang, et al. 2023. "Efficient Memory Management for Large Language Model Serving with PagedAttention." *Proceedings of the 29th Symposium on Operating Systems Principles*. https://arxiv.org/abs/2309.06180.

Lambert, Nathan, Valentina Pyatkin, Jacob Morrison, et al. 2024. *RewardBench: Evaluating Reward Models for Language Modeling*. https://arxiv.org/abs/2403.13787.

Li, Ji-An, Hua-Dong Xiong, Robert C. Wilson, Marcelo G. Mattar, and Marcus K. Benna. 2025. *Language Models Are Capable of Metacognitive Monitoring and Control of Their Internal Activations*. https://arxiv.org/abs/2505.13763.

Lightman, Hunter, Vineet Kosaraju, Yura Burda, et al. 2023. *Let's Verify Step by Step*. https://arxiv.org/abs/2305.20050.

Lindsey, Jack. 2026. *Emergent Introspective Awareness in Large Language Models*. https://arxiv.org/abs/2601.01828.

Macar, Uzay, Li Yang, Atticus Wang, Peter Wallich, Emmanuel Ameisen, and Jack Lindsey. 2026. *Mechanisms of Introspective Awareness*. https://arxiv.org/abs/2603.21396.

Marks, Samuel, and Max Tegmark. 2023. *The Geometry of Truth: Emergent Linear Structure in Large Language Model Representations of True/False Datasets*. https://arxiv.org/abs/2310.06824.

Miao, Xupeng, Gabriele Oliaro, Zhihao Zhang, et al. 2024. "SpecInfer: Accelerating Generative LLM Serving with Tree-Based Speculative Inference and Verification." *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. https://arxiv.org/abs/2305.09781.

Pearson-Vogel, Theia, Martin Vanek, Raymond Douglas, and Jan Kulveit. 2026. *Latent Introspection: Models Can Detect Prior Concept Injections*. https://arxiv.org/abs/2602.20031.

Saunshi, Nikunj, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. 2025. *Reasoning with Latent Thoughts: On the Power of Looped Transformers*. https://arxiv.org/abs/2502.17416.

Song, Siyuan, Harvey Lederman, Jennifer Hu, and Kyle Mahowald. 2025. *Privileged Self-Access Matters for Introspection in AI*. https://arxiv.org/abs/2508.14802.

Williams, Jonathan, and Esin Tureci. 2026. *Prioritize the Process, Not Just the Outcome: Rewarding Latent Thought Trajectories Improves Reasoning in Looped Language Models*. https://arxiv.org/abs/2602.10520.

Yao, Shunyu, Dian Yu, Jeffrey Zhao, et al. 2023. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2305.10601.

Zheng, Lianmin, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, et al. 2024. *SGLang: Efficient Execution of Structured Language Model Programs*. https://arxiv.org/abs/2312.07104.

Zhu, Rui-Jie, Zixuan Wang, Kai Hua, et al. 2025. *Scaling Latent Reasoning via Looped Language Models*. Project page: https://ouro-llm.github.io. https://arxiv.org/abs/2510.25741.

Zou, Andy, Long Phan, Sarah Chen, et al. 2023. *Representation Engineering: A Top-down Approach to AI Transparency*. https://arxiv.org/abs/2310.01405.